

Verification Continuum™

Verdi® UVM Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

1. Preface	7
Customer Support	7
Synopsys Statement on Inclusivity and Diversity	8
2. Introduction	9
3. Before You Begin	11
Prerequisites	11
Set Environment Variables	12
Compile Designs with the UVM Libraries	12
Compile with UVM	12
Configure the Simulation	13
Import Design and Configure Simulation in Verdi	13
Configure Simulation with UFE Flow	14
Points to Note	14
Perform Interactive Simulation with UVM Aware Debug	15
Behavior of OVM/UVM Hierarchy Tree View at the Startup of the Simulation	18
UVM Debug Panes	18
4. UVM Debug Usage	20
OVM/UVM Hierarchy Tree Pane Usage	20
Searching in the OVM/UVM Hierarchy Tree Pane	22
View Detailed Information	22
View Interface in Source Code	27
View Definition in Source Code	27
View Creation in Source Code	28
View the Connect Function Call in Source Code	28
View the Resource Information and Configuration	28
View the Sequence Data	29
View Original Type	29
View in Object Browser	30

Contents

View in Class Browser	30
Add a Component Object to nWave	31
Limitations of Adding Component Waveform to nWave	33
Interface Mapping Node in the OVM/UVM Hierarchy Tree	34
Limitations of the OVM/UVM Hierarchy Tree Pane	35
Specifying UVM Runtime Options	35
Save/Restore Session Support in UVM Aware Debug	36
Commands to Save/Restore Session for all UVM Views in Interactive Mode ..	36
Register View Save/Restore Session	38
Factory View Save/Restore Session	39
Phase View Save/Restore Session	40
Sequence View Save/Restore Session	41
Resource View Save/Restore Session	41
OVM/UVM Hierarchy View Save/Restore Session	42
Resource View	43
Viewing the Resource View	44
Viewing the Source Code and Accessor	48
Applying Quick Filter in the Resource View	49
Displaying Radix in the Resource View	50
Radix Specifier	51
Setting Radix Value for the Selected Resource Item	52
Factory View	53
Viewing the Factory View	54
Viewing a Class in the Class Browser and Source Code Panes	57
Applying Filter Views in the Factory View	58
Phase View	59
Viewing the Phase View	60
Viewing an Object in Different Panes	63
Setting Breakpoints at a Specific Phase	63
Applying Quick Filter in the Phase View	65
Sequence View	66
Viewing the Sequence View	67
Viewing the Sequence in the Interactive Debug Panes	70
Viewing the Reference of a Sequence	71
Setting Breakpoint at Specific Sequence	73
Adding a Sequence to the Waveform	74
Adding a Sequence to the Transaction and Protocol Analyzer or Transaction Browser	75

Applying Quick Filter View in Sequence View	77
Register View	78
Viewing the Register View	79
Viewing Registers in Different Panes	81
Operations in the Register Attribute Pane	82
Changing Desired Value	82
Setting Breakpoints	82
Setting Radix	84
Adding a Register to the Waveform	84
Applying Quick Filter in the Register Attribute and Access History Panes	85
Searching Registers and Register Field Items in the Register View	85
Viewing Registers and Register Fields in the Register Hierarchy Pane by Default	87
Displaying Warning Messages for Incomplete Register Hierarchy	88
Displaying Register Access Status in the Register View	89
Multitasking Task	90
GUI Operations for Multitasking Update in UVM Debug Windows	90
Applying New Settings for the Current View With multitasking Update Tasks	93
Register View	93
Resource View	94
Phase View	95
Sequence View	96
Tcl Commands for Multitasking Update in UVM Debug Windows	97
Debugging UVM Registers in Verdi Post-Processing Mode	98
Dumping UVM Register Information in Post-Processing Mode	98
Dumping Register Hierarchy and Access History Information	99
Dumping Class Objects for Register View Post-Processing in VCS	99
Simulation Options for Generating FSDB File With Register Information in Post-Processing Mode	99
Invoking Verdi in Post-Processing mode	100
Debugging UVM Register Information in Post-Processing Mode	102
Functions Supported in Register View	102
Right-click Menu Options in the Register Hierarchy and Register Attribute Panes	103
Updating Register Attribute Values According to the Cursor Time or Selected Access History in Post-Processing Mode	104
Synchronizing Time Between nWave and Register View in Post-Processing Mode	105
Adding Register Items to the Waveform	106
Adding Address Alias Table to nWave	108

Contents

Displaying Register Data in Register View by Time Range in Post-Processing Mode	111
Opening FSDB File by Time Range	111
Displaying Register Data by Time Range	111
Save/Restore Session Support for the Register View in Post-Processing Mode	112
Limitations	113
Viewing OVM/UVM Component Waveforms in nWave	113
Dumping UVM Component Waveforms	114
Viewing the UVM Hierarchy Tree	114

5. Practice

6. Known Issues and Limitations	123
UVM Debug Limitations	123
UVM Register View Debug in Post-Processing Mode: Limitations	124
Testbench Hierarchy Limitations	125

1

Preface

The UVM Debug solution in the Verdi Automated Debug Platform is designed to provide you with sufficient information about the UVM verification environment.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

2

Introduction

The Universal Verification Methodology (UVM) is a methodology for functional verification using the SystemVerilog base class library. It allows you to construct class-based verification environments using verification component objects. It also allows you to create stimulus using sequence objects.

The UVM Debug solution in the Verdi Automated Debug Platform is designed to provide you with sufficient information about the UVM verification environment. Information that helps you understand the UVM architecture - class and object structure, resource/configuration access, factory override and creation, phase transition and objection raise/drop, sequence generation and execution - and to make the debug of UVM testbench easier.

The Verdi platform supports debugging of UVM testbench designs as follows:

- View classes and objects in UVM mode in *Class/Object Browser*.
- Set predefined UVM related breakpoints, for example, break at `phase_start()` in the **Breakpoints** form.
- View UVM component instance hierarchy and TLM port connectivities in the OVM/UVM Hierarchy Tree
- View resource/configuration items in the testbench in the *Resource View* pane.
- View the set/get history of a resource/configuration item.
- View all the predefined and user-defined phases/domains and their transitions in the *Phase View* pane.
- Set breakpoints on the important phases or on the phase methods of `uvm_component`.
- View raise/drop of the objections for a certain phase and other details.
- View factory object creation and override histories in the *Factory View* pane.
- View active and executed sequences with start/end time, executing thread, corresponding sequencer and sequencer ID in the *Sequence View* pane.
- Cross-probing between UVM-aware windows and *Object/Class Browser*, source code pane, Breakpoints form, and Watch pane.

- Convenient filtering mechanism to easily locate the information you want.
- View the access operation history (such as read and write) to the register models.

Note:

The main focus of this document is UVM Aware Debug windows under Verdi's interactive mode. Verdi also provides UVM debug features for post-process mode. For Example:

- The OVM/UVM Hierarchy tree also support post-process. This is covered in the [OVM/UVM Hierarchy Tree Pane Usage](#) section.
- The Register View also supports post-process. This is covered in the [Debugging UVM Registers in Verdi Post-Processing Mode](#) section.
- The UVM transactions and sequences can be dumped into FSDB by Verdi Transaction Recorder. It is displayed and analyzed in Verdi Transaction Browser and Analyzer. See *\$VERDI_HOME/doc/Verdi_Transaction_Debug_Platform.pdf* for more details.

3

Before You Begin

Before you begin using the UVM Debug capability, note the following:

- To achieve the full capabilities of Verdi UVM Aware Debug feature windows, you must use the UVM library provided by VCS or Verdi.
- To perform Interactive Debug, you must have access to both Verdi and VCS. And VCS release version must not be newer than the Verdi release version.

If VCS release version is older than the Verdi release version, `VCS_UVM_HOME` should be used to direct VCS to use Verdi UVM library:

```
% setenv VCS_UVM_HOME <Verdi_UVM_library_path>
```

- If you use custom UVM library or third-party UVM library, Verdi instrumentations and additions must be merged in the UVM library. Contact Synopsys support to know the instructions to merge.
- Without Verdi instrumentations and additions, your access to Verdi UVM debug feature is limited.

For example:

- The factory creation history cannot be viewed in the Factory View pane.
- The OVM/UVM Hierarchy Tree is not accessible

The following sections describe how to compile the UVM testbench and how to configure the **Interactive Simulation Debug** mode to start running interactive UVM Debug features.

Prerequisites

The following sections are included:

- [Set Environment Variables](#)
- [Compile Designs with the UVM Libraries](#)
- [Configure the Simulation](#)
- [Points to Note](#)

- [Perform Interactive Simulation with UVM Aware Debug](#)
- [Behavior of OVM/UVM Hierarchy Tree View at the Startup of the Simulation](#)
- [UVM Debug Panes](#)

Set Environment Variables

The following environment variables must be set before compiling the UVM testbenches and invoking the Verdi platform.

```
% setenv VCS_HOME <VCS_INST_DIR>
% setenv LD_LIBRARY_PATH <VERDI_INST_DIR>/share/PLI/VCS \
<PLATFORM>:$LD_LIBRARY_PATH
```

You can also set these variables to help facilitate the script:

```
% setenv VERDI_HOME <VERDI_INST_DIR>
% setenv tab $VERDI_HOME/share/PLI/VCS/LINUXAMD64/novas.tab
% setenv pli $VERDI_HOME/share/PLI/VCS/LINUXAMD64/pli.a
```

Compile Designs with the UVM Libraries

The following sections describe how you can compile a design with UVM libraries.

Note:

If you want to specify a different UVM library, you can use the `VCS_UVM_HOME` environment variable to specify it.

Compile with UVM

Note:

The current default UVM version is 1.1d. To use UVM-1.2, use `-ntb_opts uvm-1.2` instead of `-ntb_opts uvm`.

To use UVM-IEEE, use `-ntb_opts uvm-ieee`.

You can load UVM-IEEE-2020 library using the `-ntb_opts uvm-ieee-2020` compile-time option.

You can load UVM-IEEE-2020-2.0 library using the `-ntb_opts uvm-ieee-2020-2.0` compile-time option.

Add the `-ntb_opts uvm` and `-debug_access[+argument]` compile option for the UVM library. For example:

```
% vcs -full64 -kdb -lca -debug_access+all -ntb_opts uvm-1.2 \ -sverilog
-l compile.log -timescale=1ns/1ns -V +incdir+../sv \ ubus_tb_top.sv
```

Note:

The `-kdb` option enables the UFE flow.

The `-debug_access+thread` option enables the debugging of the SystemVerilog threads. This is the minimum debug option to record transactions and messages into FSDB.

For more information on the arguments supported with the `-debug_access` option, see the *VCS User Guide*.

Compile in VCS UUM Flow

To compile this example with VCS, execute the following commands:

```
% vlogan -full64 -kdb -lca -timescale=1ns/1ns -ntb_opts uvm
% vlogan -full64 -kdb -lca -ntb_opts uvm +incdir+../sv ubus_tb_top.sv
% vcs -lca -kdb -debug_access+all -ntb_opts uvm ubus_tb_top
```

Note:

The `-debug_access[+argument]` VCS option is available only with VCS J-2014.12 or later versions.

Configure the Simulation

Simulation can be configured either in the Verification Compiler Platform flow or in Verdi flow. The two configuration flows are documented as below:

- [Import Design and Configure Simulation in Verdi](#)
- [Configure Simulation with UFE Flow](#)

Import Design and Configure Simulation in Verdi

To import a UVM Testbench design into Verdi, perform the following steps:

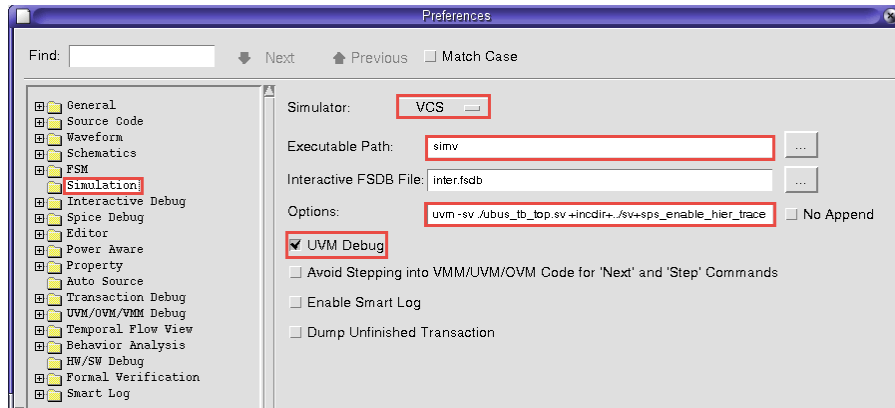
1. Invoke the Verdi platform and import the UVM testbench with the `-uvmDebug` option. For example:

```
% verdi +UVM_TESTNAME=test_read_modify_write -uvmDebug \
-ntb_opts uvm -sv ./ubus_tb_top.sv +incdir+../sv
```

2. Use the **Tools -> Preferences** option in the *nTrace* main window to configure the simulation options. Select the *Simulation* page on the *Preferences* form and follow these steps:
 - a. Select VCS as the Simulator.
 - b. Type the path of the simulator executable file in the **Executable Path** field.

- c. Add the desired simulator options in the **Options** field. For example, add the `+UVM_TESTNAME=test_read_modify_write` option to specify a UVM example.
- d. The UVM Debug option is automatically enabled when the `-uvmDebug` option is added in the Verdi command line.
- e. Click **OK** to complete the simulation settings.

Figure 1 Preference Setting for UVM Interactive Debug



3. Run the simulation.

Note:

If the **UVM Debug** option is enabled in the *Preferences* form, the `+UVM_VERDI_TRACE=UVM_AWARE` option is automatically added to the interactive console.

Configure Simulation with UFE Flow

Perform the following steps to configure simulation with the UFE flow in the Verification Compiler Platform environment:

1. Include the `-kdb` option during compilation in VCS as shown in the [Figure 1](#)
2. Execute `simv -verdi` or `simv -gui=verdi` in Verdi or `simv -gui` in the Verification Compiler Platform environment.

Points to Note

By default, some data are not available in the UVM Aware Debug windows. You have to specify special options for `simv` to record the data.

- If you want to view the sequence history in the *Sequence View* pane, add the `+UVM_TR_RECORD` option to enable transaction recording
- If you want to view the OVM/UVM hierarchy tree, add the `+UVM_VERDI_TRACE=HIER` option to enable OVM/UVM hierarchy tree and TLM port connectivity dumping.

However, if the OVM/UVM hierarchy tree is deep and complex, it may take a lot of time to traverse all the OVM/UVM components. You can add the compilation directive in compilation time, `+define+VERDI_DHIER_INTERFACE_BY_CLASS_DEFN`, which is used to optimize the traverse time during building the UVM hierarchy tree. This optimization works only if the runtime option, `+UVM_VERDI_TRACE=HIER`, is added.

- If you want to view the register read/write access history in the *Register View*, add the `+UVM_VERDI_TRACE=RAL` option to enable register history dumping.
- If you specify the `+UVM_VERDI_TRACE=MSG` option, all UVM messages are recorded into FSDB, but the messages are not displayed on console. You must use the `+UVM_VERDI_TRACE=MSG+PRINT` option to view the UVM messages on console.

Perform Interactive Simulation with UVM Aware Debug

Prerequisite: Load the design and the UVM testbench in the Verdi platform and complete the simulation configuration.

Perform the following steps to start the **Interactive Simulation Debug** mode:

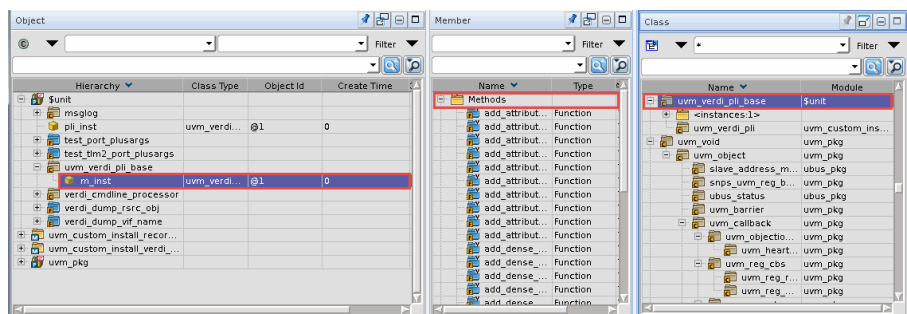
1. Select **Tools** -> **Run Simulation** option to start the Interactive Simulation Debug mode.

After the simulation finishes execution phase, the *Object*, *Member*, *Source Code*, and *Interactive Console* panes open automatically. All the defined classes and the instantiations are shown with a hierarchical view in the *Class* pane.

2. Select an item either the *Class* or *Object* panes, to automatically display the related content in the *Member* pane. In the *Object* pane, right-click the object and select the **Show in Class Browser** option to view all the defined classes and the instantiations are shown with a hierarchical view.

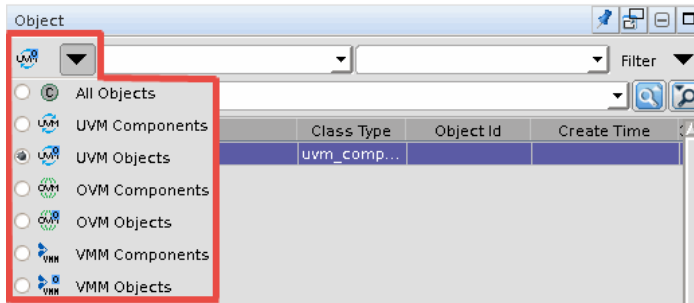
You can observe the hierarchical and dynamic changes of objects with the instances trees in the *Object* pane.

Figure 2 *Classes Derived from UVM Object and Object Names*



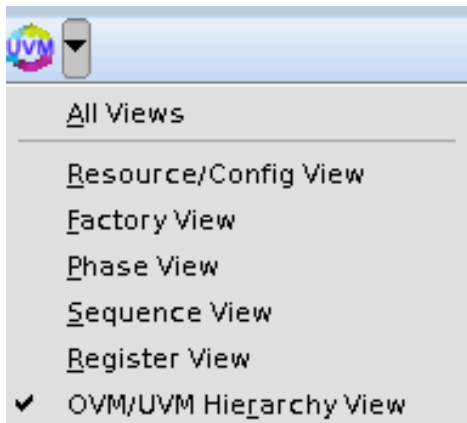
3. Select the **UVM Components** or UVM Objects viewing modes in the **Mode** menu of the *Object* pane to focus on the UVM library group of interest, as show in the following figure:

Figure 3 Mode Menu in the Object Browser



4. After enabling the **Interactive Simulation Debug** mode, use the UVM Menu associated with the UVM icon  to select the UVM panes, as shown in the following figure:

Figure 4 UVM Debug Panes



You can also select the **All Views** option to select all the *UVM Aware* panes. Verdi displays the UVM icon  in the toolbar only when the current loaded design includes the UVM package. Otherwise, Verdi hides this icon in the toolbar. You can disable this by setting the environment variable `NOVAS_UVM_ICON_SHOW_DEPAND_UVM_PKG` as 0. The default value is 1.

Note:

If the `-uvmDebug` option is not added while invoking the Verdi platform and the UVM Debug options are not enabled in the *Preferences* form, a warning message is displayed when opening the *UVM-Aware* panes. The warning

message specify that the UVM debug features for interactive mode cannot be completely supported with the above situation.

Behavior of OVM/UVM Hierarchy Tree View at the Startup of the Simulation

The following is the behavior of the *OVMM/UVM Hierarchy Tree View* at the startup of the simulation:

Mode	Description
Interactive Simulation Debug Mode	When FSDB is loaded, UVM main menu (Tools -> UVM/OVM/VMM Debug)/UVM toolbar icon  is enabled, and all the sub-menu options of the toolbar icon are enabled. Main window View -> OVM/UVM Hierarchy Tree option is also enabled. If FSDB is not loaded, then the UVM main menu (Tools -> UVM/OVM/VMM Debug)/UVM toolbar icon  and <i>OVMM/UVM Hierarchy Tree View</i> are disabled. By default, <i>OVMM/UVM Hierarchy Tree View</i> is disabled.
Post-processing Debug Mode	When FSDB is loaded, UVM main menu (Tools -> UVM/OVM/VMM Debug)/UVM toolbar icon  is enabled, and the <i>Register View</i> and OVMM/UVM Hierarchy View options are enabled. Main window View -> OVM/UVM Hierarchy Tree option is enabled. If FSDB is not loaded, then the UVM main menu (Tools -> UVM/OVM/VMM Debug)/UVM toolbar icon  and <i>OVMM/UVM Hierarchy Tree View</i> are disabled. By default, <i>OVMM/UVM Hierarchy Tree View</i> is disabled.

The OVM/UVM Hierarchy Tree View status (hidden/display) is saved to the `novas.rc` file.

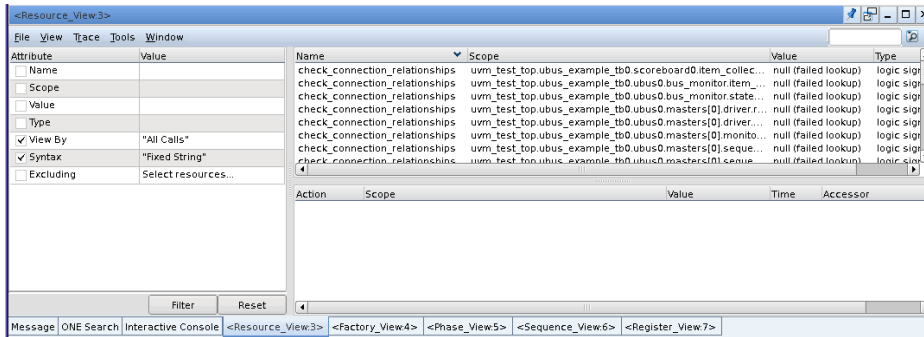
If you invoke Verdi next time, the *OVMM/UVM Hierarchy Tree View* status is the same as last time you exit Verdi (if `novas.rc` file exists).

UVM Debug Panes

The UVM Debug feature support is integrated with the **Interactive Simulation Debug** mode in the Verdi platform. The *OVMM/UVM Hierarchy Tree* view is docked in the same

location as the *Object Browser* and *Class Browser*. The UVM related panes are opened and docked automatically as new tabs in the same location as the **Interactive Console** tab of the *Message* pane after selecting the *All Views* option in the UVM menu as shown in the following figure:

Figure 5 UVM Aware Layout



The UVM Debug panes: the *OVN/UVM Hierarchy Tree View*, *Resource View*, *Factory View*, *Phase View*, *Sequence View* and *Register View* panes are described in the following chapters. You can drag and drop or dock/undock these panes the same as other panes in the Verdi platform to the desired location as needed.

4

UVM Debug Usage

This chapter describes the usage of the following UVM Debug panes:

- [OVM/UVM Hierarchy Tree Pane Usage](#)
- [Save/Restore Session Support in UVM Aware Debug](#)
- [Resource View](#)
- [Factory View](#)
- [Phase View](#)
- [Sequence View](#)
- [Register View](#)
- [Multitasking Task](#)
- [Debugging UVM Registers in Verdi Post-Processing Mode](#)
- [Viewing OVM/UVM Component Waveforms in nWave](#)

Note:

The options in each of these views are documented in the *UVM Interactive Simulation Debug* chapter of the *Verdi Command Reference Guide*.

OVM/UVM Hierarchy Tree Pane Usage

The Verdi platform supports the *OVM/UVM Hierarchy Tree* pane to display the component hierarchy of a testbench. The *OVM/UVM Hierarchy Tree* pane is docked at the upper left corner of the main window, when invoked. Along with the other UVM Aware window, it is

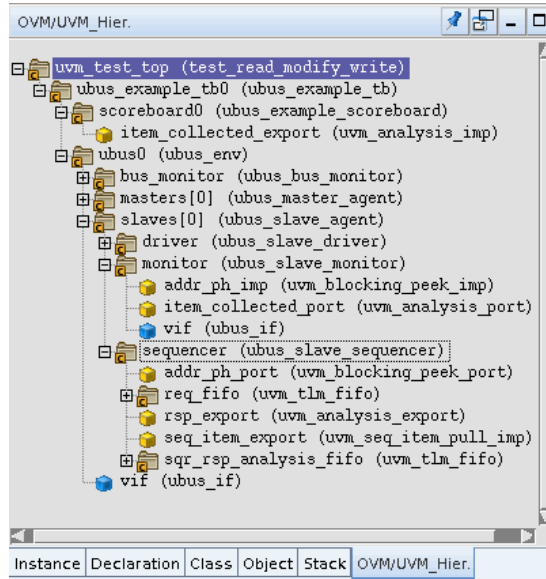
invoked from the  *UVM/OVM/VMM Debug Windows* icon.

When you select a component in the *OVM/UVM Tree Hierarchy* pane, you can view the detailed information of the component, perform a search in the tree hierarchy, drag and drop components from the hierarchy tree to the *Transaction and Protocol Analyzer* and vice versa.

Note:

OVM/UVM hierarchy tree supports both interactive and post-process mode. If you want to view the OVM/UVM hierarchy tree, add the `+UVM_VERDI_TRACE=HIER` option at simulation runtime to enable OVM/UVM hierarchy tree and TLM port connectivity dumping.

Figure 6 OVM/UVM Hierarchy Tree Pane



The following sections describe the OVM/UVM hierarchy tree pane usage in detail:

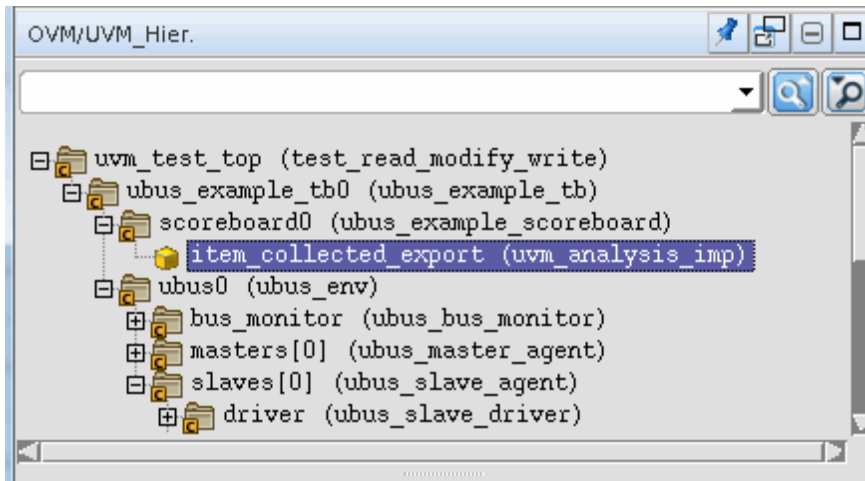
- [Searching in the OVM/UVM Hierarchy Tree Pane](#)
- [View Interface in Source Code](#)
- [View Definition in Source Code](#)
- [View Creation in Source Code](#)
- [View the Connect Function Call in Source Code](#)
- [View the Resource Information and Configuration](#)
- [View the Sequence Data](#)
- [View Original Type](#)
- [View in Object Browser](#)
- [View in Class Browser](#)

- [Add a Component Object to nWave](#)
- [Interface Mapping Node in the OVM/UVM Hierarchy Tree](#)

Searching in the OVM/UVM Hierarchy Tree Pane

Select the **Show Navigation Text** field to view the **Search** bar in the *OVM/UVM Hierarchy Tree* pane.

Figure 7 Searching in the OVM/UVM Hierarchy Tree Pane



For more information regarding the *OVM/UVM Hierarchy Tree* pane right-click option menu, see the *Verdi Command Reference Guide*.

View Detailed Information

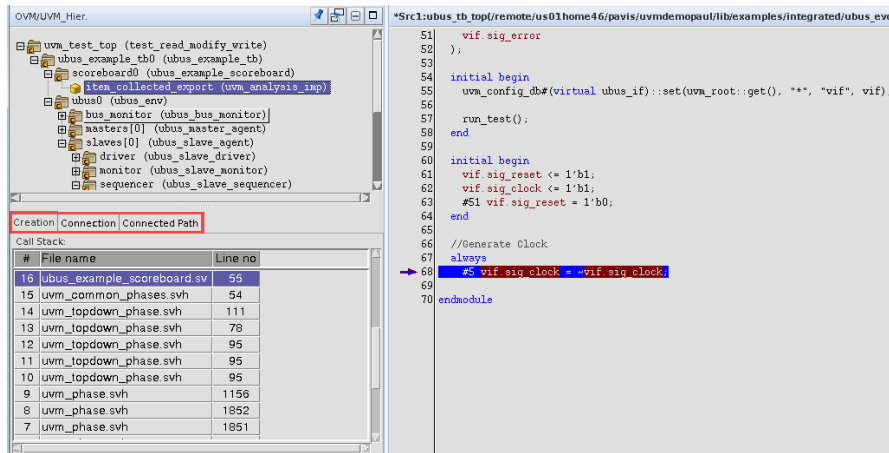
Select the **Show Detailed Information** option from the right-click option menu in the *OVM/UVM Hierarchy Tree* pane to view the detailed information for the **Show Creation** and **Show Connections** options. A new panel is created with three tabs: **Creation**, **Connection**, and **Connected Path**.

Note:

The **Connection** and **Connected Path** tabs are available only when the selected node is a port/export.

The connection parameters of a component is shown in the following figure:

Figure 8 Detailed Information Folders



Tab Name	Description
Creation	Displays the Call Stack creation for the selected node. On double-clicking on the node in the Call Stack list, the source code of the corresponding node is displayed in the <i>Source Code</i> pane.
Connection	This folder is enabled only on selecting a port in the <i>OVM/UVM Hierarchy Tree</i> pane. The following fields are displayed: Connection: Displays multiple port connections of the selected port. Call Stack grid: Displays the Call Stack of the connection corresponding to the value that appears in the Connection field.
Connection Paths	This folder is enabled only on selecting a port in the <i>OVM/UVM Hierarchy Tree</i> pane. The following fields are displayed: Path: For a multi-path selection, the last three segments of a path are displayed. If a port has multiple paths, switch to the port connection folder in the port grid. Ports: Displays connection details. Double-click a grid item to switch the definition.

Creation Tab Usage

The **Creation** tab displays the calling sequence (Call Stack) of the instrumentation code tracking the component creation in the testbench. #0 represents the top of the Call Stack while a bigger number represents a deeper level in the Call Stack.

Double-click other levels in the Call Stack to examine the corresponding statement in the *Source Code* pane.

Figure 9 Call Stack Information in the Creation Tab

Creation	Connection	Connected Path
Call Stack:		
#	File name	Line no
16	tb_env.svh	93
15	uvm_common_phases.svh	54
14	uvm_topdown_phase.svh	111
13	uvm_topdown_phase.svh	78
12	uvm_topdown_phase.svh	95
11	uvm_phase.svh	1156
10	uvm_phase.svh	1852
9	uvm_phase.svh	1851
8	uvm_phase.svh	1851
7	uvm_phase.svh	1847
6	uvm_root.svh	426
5	uvm_root.svh	423
4	uvm_root.svh	423
3	uvm_globals.svh	40

Connection Tab

The **Connection** tab displays the calling sequence (Call Stack) of the instrumentation code tracking the port connection in the testbench as shown in the following figure. #0 represents the top of the Call Stack while a bigger number represents a deeper level in the Call Stack. By default, the statement that connects the port is highlighted.

Double-click other levels in the Call Stack to examine the corresponding statement in the *Source Code* pane.

When a port has multiple connections, the selection menu beside the **Port** is used to select one connection and examine its corresponding Call Stack.

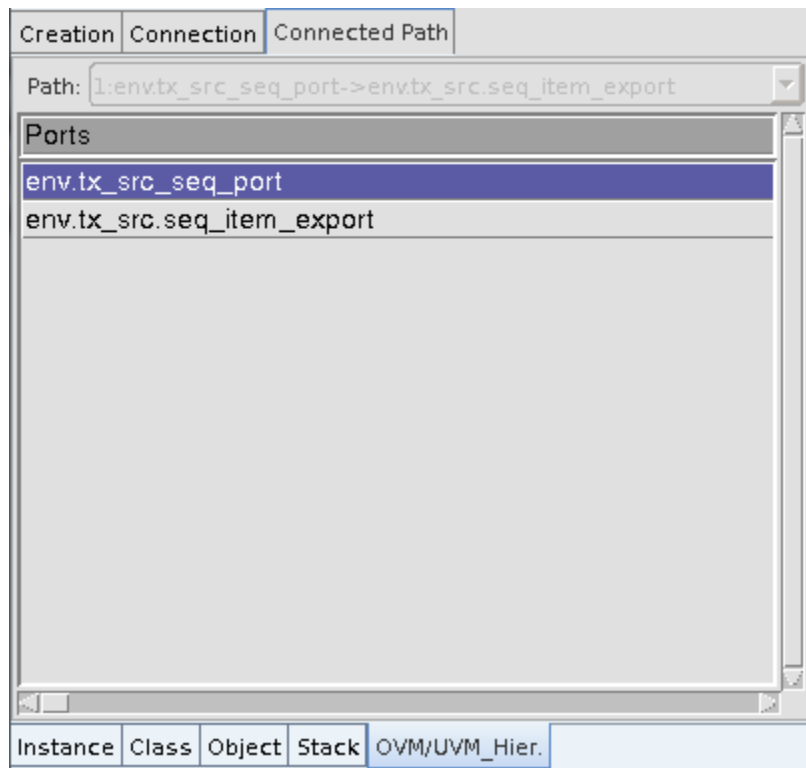
Figure 10 Calling Sequence of the Instrumentation Code in the Connection Tab

Creation Connection Connected Path		
Port: To 1:envtx_src.seq_item_export		
#	File name	Line no
16	tb_env.svh	118
15	uvm_common_phases.svh	96
14	uvm_bottomup_phase.svh	108
13	uvm_bottomup_phase.svh	81
12	uvm_bottomup_phase.svh	61
11	uvm_phase.svh	1156
10	uvm_phase.svh	1852
9	uvm_phase.svh	1851
8	uvm_phase.svh	1851
7	uvm_phase.svh	1847
6	uvm_root.svh	426
5	uvm_root.svh	423
4	uvm_root.svh	423
3	uvm_globals.svh	40

Connected Path Tab

The **Connected Path** tab displays the connected path information of the selected port. The selection menu beside **Path** is used to select a path passing through the port. The ports on the selected path is displayed in the bottom table. The following figure illustrates the difference between a **connection** and a **connected path**. A connection finds the immediate connected ports and connected path finds all the paths passing through the port.

Figure 11 Connected Path Information of the Selected Port



Interaction Between the Views

A node from the *Testbench Hierarchy* tree or a port from the **Connected Path** tab is dragged and then dropped in another pane. The drop behavior to different windows is described as follows:

- **Drop into the Testbench Hierarchy tree:** Highlight the corresponding node in the *Testbench Hierarchy* tree and show its definition in the *Source Code* pane.
- **Drop into tBrowser:** Display the associated transactions of the component/port in *Transaction and Protocol Analyzer*.
- **Drop into other windows:** Not supported.

Note:

Drag and drop to the *nWave* window is only be available when transactions are dumped in the FSDB file.

View Interface in Source Code

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Interface** option. This option displays the interface code of the selected component in the *Source Code* pane.

Figure 12 Result of the Show Interface Option

```
*Src1:router_test_top.host(/remote/us01home46/pavis/Desktop/uvvm_guide_cases/ces_labs/rtl/ral/host_io.sv)
3 interface host_io(input logic clk);
4     logic        wr_n;
5     logic [15:0] address;
6     wire  [15:0] data;
7
8     clocking cb @(posedge clk);
9     inout  data;
10    output  address;
11    output  wr_n;
12 endclocking
13
14    clocking mon @(posedge clk);
15    input  data;
16    input  address;
17    input  wr_n;
18 endclocking
19
20    modport dut(input clk, input wr_n, address, inout data);
21 endinterface: host_io
22 `endif
```

View Definition in Source Code

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the *Show Definition* option. This option displays the source code definition of the selected component in the *Source Code* pane.

Figure 13 Result of the Show Definition Option

```
*Src1:test_seq_lib_cfg.env.h_agent.drv(/remote/us01home46/pavis/Desktop/uvvm_guide_cases/ces_labs/lab7/host_agent.sv)
51 virtual host_io `sig;
52 event go;
53
54 `uvm_component_utils(host_driver)
55
56 function new(string name, uvm_component parent);
57     super.new(name, parent);
58     `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
59 endfunction
60
61 virtual function void build_phase(uvm_phase phase);
62     `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
63     if (!uvm_config_db#(virtual host_io)::get(this, "", "host_io", sig)) begin
64         `uvm_fatal("CFGERR", "DUT host interface not set");
65     end
66 endfunction
67
68 virtual task pre_reset_phase(uvm_phase phase);
69     sigs.wr_n    = 'x;
70     sigs.address = 'x;
71     sigs.data    = 'x;
72 endtask
```

View Creation in Source Code

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Creation** option. This option displays the source code creation of the selected component in the *Source Code* pane.

Figure 14 Result of the Show Creation Option

```
*Src1:test_seq_lib_cfg.env.h_agent.build_phase(/remote/us01home46/pavis/Desktop/uvm_guide_cases/ces_labs/lab7/host_agent.sv)
24 virtual function void build_phase(uvm_phase phase);
25   super.build_phase(phase);
26   'uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
27
28   // The agent retrieves virtual interface.
29   uvm_config_db#(virtual host_io)::get(this, "", "host_io", sigs);
30
31   if (is_active) begin
32     seqr = host_sequencer::type_id::create("seqr", this);
33     drv = host_driver::type_id::create("drv", this);
34     uvm_config_db#(virtual host_io)::set(this, "drv", "host_io", sigs);
35     uvm_config_db#(virtual host_io)::set(this, "seqr", "host_io", sigs);
36   end
37   mon = host_monitor::type_id::create("mon", this);
38   uvm_config_db#(virtual host_io)::set(this, "mon", "host_io", sigs);
39 endfunction
```

View the Connect Function Call in Source Code

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Connections** option. This option displays the connect function call code of the selected component in the *Source Code* pane.

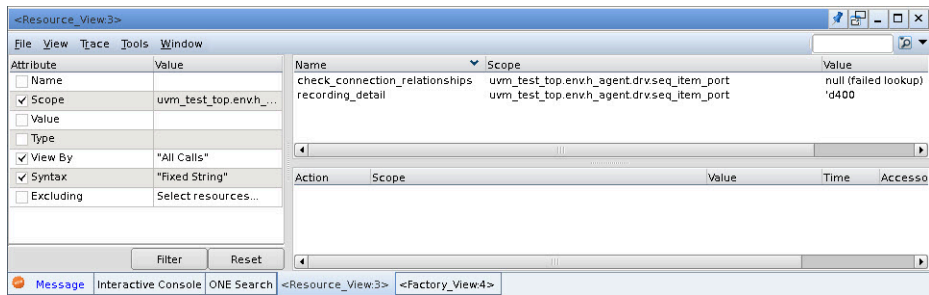
Figure 15 Result of the Show Connections Option

```
*Src1:test_seq_lib_cfg.env.h_agent.connect_phase(/remote/us01home46/pavis/Desktop/uvm_guide_cases/ces_labs/lab7/host_agent.sv)
41 virtual function void connect_phase(uvm_phase phase);
42   super.connect_phase(phase);
43   'uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
44   if (is_active) begin
45     drv.seqr_item_port.connect(seqr.seqr_item_export);
46   end
47 endfunction
48 endclass
49
```

View the Resource Information and Configuration

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Resource Data** option. This option displays the all available configuration or resource information of the selected component in the *Resource View* pane.

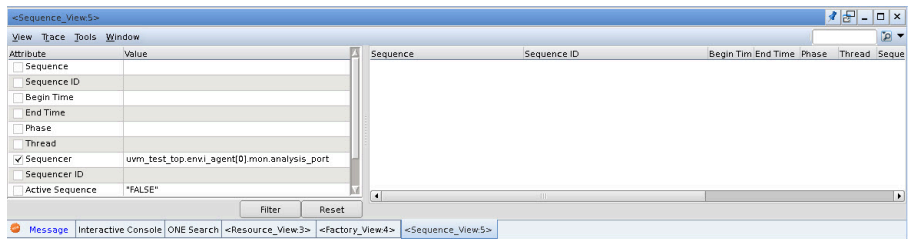
Figure 16 Result of the Show Resource Data Option



View the Sequence Data

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Sequence Data** option. This option displays the sequence data generated from the selected component in the *Sequence View* pane.

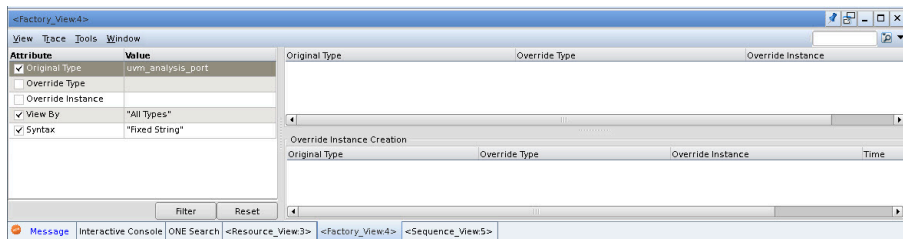
Figure 17 Result of the Show Sequence Data Option



View Original Type

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show Original Type** option. This option displays the original type of the selected component in *Factory View* pane.

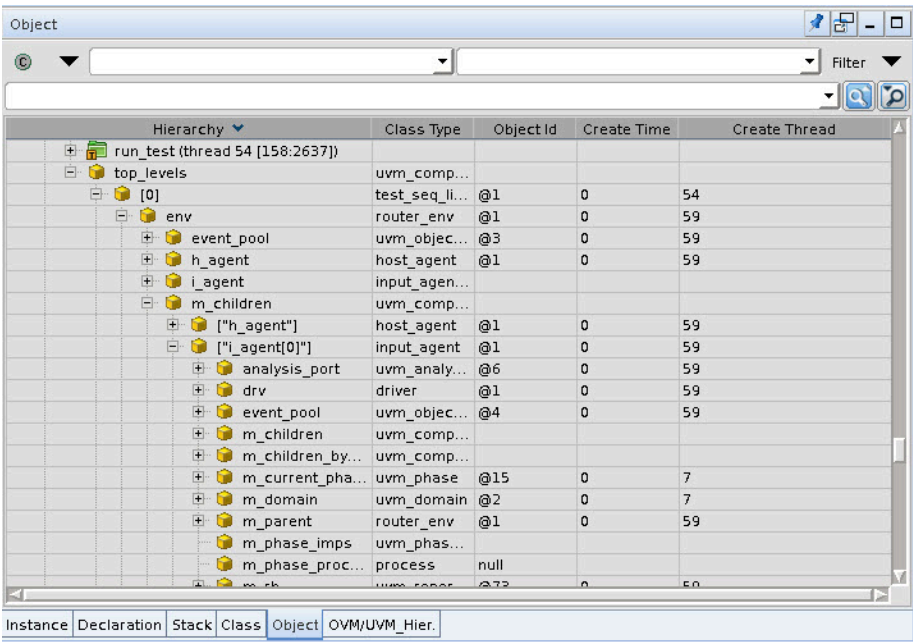
Figure 18 Result of the Show Original Type Option



View in Object Browser

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show in Object Browser** option. This option displays the selected component in the *Object Browser* pane.

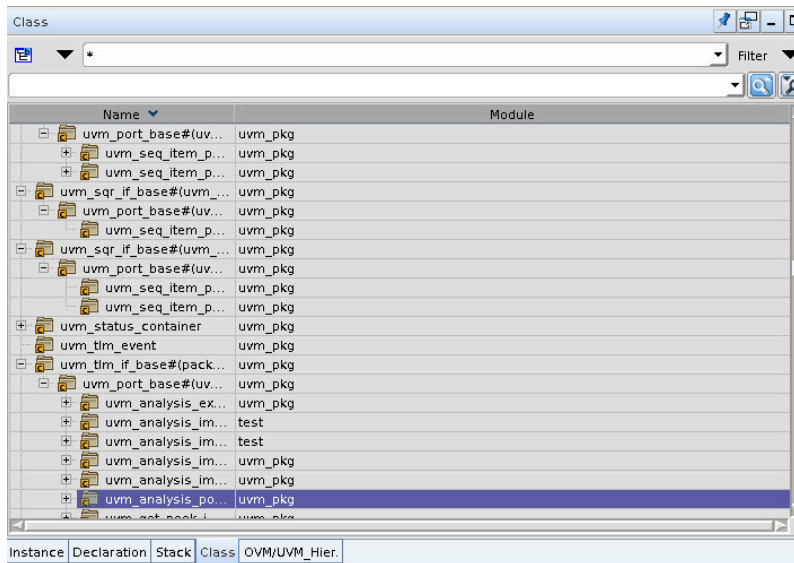
Figure 19 Result of the Show in Object Browser Option



View in Class Browser

Select a component on the *OVM/UVM Hierarchy Tree* pane and right-click on the component to select the **Show in Class Browser** option. This option displays the class of the selected component in the *Class Browser* pane.

Figure 20 Result of the Show in Class Browser Option



Add a Component Object to nWave

Verdi provides a right-click menu item **Add to Waveform** that you can use to dump a component object to *nWave* window. To use this functionality, right-click on the component object for which you want to dump the waveform to the *nWave* window, and select the option **Add to Waveform**. The component object in the *nWave* window is shown in [Figure 21](#)

Note:

The **Add to Waveform** menu option is only enabled when the selected component object exists in FSDB.

In the interactive mode, this item is always enabled because the dumper can dump the object values even if they are not dumped in the FDSB file. In the batch mode, this item should be enabled in *OVM/UVM Hierarchy Tree* when the component object exists in the FSDB file. The operation of this menu item is in the following order:

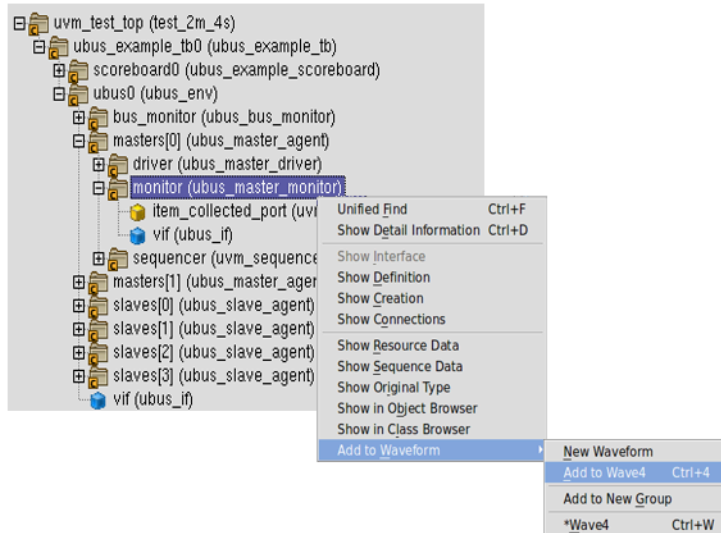
1. If the object is not in FSDB, it is enabled for the dynamic object dumping.
2. Load the FSDB data of this object into *nWave*.

Note:

The operation is only applicable to single selection.

In the post-process mode, add `+UVM_VERDI_TRACE=COMPWAVE` as the runtime option. Refer to the [Viewing OVM/UVM Component Waveforms in nWave](#) chapter for detailed usage.

Figure 21 Adding a Waveform to nWave

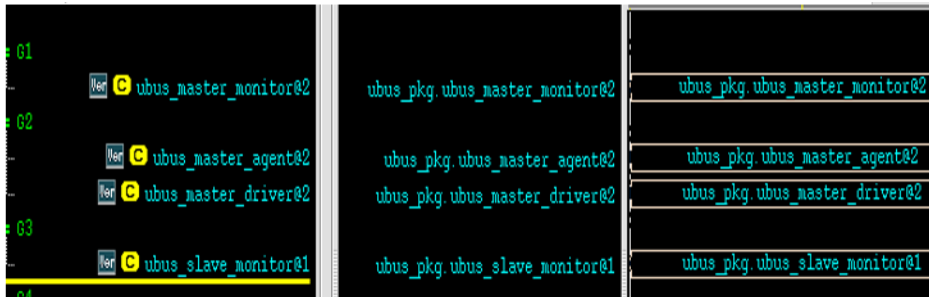


Note:

A component contains other components. The **uvm_component** defines which of the operations of the **Add to Waveform** menu item is different for different types of component objects. On clicking the **Add to Waveform** option, an error message is displayed as follows: Unrecognized signal: `@{TESTBENCH}.svt_uart_uvm_pkg.` no further operation. This indicates that the component is protected with VIP codes. VIP codes are not supported in Verdi.

The component waveform is displayed in the *nWave* window as shown in the following figure:

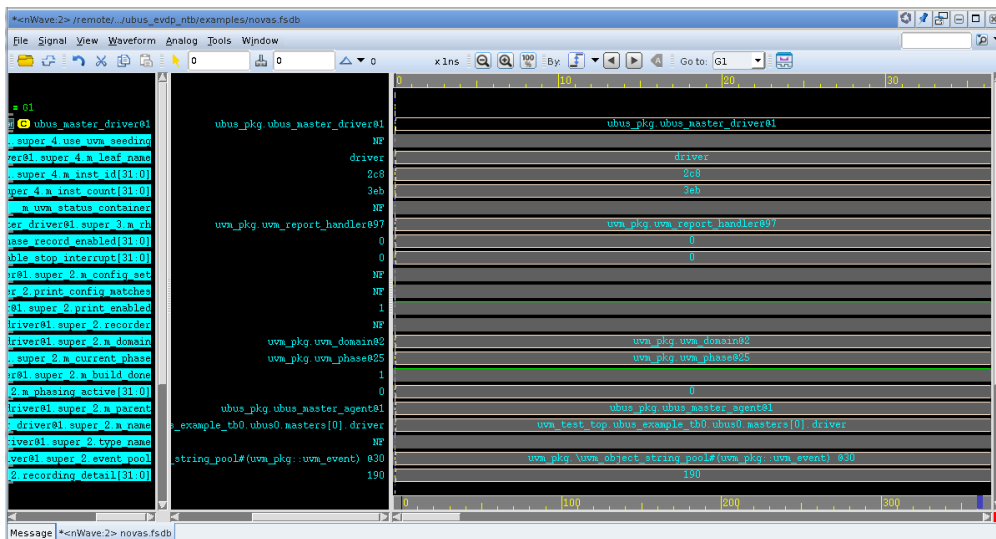
Figure 22 Component Waveform Displayed in *nWave*



In component waveform shown in Figure 22. The signal name is in the format <class name>@<object id>, for example, ubus_master_driver@1. The <object id> is the internal ID from simulator indicating the ordering number of the object for the specified class.

After adding component to *nWave*, you can double-click it to expand it, as shown in the following figure:

Figure 23 Component Member Expanded



Limitations of Adding Component Waveform to *nWave*

- Only the VCS simulator is supported.
- Only one level of members of the class is dumped. For example, the members of class of `uvm_component` is not displayed in *nWave*.

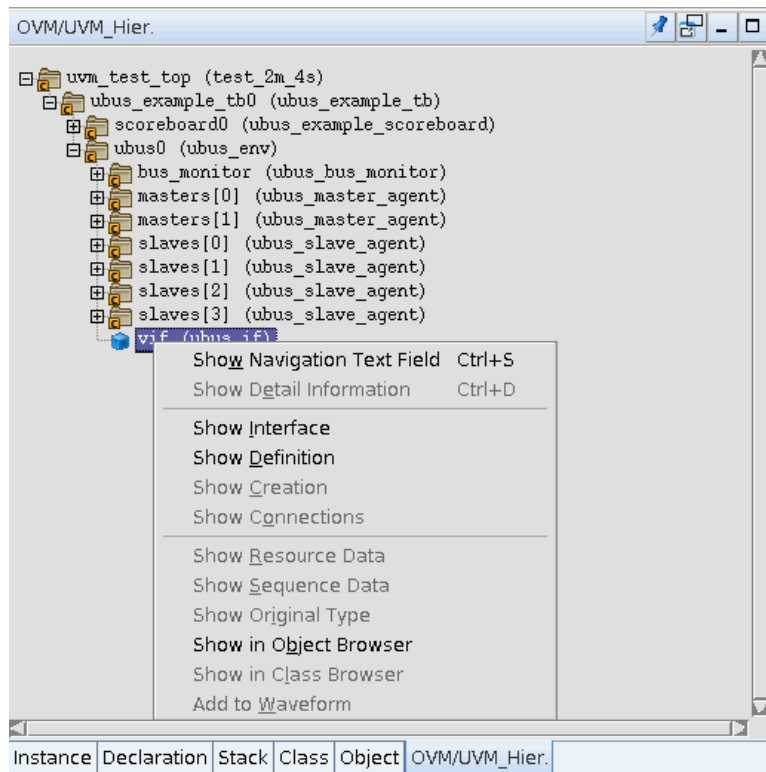
- VCS Partition Compile is not supported.
- Dumping of objects in VIP encrypted classes is not supported.
- The component waveform does not display the following data types:

char var (as is not in the LRM data types).

Interface Mapping Node in the OVM/UVM Hierarchy Tree

In the *OVM/UVM Hierarchy Tree* pane, you can view the virtual interface under the corresponding component. From the virtual interface node, you can add the connected interface signals into *nWave* using the **Add to Waveform** option. You can double-click on the node to view the source code definition of the variable.

Figure 24 Virtual Interface Node in the OVM/UVM Hierarchy Tree



The following new options are supported for the virtual interface nodes:

Option	Action
Show Navigation Text Field	Use this option to search for a string in the <i>OVM/UVM Hierarchy Tree</i> pane.

Option	Action
Show in Object Browser	Use this option to automatically pop the Object Browser to the front tab and automatically scroll and highlight the interface.
Show Definition	Use this option to view the source definition of the variable. If the option is unable to jump to the related source codes, then it displays the following error message: <code>Failed to find ubus_if in design</code> . It means that the selected virtual interface could be protected (the component might have VIP protected codes). VIP codes are not supported in Verdi.
Show Interface	Use this option to find the connected interface signal source code in <i>nTrace</i> .
Add to Waveform	Use this option to dump the waveform of the selected component object to <i>nWave</i> .

Limitations of the OVM/UVM Hierarchy Tree Pane

- If there is no data in the FSDB file, you cannot open an empty *OVN/UVM Hierarchy Tree* pane. The option is enabled only after the data is available in FSDB.
- Unlike the rest of the UVM windows, in the *OVN/UVM Hierarchy Tree* pane, a check mark appears before the option in the menu to indicate whether the pane is opened or not. If the check mark is unchecked, the pane is removed. Such a behavior does not exist for the other *UVM Aware Debug* windows.

Specifying UVM Runtime Options

Verdi only adds the following runtime option in the interactive console if the **UVM Debug** check box is enabled:

```
+UVM_VERDI_TRACE=UVM_AWARE
```

For simulation with UVM not provided by Verdi or VCS, you must specify the following runtime options in the interactive mode even if the **UVM Debug** check box is enabled:

```
+UVM_CONFIG_TRACE, +UVM_PHASE_TRACE, +UVM OBJECTION_TRACE,  
+UVM_RESOURCE_DB_TRACE,  
  
+UVM_LOG_RECORD
```

Save/Restore Session Support in UVM Aware Debug

Verdi supports session save and restore for all UVM views. The following sections describe this support in detail:

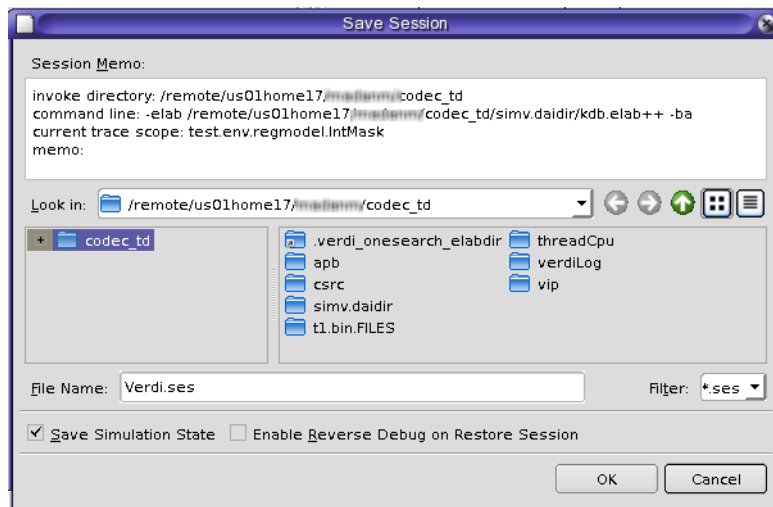
- [Commands to Save/Restore Session for all UVM Views in Interactive Mode](#)
- [Save/Restore Session Support for the Register View in Post-Processing Mode](#)

Commands to Save/Restore Session for all UVM Views in Interactive Mode

Verdi provides the following two functionalities to save the current debugging session:

- Using **File -> Save Session** command. Select this command to view the *Save Session* dialog box, as shown in the following figure:

Figure 25 Save Session Dialog Box

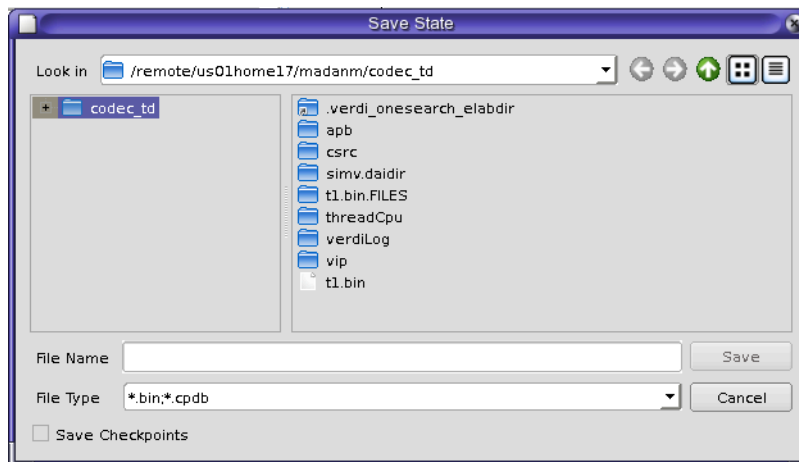


You can use the **Save Session** command to save the Verdi session with current simulation state. You must enable the **Save Simulation State** check box to save the

current Verdi session. You can use the **File -> Restore Session** command to restore the saved Verdi session.

- Using **Simulation -> Save State** command. Select this command to view the *Save State* dialog box, as shown in the following figure:

Figure 26 *Save State Dialog Box*



The **Save State** command allows you to save the current simulation state and dumped FSDB file. It does not save the current Verdi session.

The **Simulation -> Restore State** command allows you to restore the current simulation state and the FSDB file. All interactive debug windows, including the UVM Aware windows, are refreshed to reflect the current simulation state.

During the debug process, if you want to restart debugging again from the saved simulation state, you can restore the simulation state again. In this case, only the simulation state and FSDB are restored. The current Verdi session (for example, which signals are present in *Wave* window, and so on) is not restored.

The following sections describe what would be saved and restored in each UVM Aware view:

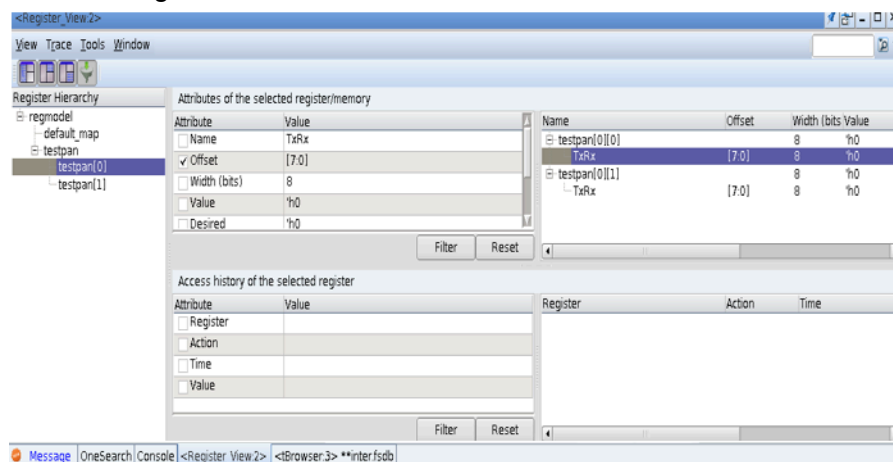
- [Register View Save/Restore Session](#)
- [Factory View Save/Restore Session](#)
- [Phase View Save/Restore Session](#)
- [Sequence View Save/Restore Session](#)
- [Resource View Save/Restore Session](#)
- [OVM/UVM Hierarchy View Save/Restore Session](#)

Register View Save/Restore Session

In the *Register View*, the following items are restored as they were before the save session:

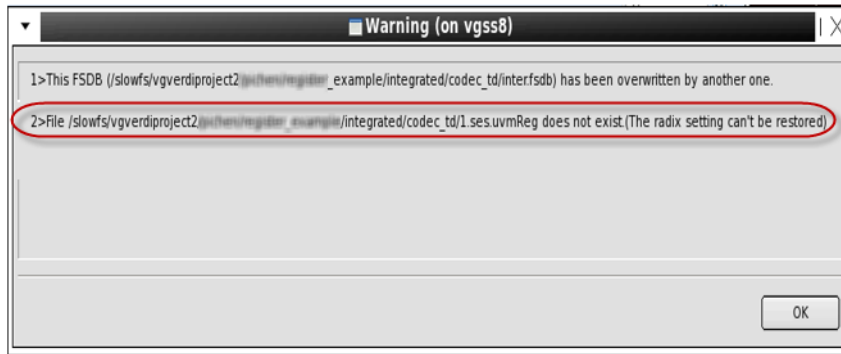
- *Register View* pane layout (*Register Hierarchy/Register Attribute/Register Access History* pane on or off)
- Register hierarchy data and selected items if exist
- Register attribute data and selected items if exist
- Register access history of the selected register data and selected items if exist
- Quick Filter on/off status, the values inside it and the status of check boxes
- If you have performed the radix display change on some signals before save session, those are restored too.

Figure 27 Register View Save/Restore Session



- Verdi displays a warning message if it cannot find original radix change file during the restore session.

Figure 28 Warning Message

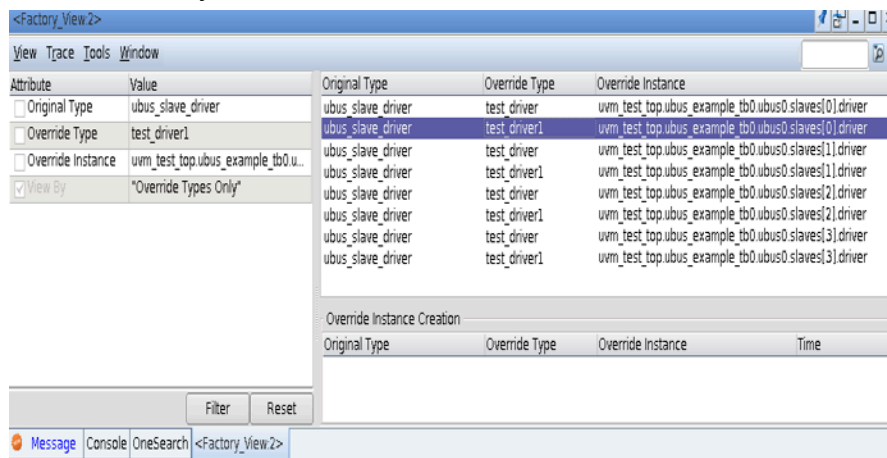


Factory View Save/Restore Session

In the *Factory View*, the following items are restored as they were before the save session:

- Factory View* pane layout (*Factory Override Instance Creation* pane on/off)
- Factory data and selected items if exist
- Factory override instance creation data and selected items if exist
- Quick Filter on/off status, the values inside it and the status of check boxes

Figure 29 Factory View Save/Restore Session

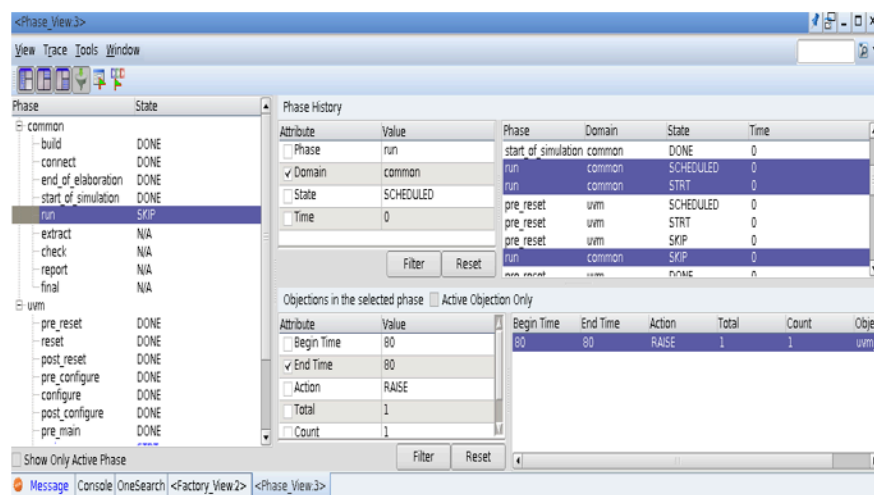


Phase View Save/Restore Session

In the *Phase View*, the following items are restored as they were before the save session:

- *Phase View* pane layout (*Phase Hierarchy/Phase History/Phase Objections* pane on or off)
- Phase hierarchy data and selected items if exist
- Phase history data and selected items if exist
- Phase objections data and selected items if exist
- The **Active Objection Only** check box status
- Quick Filter on/off status, the values inside it and the status of check boxes

Figure 30 *Phase View Save/Restore Session*

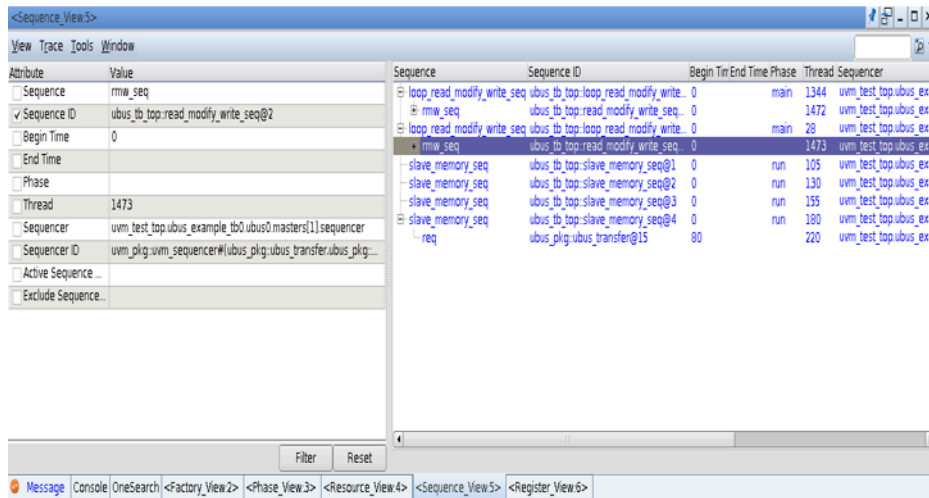


Sequence View Save/Restore Session

In the *Sequence View*, the following items are restored as they were before the save session:

- Sequence data and selected items if exist
- Quick Filter on/off status, the values inside it and the check box status

Figure 31 Sequence View Save/Restore Session

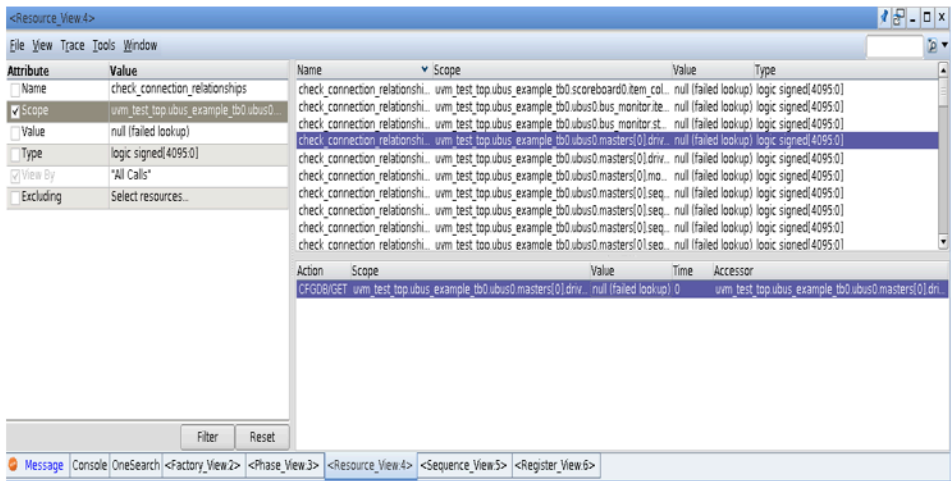


Resource View Save/Restore Session

In the *Resource View*, the following items are restored as they were before the save session:

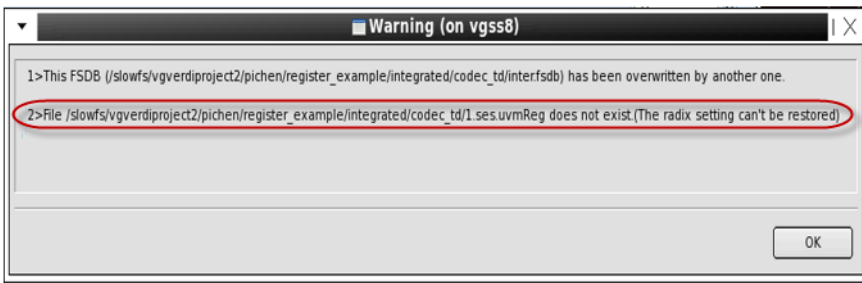
- *Resource View* pane layout (*Resource Data/Resource Access History* pane on or off)
- Resource data and selected items if exist
- Resource access history data and selected items if exist
- Quick Filter on/off status, the values inside it and the check box status
- If you do the radix display change on some signals before the save session, those are restored too.

Figure 32 Resource View Save/Restore Session



- If Verdi cannot find the original radix change file after the restore session, the following warning message is issued:

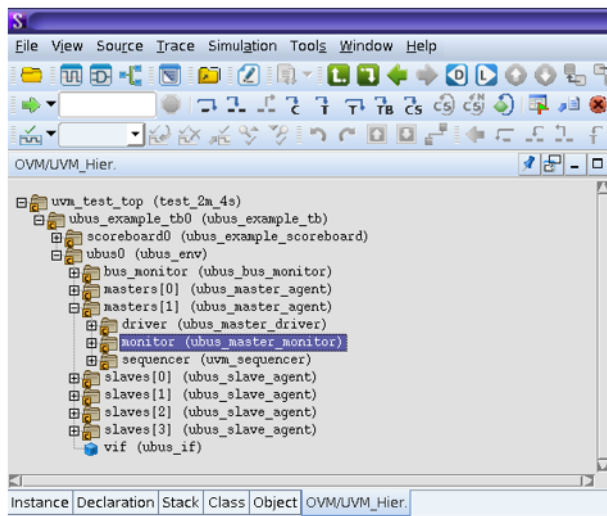
Figure 33 Warning Message



OVM/UVM Hierarchy View Save/Restore Session

In *OVM/UVM Hierarchy* view, the data is restored and the selected item is highlighted as it was before the save session.

Figure 34 OVM/UVM Hierarchy View



Resource View

The *Resource View* is a configuration interface which displays all available configurations/ resources in the design. It allows you to exchange information across different components to configure topology, mode of operation, and runtime parameters.

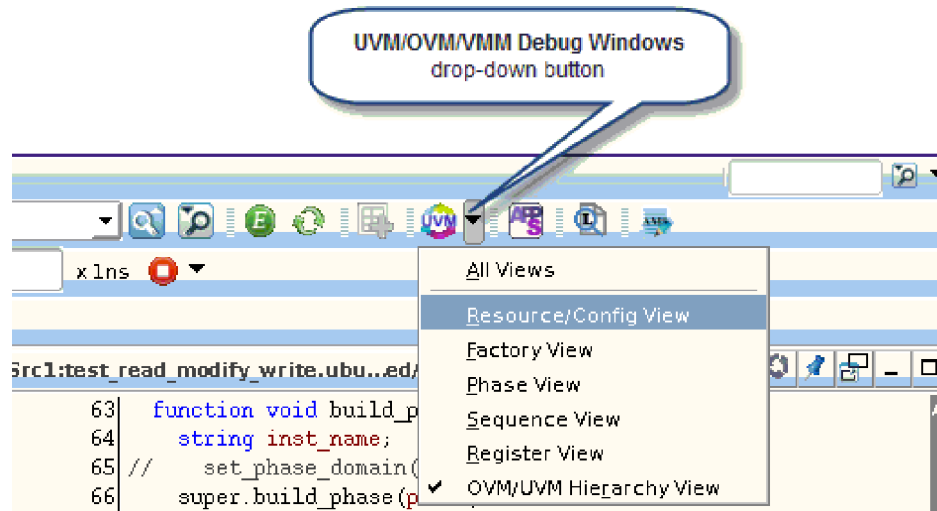
This section describes the *Resource View* in the following topics:

- [Viewing the Resource View](#)
- [Viewing the Source Code and Accessor](#)
- [Applying Quick Filter in the Resource View](#)
- [Displaying Radix in the Resource View](#)

Viewing the Resource View

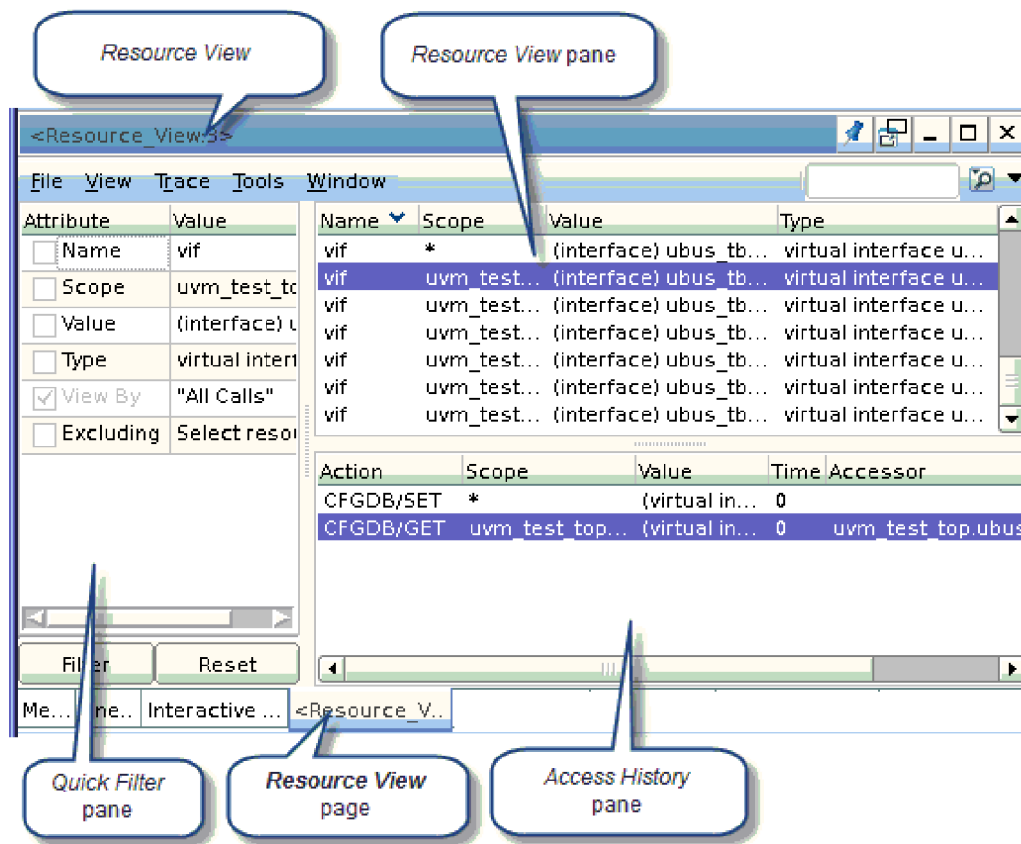
To view the *Resource View*, click the **UVM/OVM/VMM Debug Windows** drop-down button and select the **Resource/Config View** option from the drop-down menu as shown in the following figure:

Figure 35 Viewing the Resource View



You can view the *Resource View* in the **Resource View** page of the *UVM Debug* pane. It contains *Resource View* pane, *Access History View* pane, and *Quick Filter* pane, as shown in the following figure. This view updates when the simulation stops.

Figure 36 Resource View



The *Resource View* pane displays all available configurations in your design. This view contains four columns, namely, **Name**, **Scope**, **Value**, and **Type**, as shown in [Figure 36](#), and displays names and values of function arguments defined in your design. The following table describes the columns of the *Resource View* pane. You can select only one item at a time.

Column	Description
Name	Displays the name argument of the <i>get</i> function of <code>uvm_config_db</code>
Scope	Displays the scope argument of a function
Value	Displays the value or val argument of a function
Type	Displays the argument type

Note:

The recording for the following common configuration items is disabled by default:

```
check_connection_relationship, recording_detail,  
default_sequence
```

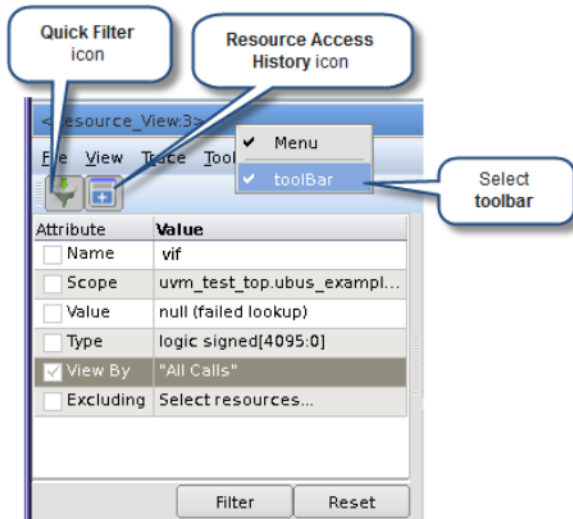
You must use the `+UVM_INC_INTERNAL_RSRC` runtime option to enable recording for these configuration items.

The *Access History* pane displays the set/get history of the resource/configuration item selected in the *Resource View* pane. This view contains five columns, namely, **Action**, **Scope**, **Value**, **Time**, and **Accessor**, as shown in [Figure 36](#). You can double-click an item in the *Access History* pane to view the code from where the set/get function is called. You can select only one item at a time. The following table describes the columns of the *Access History* pane.

Column	Description
Action	Displays the <code>set/get</code> history of the resource item selected in the <i>Resource View</i> pane.
Scope	Displays the actions of the scope patterns that match the selected scope. Double-click this item to view the source line of the set/get call in the <i>Source Code</i> pane
Value	Displays the <code>value</code> or <code>val</code> argument of a function
Time	Displays the time at which action is performed
Accessor	Displays the objects that set or get the resource/configuration information

The *Resource View* toolbar is hidden by default. To view the toolbar, right-click on the *Resource View* header and select **toolbar** as shown in the following figure:

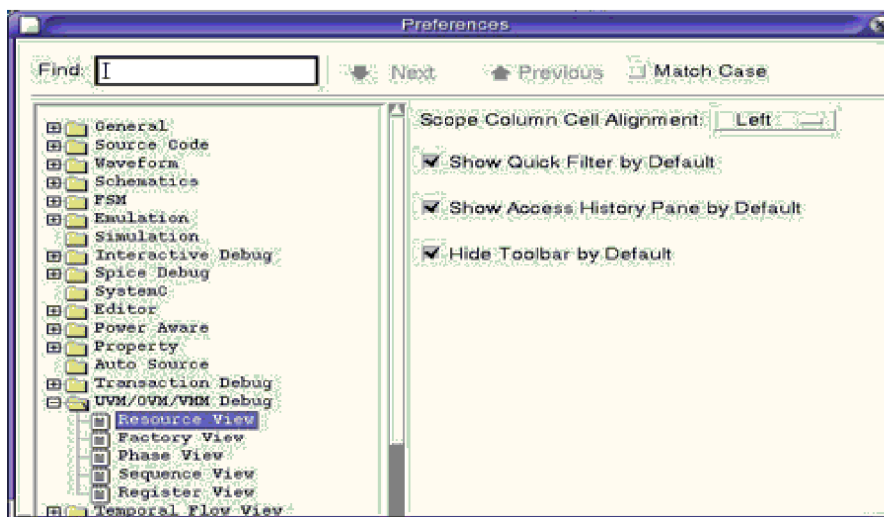
Figure 37 Viewing Toolbar



You can click the **Quick Filter** and **Resource Access History** icons to show/hide the *Quick Filter* and *Access History* panes.

You can use the *Resource View* page of the *Preferences* form (invoked with **Tools -> Preferences**) to customize the display of the *Resource View*.

Figure 38 Resource View Preference Setting



The following sections describe the features and usage for the *Resource View*.

Note:

With the addition of the Verdi OVM library, the options available in the *Resource View* for UVM are also available for OVM.

The following tasks are included:

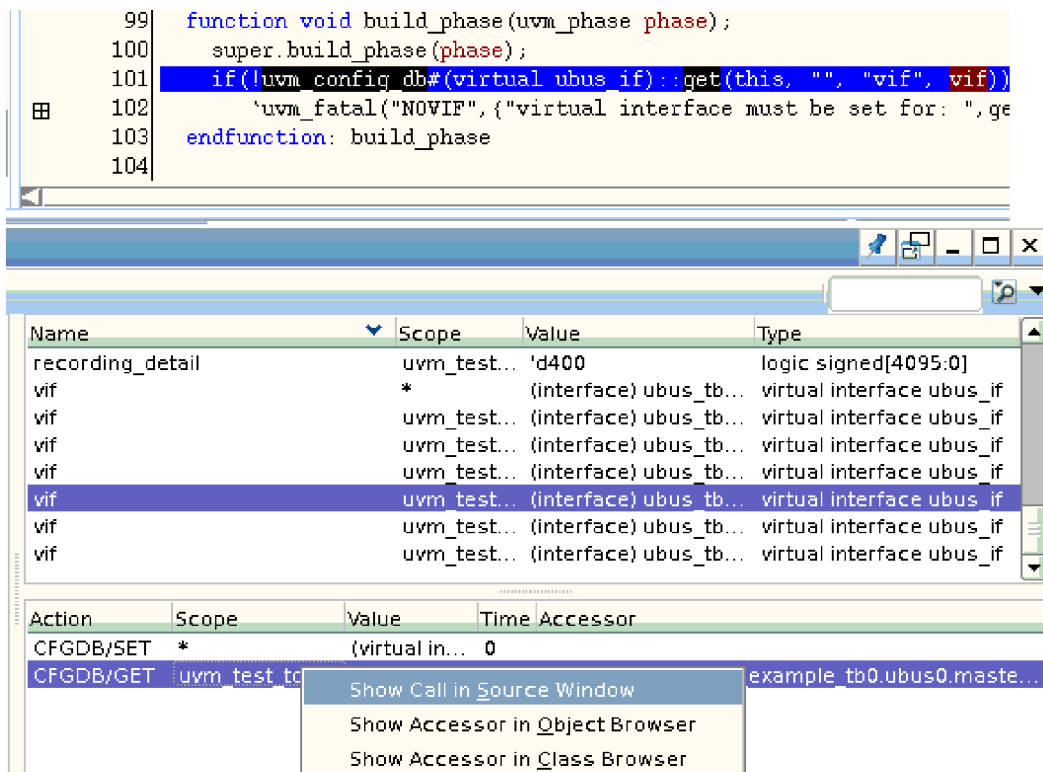
- [Viewing the Source Code and Accessor](#)
- [Applying Quick Filter in the Resource View](#)

Viewing the Source Code and Accessor

Perform the following steps to view the source code of a resource or configuration:

1. Select a resource or configuration name in the *Resource View* pane, the history of the corresponding item is displayed in the *Access History* pane as shown in [Figure 36](#).
2. Click the **Show Call in Source Window** option in the right-click option menu and double-click the history in the *Access History* pane to view the code where the `set/get` function is called in the *Source Code* pane. You can select only one item at a time.

Figure 39 Viewing the History and Source Code



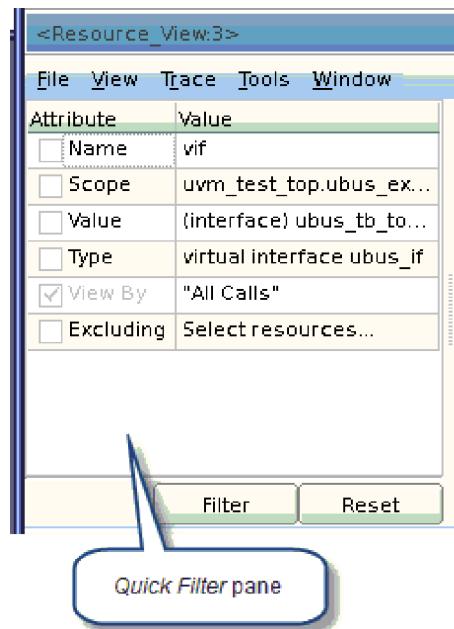
3. To view the accessor class or object of the selected function in the *Class Browser* or *Object Browser* panes, use the **Show Accessor in Class Browser** or **Show Accessor in Object Browser** options in the right-click menu, you can see the line of the function-called location in the *Source Code* pane and the accessor of this function in the *Object Browser* pane.
4. For virtual interface resources, select an item with the **Virtual Interface** option in the *Resource View* pane and use the **Show Source for Virtual Interface** option in the right-click option menu to view the definition code in the *Source Code* pane.

Applying Quick Filter in the Resource View

You can use the *Quick Filter* pane to filter the resource or configuration based on the attribute type.

Select an item in the *Resource View* pane, the attributes of this item are displayed in the *Quick Filter* pane, as shown in [Figure 40](#). You can choose these attributes as the filtering input to be applied to the entire *Resource View* pane. The **Value** column is editable for the **Name**, **Scope**, **Value**, and **Type** attributes as needed.

Figure 40 Quick Filter

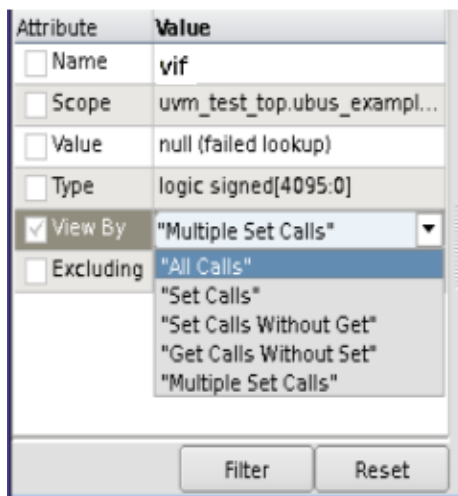


The value of the **View By** attribute is **All Calls** by default. As shown in [Figure 41](#), the value of this attribute can be selected in the drop-down menu.

Note:

The attribute **View By** is checked by default and cannot be unchecked.

Figure 41 Filtering Options

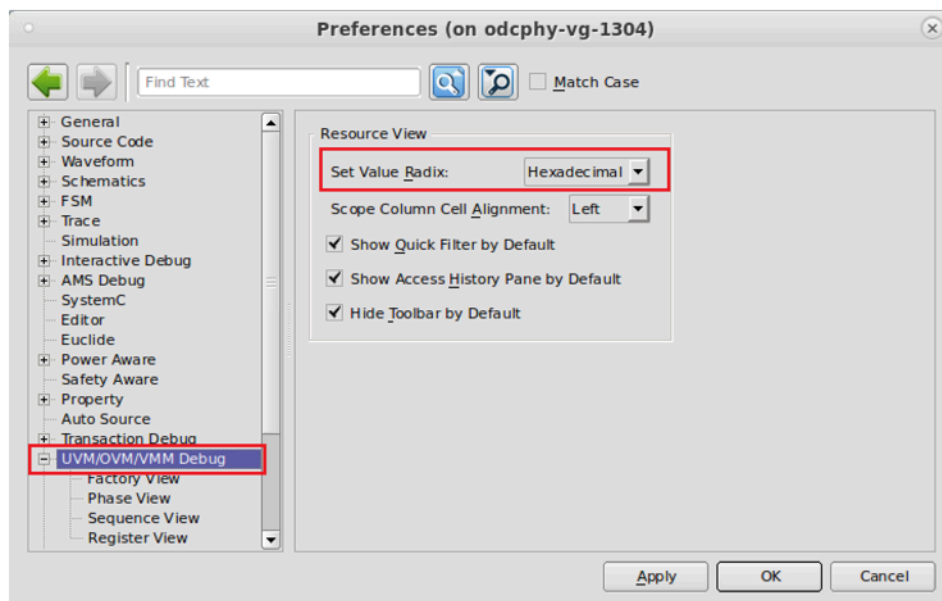


Displaying Radix in the Resource View

You can set the radix from the *Resource View* preference page as follows:

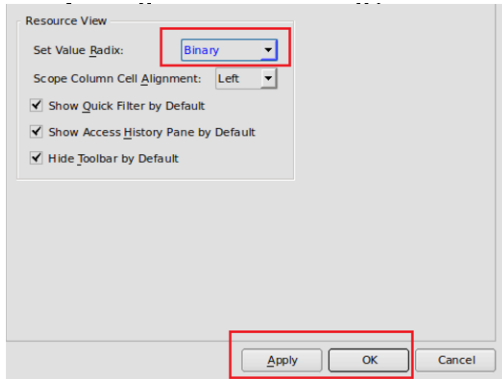
Tools -> Preferences -> UVM/OVM/VMM Debug -> Resource View *page* -> Set Value Radix

Figure 42 *Setting Radix From the Resource View Page*



The Hexadecimal option is displayed by default for Set Value Radix. You can use the drop-down list to select the desired option. The selected option is displayed in blue color as shown in the following figure:

Figure 43 Setting Radix Value in the Resource View



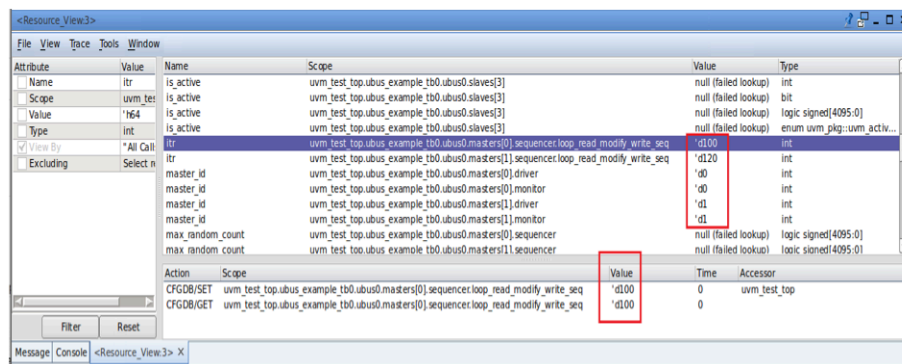
Click **Apply** and then **OK** to apply the changes in the *Resource View*.

Radix Specifier

Verdi always displays the radix specifier in the *Resource View* pane as follows:

Radix	Specifier
Decimal	<code>`d</code>
Binary	<code>`b</code>
Octal	<code>`o</code>
Hexadecimal	<code>`h</code>

Figure 44 Viewing Radix Specifier in the Resource View Page

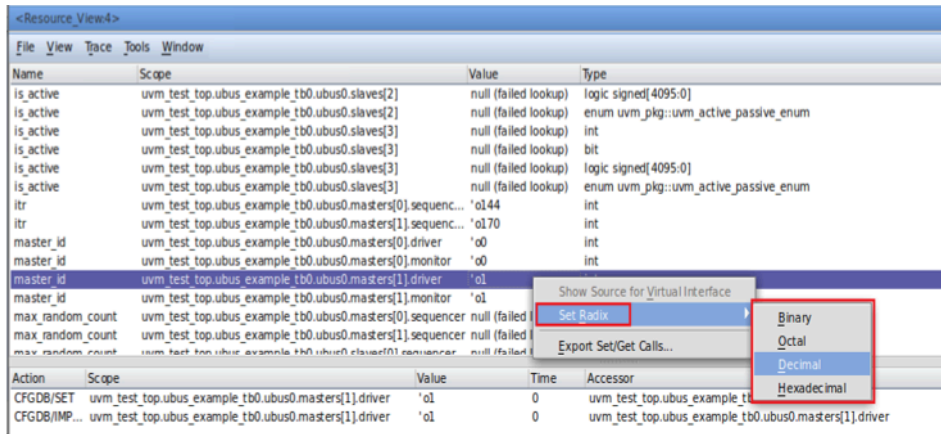


Setting Radix Value for the Selected Resource Item

You can set the radix value for the selected resource item as follows:

- Using the **Set Radix** right-click option:
RMB -> Set Radix -> Binary | Octal | Decimal | Hexadecimal

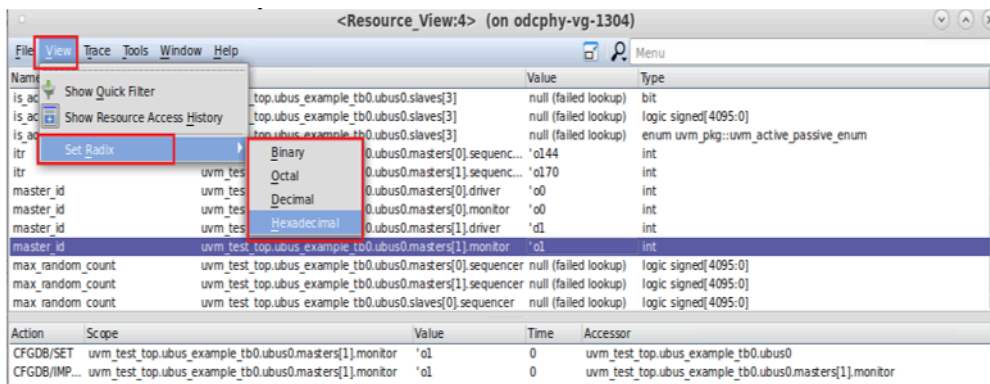
Figure 45 Setting Radix for an Item in the Resource View



- Using the **Set Radix** option in the *Resource View* toolbar:

View -> Set Radix -> Binary | Octal | Decimal | Hexadecimal

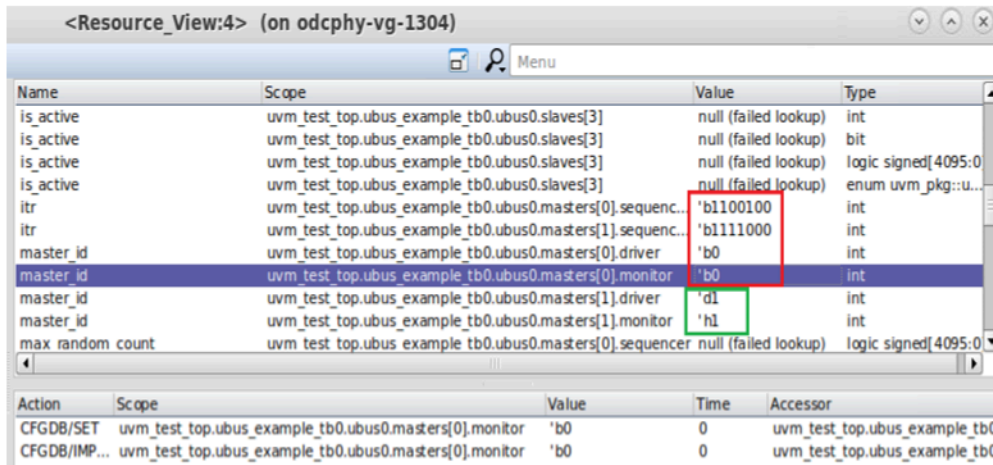
Figure 46 Setting Radix for an Item From the Resource View Toolbar



The changed radix is not affected by the Preference page settings. This radix value is kept until the *Resource View* is closed. For example, in the following figure, if the radix in the green box is already changed using the **Set Radix** option from RMB or toolbar,

then changing the radix to binary using the Preference page will not have any impact on the radix. The value in the green box maintains its original radix.

Figure 47 Setting Radix for an Item in the Resource View



Setting Radix Value Using Tcl Commands

You can use Tcl commands to set the radix value for the selected resource item as follows:

Radix to Set	Tcl Command
Decimal	uvmSetPreference -rsrcValueRadix Dec
Binary	uvmSetPreference -rsrcValueRadix Bin
Octal	uvmSetPreference -rsrcValueRadix Oct
Hexadecimal	uvmSetPreference -rsrcValueRadix Hex

Factory View

The *Factory View* provides an interface to view the list of all classes or overridden classes that define the UVM factory facility. It also provides a filtering functionality to filter the desired factories.

Note:

With the addition of the Verdi OVM library, the options available in the *Factory View* for UVM are also available for OVM.

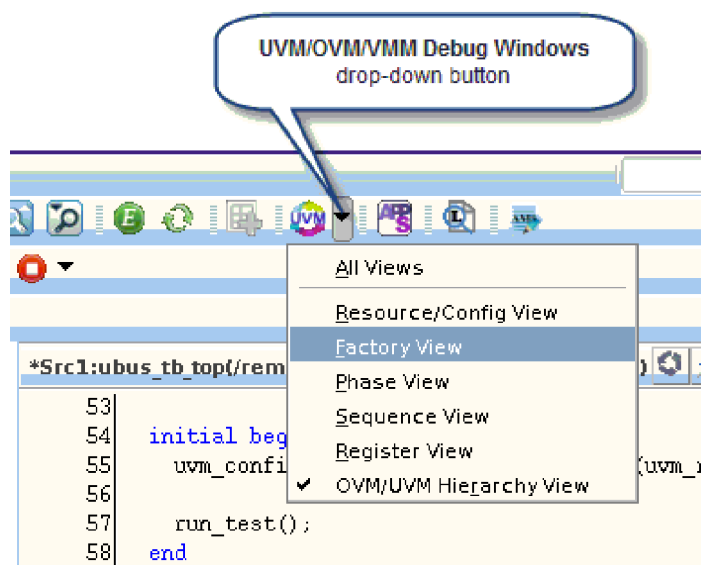
This section describes the *Factory View* in the following topics:

- [Viewing the Factory View](#)
- [Viewing a Class in the Class Browser and Source Code Panes](#)
- [Applying Filter Views in the Factory View](#)

Viewing the Factory View

To view the *Factory View*, click the **UVM/OVM/VMM Debug Windows** drop-down button and select the **Factory View** option from the drop-down menu as shown in the following figure:

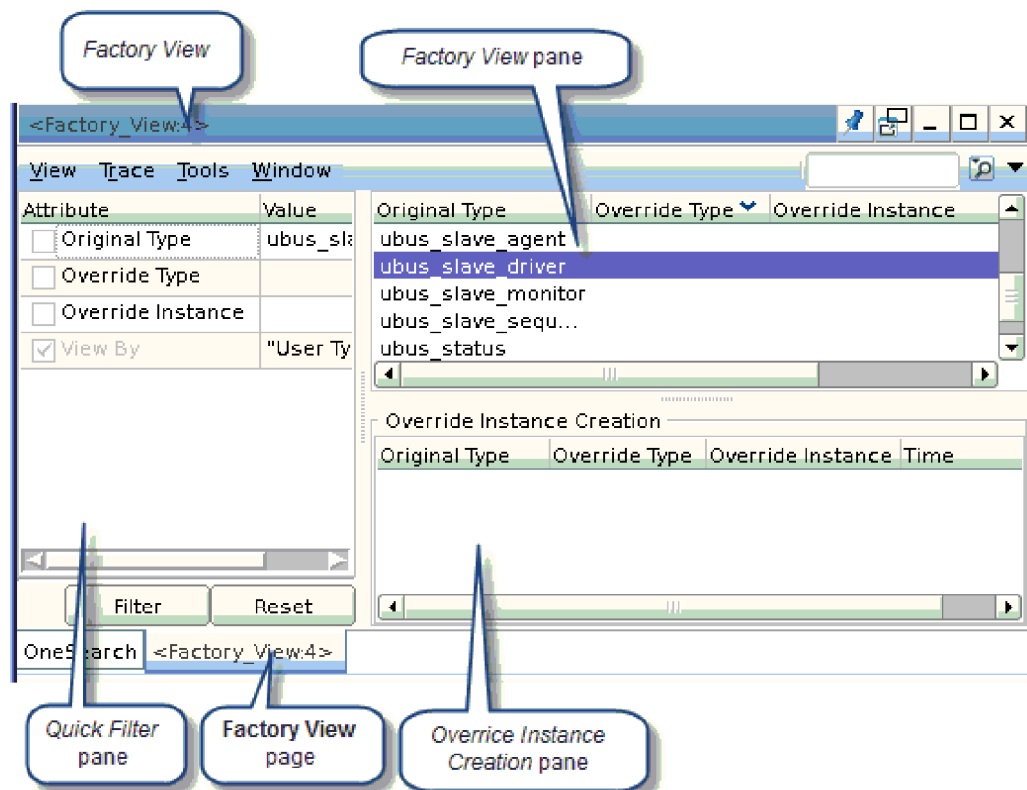
Figure 48 Viewing the Factory View



You can view the *Factory View* in the **Factory View** page of the *UVM Debug* pane. This view displays all classes or overridden classes and updates for each simulation stop. It

contains *Factory View* pane, *Override Instance Creation* pane, and *Quick Filter* pane as shown in the following figure:

Figure 49 Factory View



By default, the following data types are displayed in the *Factory View* pane:

- The original class types that are overridden. The data is retrieved from the FSDB file.
- The class types that are registered in the factory. The data is retrieved from the simulator.

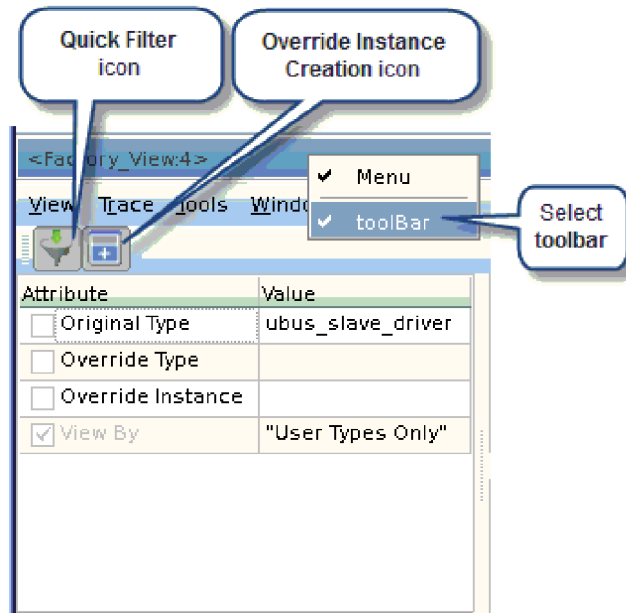
The following table describes the columns of the *Factory View* pane. You can sort the data in each column by clicking the column name.

Column	Description
Original Type	Displays the list of original types
Override Type	Displays the type of instance override
Override Instance	Displays instance overrides

The *Override Instance Creation* pane is displayed by default. This view displays the class instances generated by the override class selected in the *Factory View* pane. The **Time** column shows the time at which the action is performed.

The *Factory View* toolbar is hidden by default. To view the toolbar, right-click on the *Factory View* header and select *toolbar* as shown in the following figure:

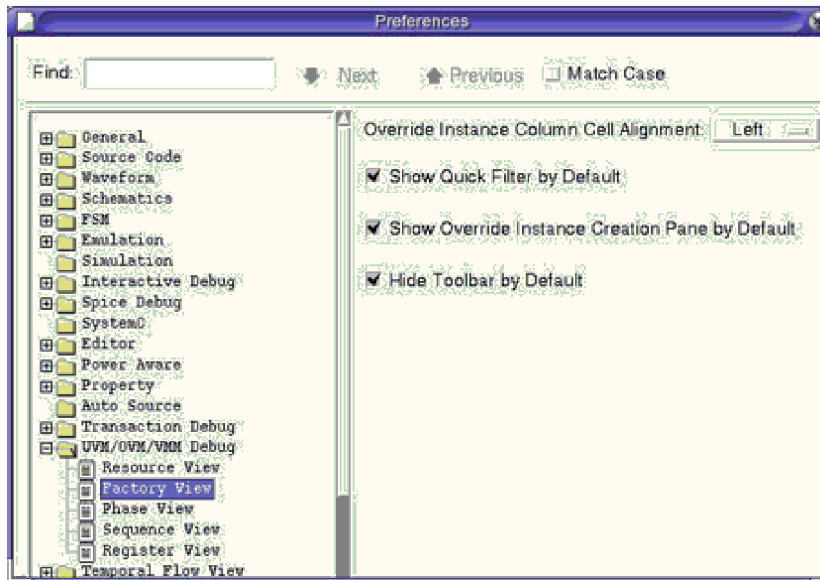
Figure 50 Viewing Toolbar



You can click the **Quick Filter** and **Override Instance Creation** icons to show/hide the *Quick Filter* pane and *Override Instance Creation* pane.

You can use the *Factory View* page of the *Preferences* form (invoked with **Tools -> Preferences**) to customize the display of the *Factory View*.

Figure 51 Factory View Preference Setting



The following sections describe the features and usages of the *Factory View*:

- [Viewing a Class in the Class Browser and Source Code Panes](#)
- [Applying Filter Views in the Factory View](#)

Viewing a Class in the Class Browser and Source Code Panes

Select an overridden class in the *Factory View* pane, the instances of this class are displayed in the *Override Instance Creation* pane.

The following table describes the options in the right-click option menu. These options are used to check the original or overridden classes.

Option	Description
Show Original type in Class Browser	Displays the original class of the selected class in the <i>Class Browser</i> pane
Show Override type in Class Browser	Displays the overridden class of the selected class in the <i>Class Browser</i> pane
Show Declaration of Original Type	Displays the source line of the selected original class in the <i>Source Code</i> pane
Show Declaration of Override Type	Displays the source line of the selected override class in the <i>Source Code</i> pane

Option	Description
Show Reference of Override Setup	Displays the source line of the selected override instance in the <i>Source Code</i> pane

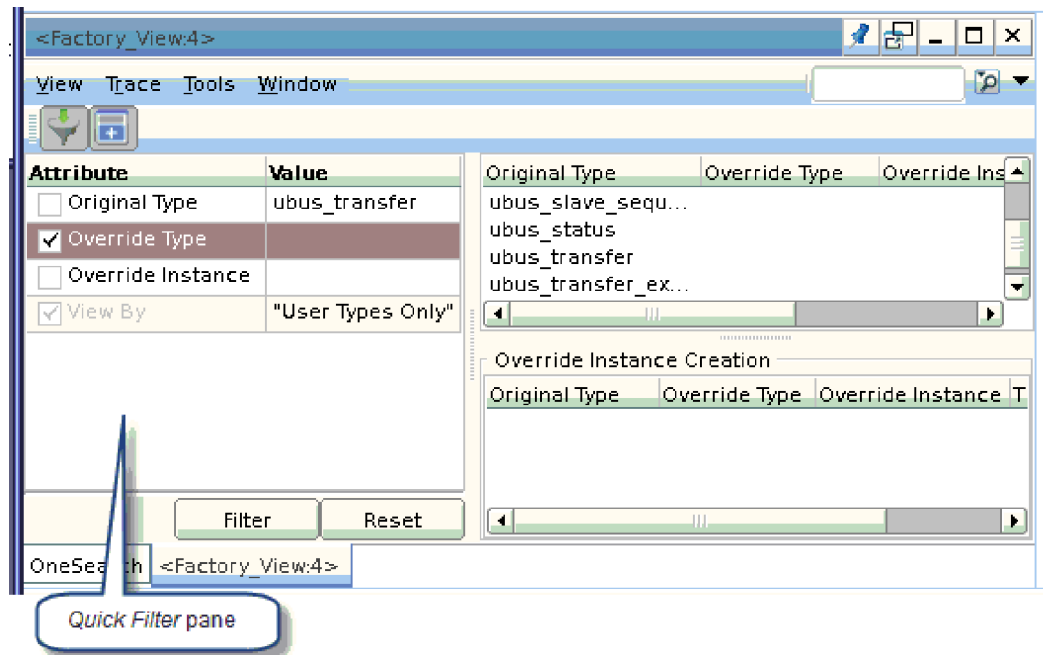
For the instances in the *Override Instance Creation* pane, you can use the following options in the right-click menu:

Option	Description
Show Reference of Instance Creation	Displays the source line of the selected instance creation in the <i>Source Code</i> pane
Show Definition of Override Instance	Displays the source line of the selected overridden instance in the <i>Source Code</i> pane

Applying Filter Views in the Factory View

The *Quick Filter* pane allows you to configure the type of information to be displayed for the **Original Type**, **Override Type**, and **Override Instance** columns in the *Factory View* pane.

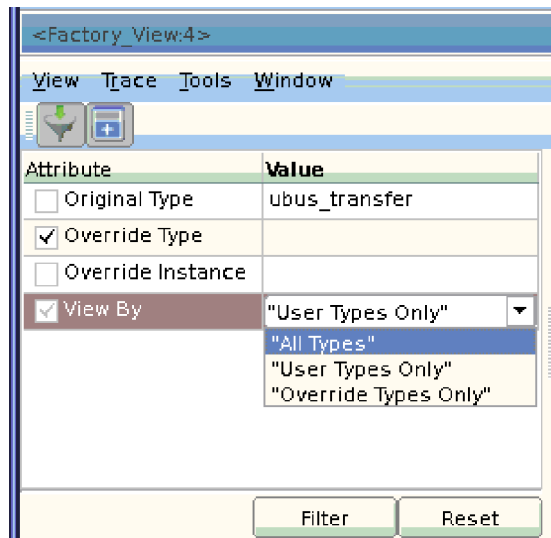
Figure 52 Quick Filter Pane



The default value of the **View By** attribute is **User Types Only**, see [Figure 53](#). You can select the value of this attribute from the drop-down menu.

Use the **All Types**, **User Types Only** or **Override Types Only** option to display all classes, user-defined classes, or overridden classes in the *Factory View* pane.

Figure 53 Filtering Options in the Quick Filter



Note:

The **View By** attribute is selected by default. You cannot deselect this attribute.

Phase View

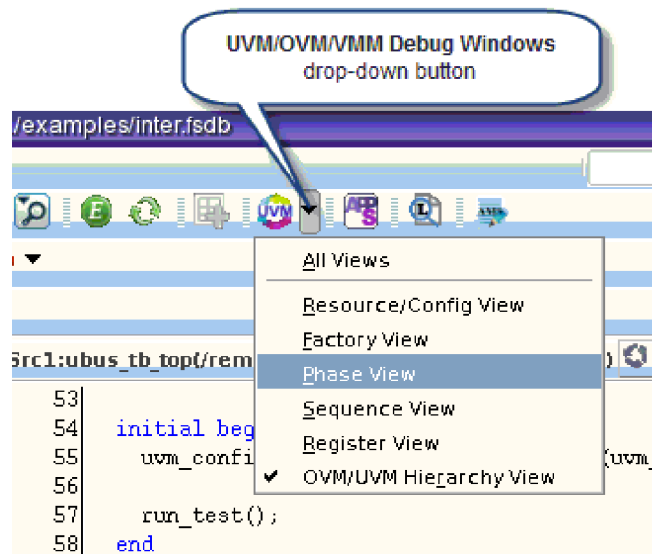
The *Phase View* displays all the predefined phases of common and UVM domain. This section describes the *Phase View* in the following topics:

- [Viewing the Phase View](#)
- [Viewing an Object in Different Panes](#)
- [Setting Breakpoints at a Specific Phase](#)
- [Applying Quick Filter in the Phase View](#)

Viewing the Phase View

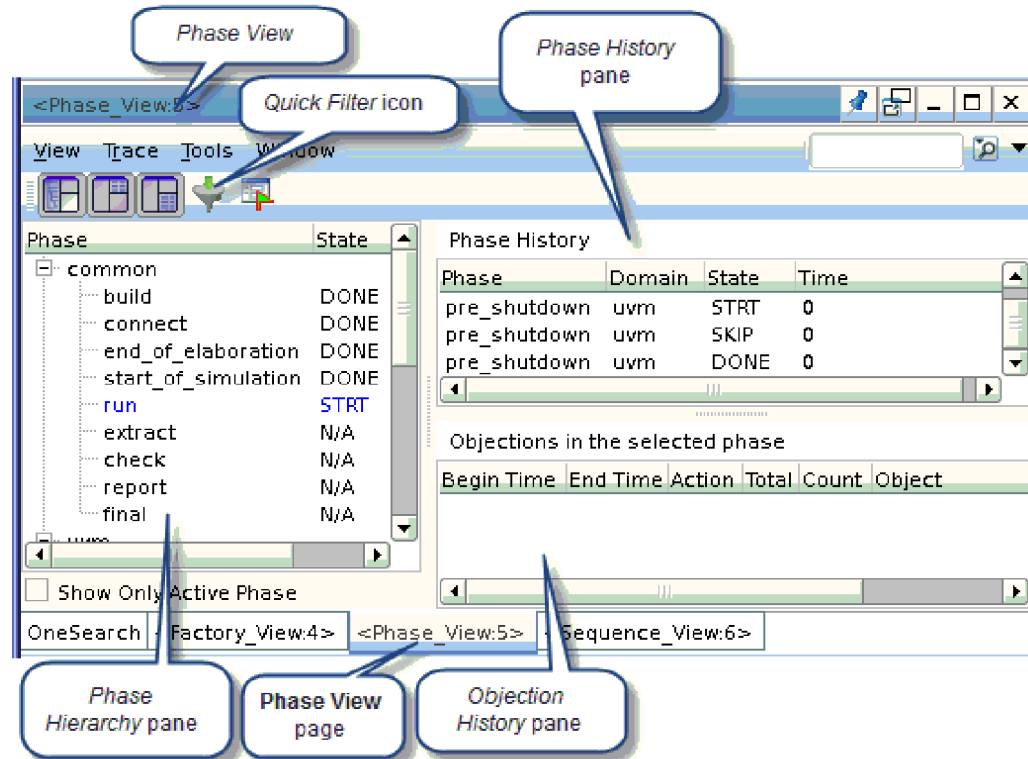
To view the *Phase View*, click the **UVM/OVM/VMM Debug Windows** drop-down button and select the **Phase View** option from the drop-down menu as shown in the following figure:

Figure 54 Viewing the Phase View



You can view the *Phase View* in the **Phase View** page of the *UVM Debug* pane. It contains *Phase History* pane, *Objection History* pane, and *Phase Hierarchy* pane as shown in the following figure:

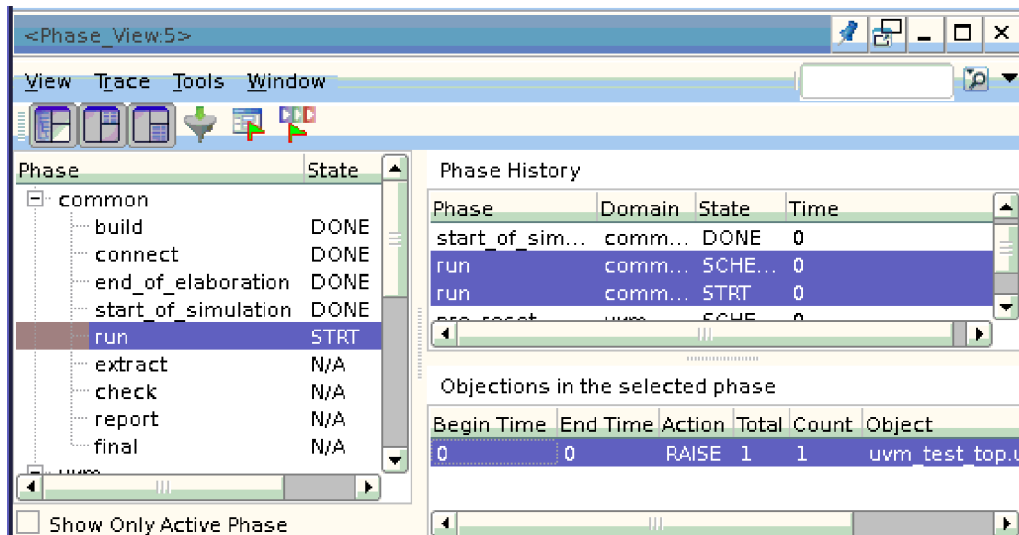
Figure 55 *Phase View*



The *Phase Hierarchy* pane displays all the predefined and user-defined phases of common and UVM domain. You can select the **Show Only Active Phase** check box to view only the executing phases.

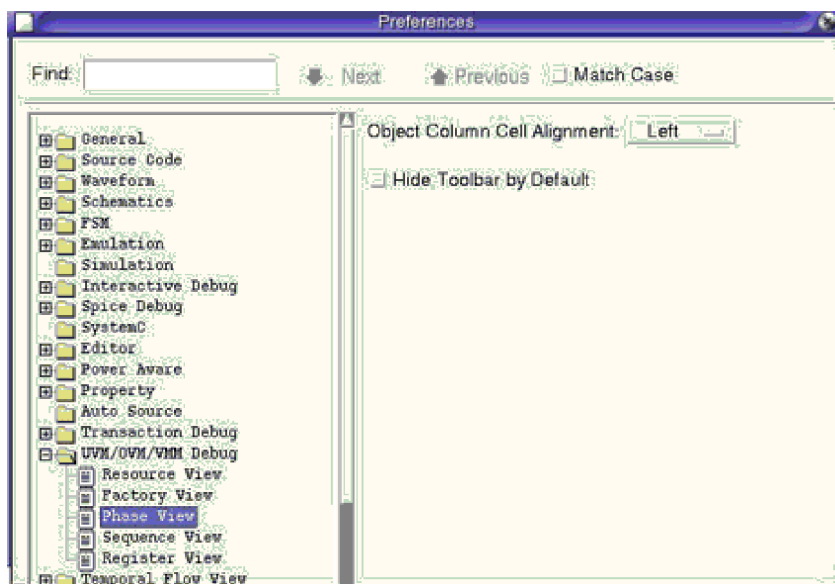
The *Phase History* pane displays the information of the phase selected in the *Phase Hierarchy* pane as shown in [Figure 56](#).

Figure 56 Phase History Pane



You can use the *Phase View* page of the *Preferences* form (invoked with **Tools -> Preferences**) to customize the display of the *Phase View*.

Figure 57 Phase View Preference Setting



The following sections describe the features and usages of the *Phase View*:

- [Viewing an Object in Different Panes](#)
- [Setting Breakpoints at a Specific Phase](#)
- [Applying Quick Filter in the Phase View](#)

Viewing an Object in Different Panes

Perform the following steps to view the details of the object in various panes:

1. Select an objection item in the *Objection History* pane and double-click on the item to view the source code in the *Source Code* pane.

The **Show Source** option in the right-click option menu displays the declaration of the selected objection source code.

2. Click the **Show Object in Object Browser** and **Show Object in Class Browser** options to view the selected objection in the *Object Browser* and *Class Browser* panes.

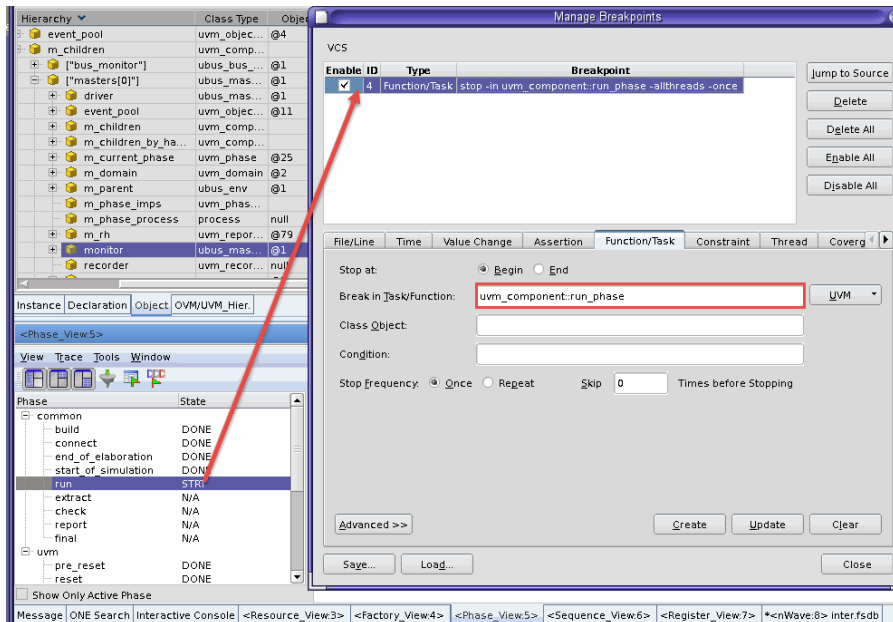
Setting Breakpoints at a Specific Phase

Perform the following steps to set a breakpoint at a specific phase:

1. Select a phase and click the  icon or use the **Set Breakpoints for Phase** right-click option menu option to add the breakpoint for the phase. Click **Create** to create the breakpoint.

The **Manage Breakpoints** form is invoked and the selected phase function is added in the **Break in Task/Function** field automatically. As shown in [Figure 58](#), the corresponding phase function of `uvm_component` (for example, `uvm_component::run_phase`) is automatically entered in the **Break in Task/Function** field.

Figure 58 Adding a Breakpoint



2. Click the **UVM** button to open the UVM menu and create breakpoint in the *Manage Breakpoints* form for other phases as shown in Figure 59. The UVM component breakpoints are divided into seven phases, namely, `uvm_root::phase_started`, `uvm_root::build_phase`, `uvm_root::run_phase`, Build, Run, Cleanup, and Report. The last four categories also contain a list of phase methods that is used as a breakpoint.

You can directly select the `uvm_root:phase_started`, `uvm_root::build_phase`, and `uvm_root::run_phase` options to stop the simulation execution in these tasks.

Note:

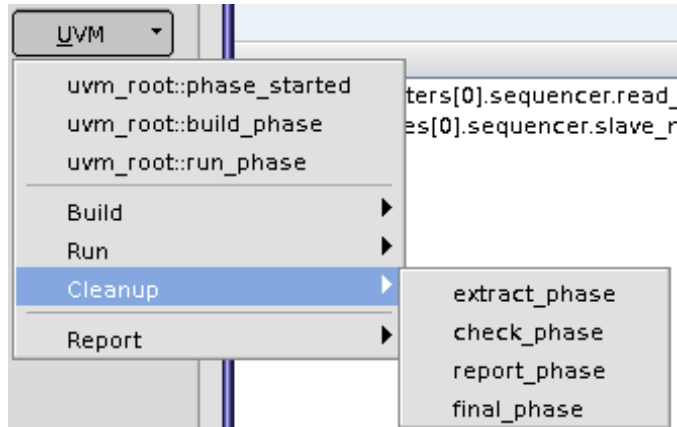
When a breakpoint is set up using the *Phase View*, the **-once** option is enabled by default.

3. Select a task in the **Build, Run, Cleanup**, or **Report** stage to set a breakpoint at that phase.

For example, to stop the simulation execution in the `report_phase` task, click **Cleanup** and select the `report_phase` task to add the `report_phase` task in the **Break in Task/Function** field. Click the **Create** button to add this breakpoint.

For more information on adding breakpoints, see the *SVTB Interactive Simulation Debug User Guide*.

Figure 59 Select the Cleanup Phase in the Menu



Applying Quick Filter in the Phase View

Perform the following steps to apply the quick filter in the *Phase View*:

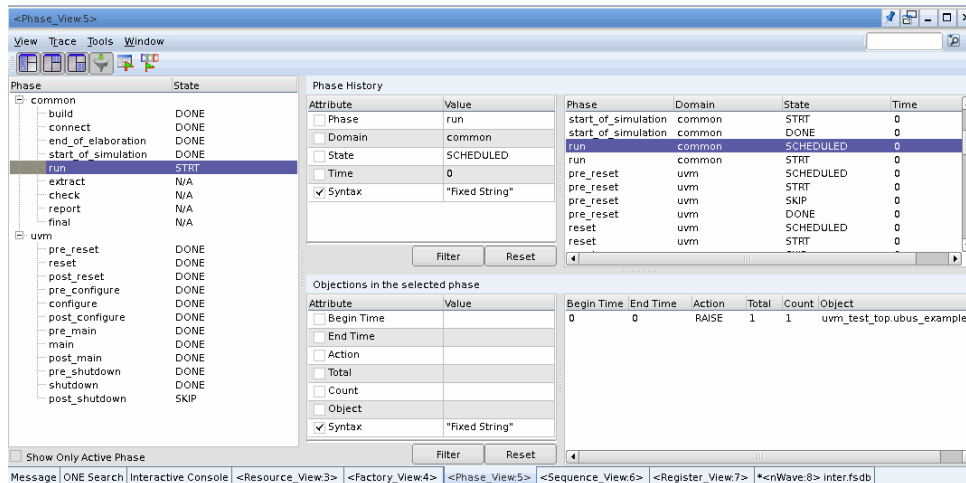


1. Click the  icon to invoke the *Quick Filter* pane.
2. Select an item in the *Phase History* pane. The attributes of the selected item are displayed in the *Quick Filter* pane.

Note:

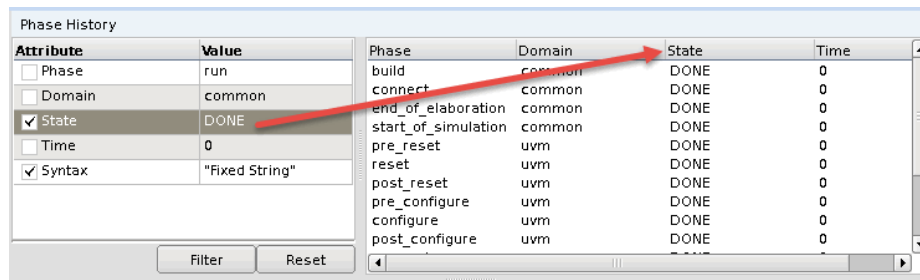
Clicking the **Quick Filter** icon on the toolbar enables the quick filtering feature on both the *Phase History* pane and *Objection History* pane.

Figure 60 Filtering Attributes in the Corresponding Panes



The usage is the same as the description in the [Applying Quick Filter in the Resource View](#). The following figure shows the results of filtering the phase history with the condition **State equals to DONE** in the *Phase History* pane as an example.

Figure 61 Filtering Option in the Phase History Pane



Sequence View

The *Sequence View* allows you to analyze the sequence execution with history, and links this with other debug features. This view displays the sequence groups in the tree view. This section describes the *Sequence View* in the following topics:

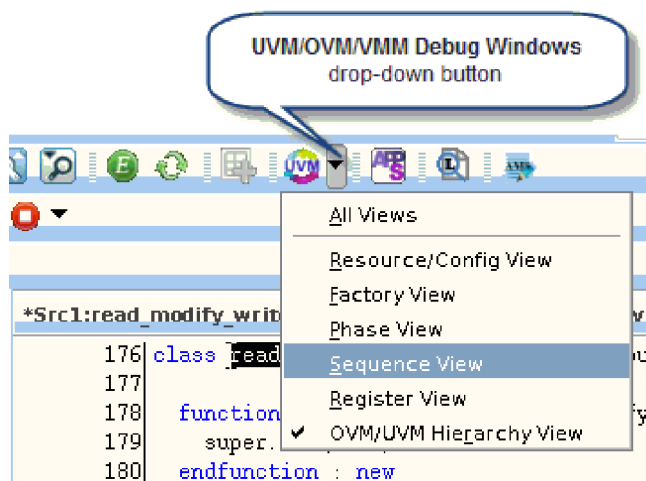
- [Viewing the Sequence View](#)
- [Viewing the Sequence in the Interactive Debug Panes](#)

- [Viewing the Reference of a Sequence](#)
- [Setting Breakpoint at Specific Sequence](#)
- [Adding a Sequence to the Waveform](#)
- [Adding a Sequence to the Transaction and Protocol Analyzer or Transaction Browser](#)
- [Applying Quick Filter View in Sequence View](#)

Viewing the Sequence View

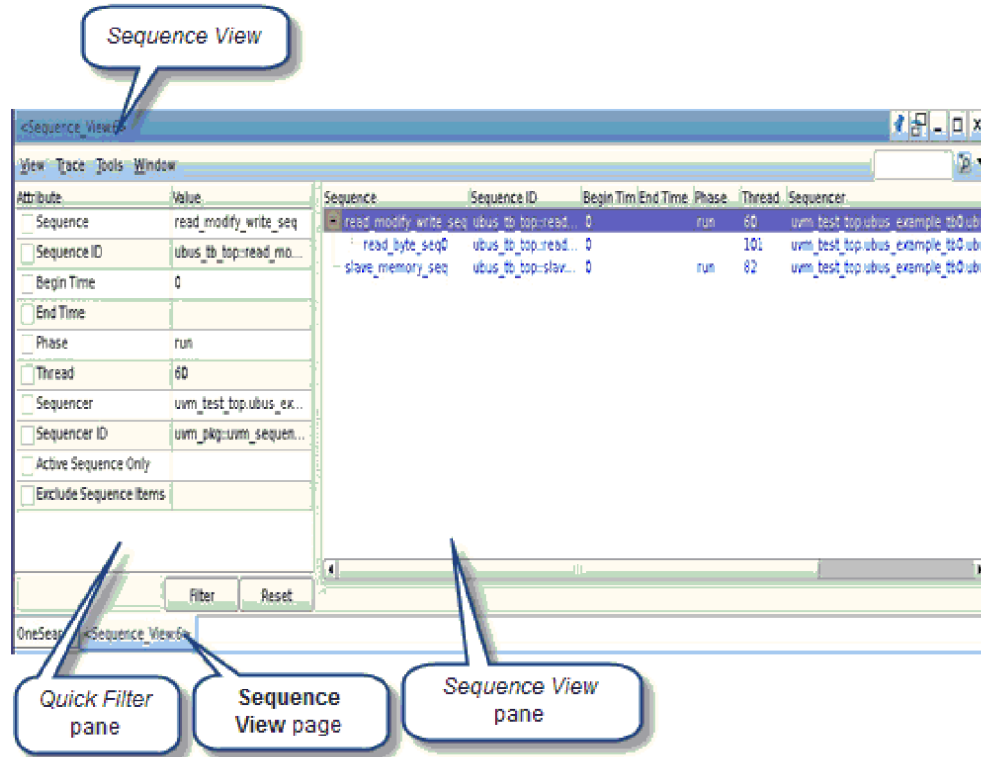
To view the *Sequence View*, click the **UVM/OVM/VMM Debug Windows** drop-down button and select the **Sequence View** option from the drop-down menu as shown in the following figure:

Figure 62 Viewing the Sequence View



You can view the *Sequence View* in the **Sequence View** page of the *UVM Debug* pane. It contains *Sequence View* pane and *Quick Filter* pane as shown in the following figure:

Figure 63 Sequence View



The *Sequence View* pane displays all the information related to the active and transaction-based sequences and sequence items in the UVM/OVM flow. The sequences are displayed in a hierarchical tree structure which represents the parent/child relationship of the sequences.

Note:

If you want to view the sequence history in the *Sequence View* pane, specify the `+UVM_TR_RECORD` option to enable transaction recording. This option allows you to see the inactive sequences in the *Sequence View* pane.

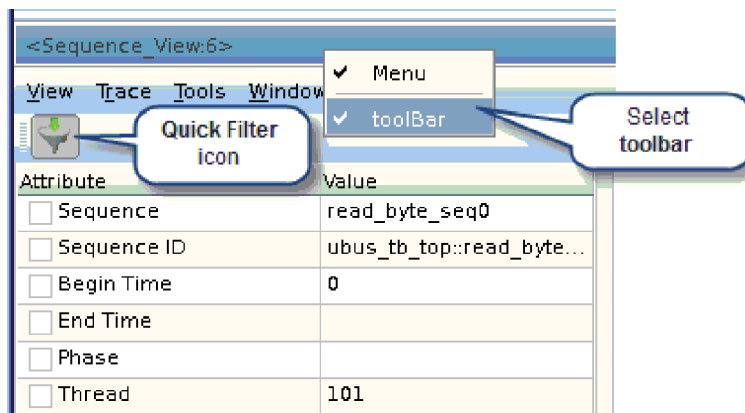
The following table describes the columns of the *Sequence View* pane. You can sort the data in each column by clicking the column name.

Column	Description
Sequence	Displays the list of sequences
Sequence ID	Displays the object ID of a sequence

Column	Description
Begin Time	Displays the start time of a sequence
End Time	Displays the end time of a sequence
Phase	Displays the phase name and the domain of a sequence
Thread	Displays the thread executing the sequence
Sequencer	Displays the sequencer initiating the sequence
Sequencer ID	Displays the object ID of a sequencer

The toolbar is hidden by default. To view the toolbar, right-click on the *Sequence View* header and select *toolbar* as shown in the following figure:

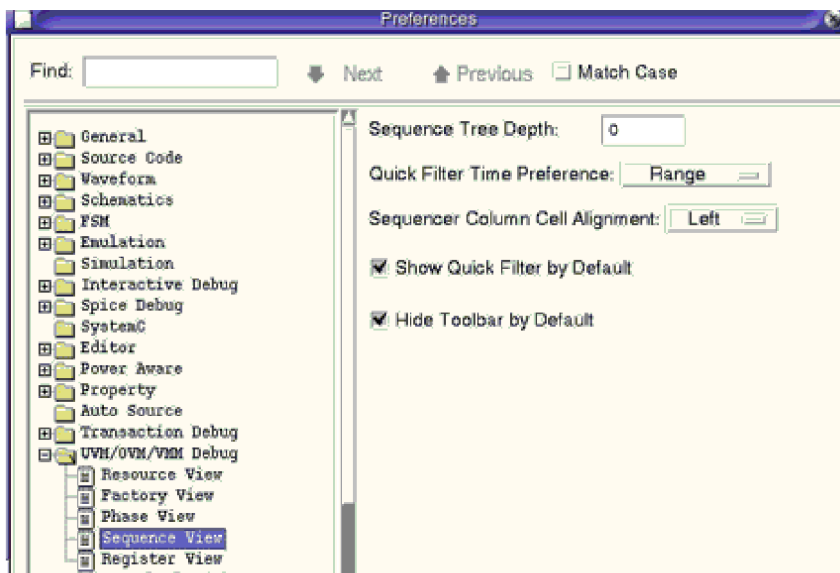
Figure 64 Viewing Toolbar



You can click the **Quick Filter** icon to show/hide the *Quick Filter* pane.

You can use the *Sequence View* page of the *Preferences* form (invoked with **Tools -> Preferences**) as shown in [Figure 65](#) to customize the display of the *Sequence View*.

Figure 65 Sequence View Preference Setting



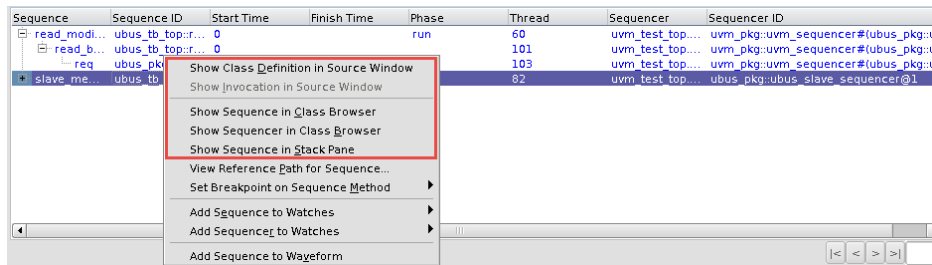
The following sections describe the features and usages of the *Sequence View*:

- [Viewing the Sequence in the Interactive Debug Panes](#)
- [Viewing the Reference of a Sequence](#)
- [Setting Breakpoint at Specific Sequence](#)
- [Adding a Sequence to the Waveform](#)
- [Adding a Sequence to the Transaction and Protocol Analyzer or Transaction Browser](#)
- [Applying Quick Filter View in Sequence View](#)

Viewing the Sequence in the Interactive Debug Panes

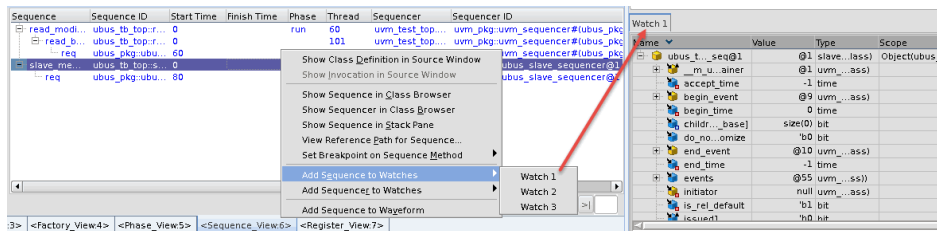
As shown in [Figure 66](#) There are several options to view the class definition, sequence and sequencer in the different panes. For example, you can select a sequence and use the **Show Class Definition in Source Window** option in the right-click option menu to view the class definition in the *Source Code* pane.

Figure 66 Right-click Option Menu Items in the Sequence View Pane



Click the **Add Sequence to Watches** and **Add Sequencer to Watches** options to add the selected sequencer (as shown in the following figure) or sequence object to the **Watch** tab.

Figure 67 Adding a Sequence to the Watch Tab

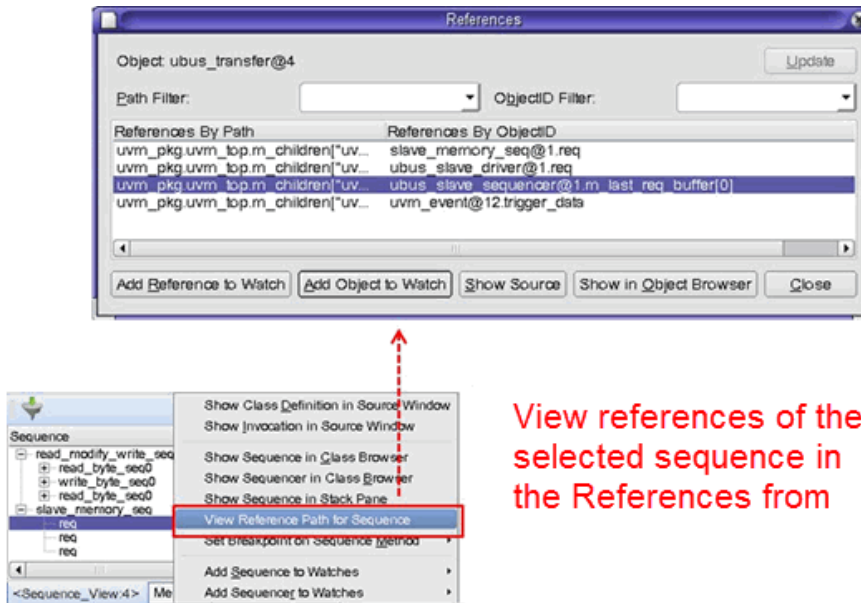


Viewing the Reference of a Sequence

Perform the following step to view the reference of a sequence in the *Sequence View* pane:

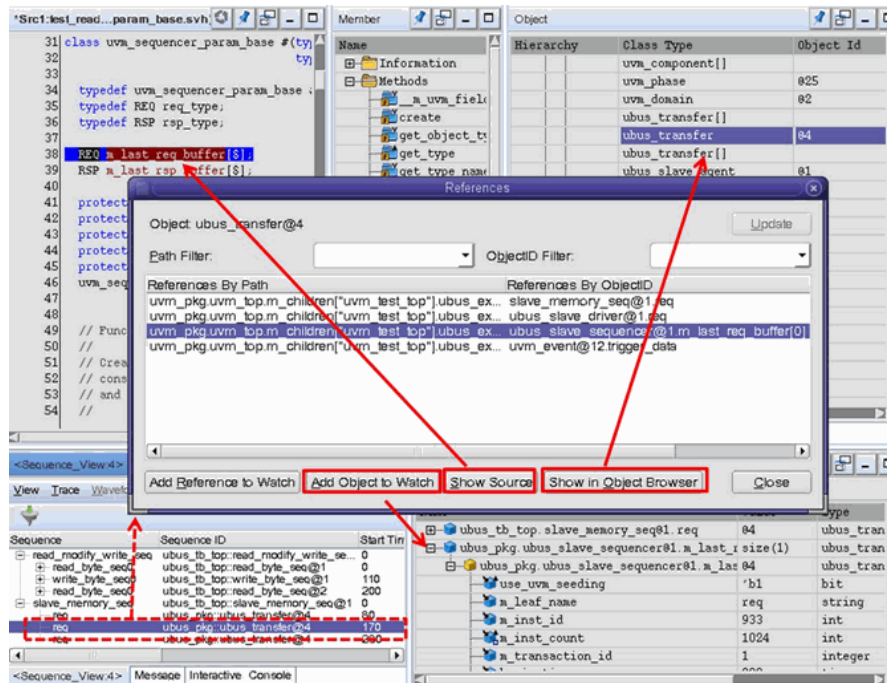
Click the **View Reference Path for a Sequence** option in the right-click option menu to view the references of a selected sequence in the *References* form. As shown in the [Figure 68](#), all the references of the `req` selected in the *Sequence View* pane are shown in the *References* form.

Figure 68 Viewing the Reference Path of a Sequence



As shown in Figure 69, you can view more information about the selected reference in the different panes by using the options in the *References* form.

Figure 69 Viewing More Reference Information



Setting Breakpoint at Specific Sequence

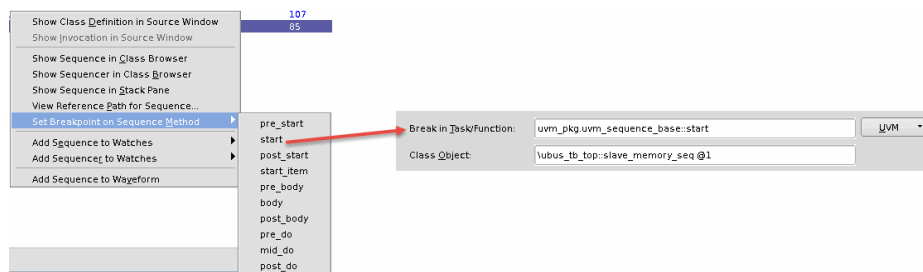
Perform the following step to set a breakpoint at a specific sequence:

Click the **Set Breakpoint on Sequence Method** option in the right-click option menu to create a breakpoint on the sequence method.

For example, select the `read_byte_seq0` sequence and use the **Set Breakpoint on Sequence Method -> Start** option in the right-click option menu to set a breakpoint on the start task.

As shown in the [Figure 70](#), the **Break in Task/Function** field is automatically entered with `uvm_pkg.uvm_sequence_base::start` and the `-object_id \ubus_tb_top::read_byte_seq @1` option is also added to the *Additional Options* field. The following sequence methods are supported: `pre_start`, `start`, `post_start`, `start_item`, `pre_body`, `body`, `post_body`, `pre_do`, `mid_do`, and `post_do`.

Figure 70 Setting a Breakpoint on a Task

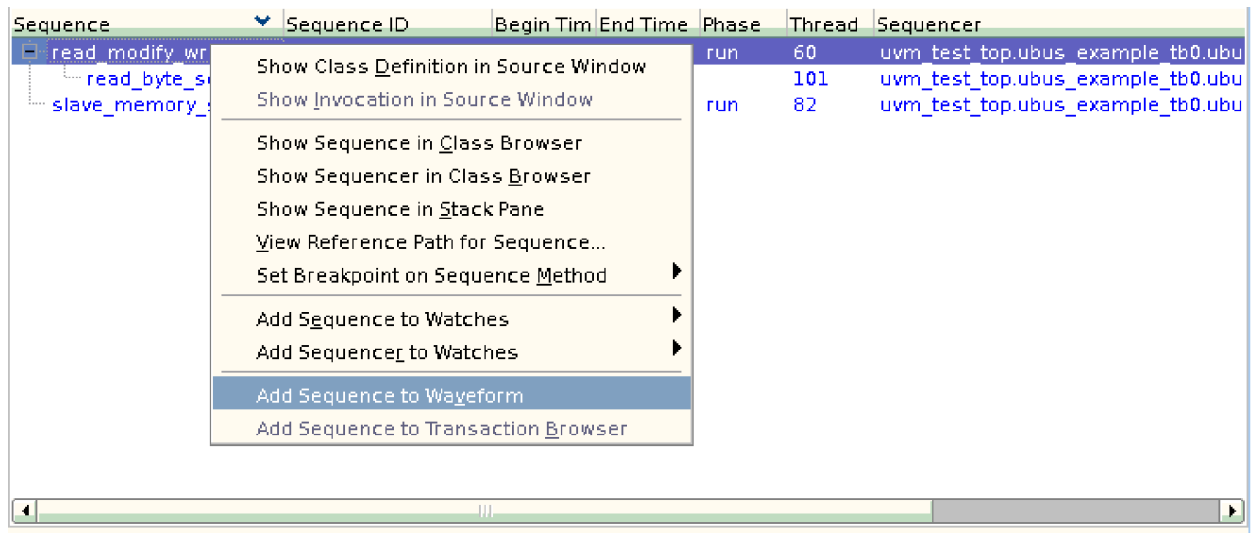


Adding a Sequence to the Waveform

Perform the following steps to add a sequence to the waveform:

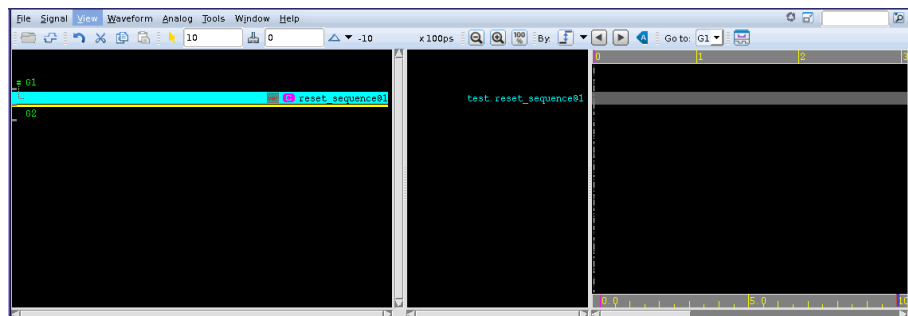
1. Select a sequence in the *Sequence View* pane.
2. Select **Trace -> Add to Waveform** option or alternatively right-click on the selected sequence and select the **Add Sequence to Waveform** option as shown in the following figure:

Figure 71 Adding Sequence to Waveform



On adding the sequence to the waveform, the waveform window is displayed as follows:

Figure 72 Sequence in Waveform



Adding a Sequence to the Transaction and Protocol Analyzer or Transaction Browser

Perform the following steps to add a sequence to the *Transaction Browser* pane or *Transaction and Protocol Analyzer* pane:

Note:

Note: By default, only finished transactions is added to *Transaction Browser*. To be able to add unfinished transactions (for example, transactions that are not ended yet), you must enable **Dump Unfinished Transaction** option from **Tools -> Preferences -> Simulation** form before starting simulation. However, this has some impact on the simulation performance.

1. Select a sequence in the *Sequence View* pane.
2. Select **Trace -> Add Sequence to Transaction Browser** option or alternatively right-click on the selected sequence and select the **Add Sequence to Transaction Browser** option as shown in [Figure 73](#).

If the sequence ID is already in the *Transaction Browser*, it jumps to the selected sequence.

Note:

The **Add Sequence to Transaction Browser** command adds the selected sequence to the *Transaction and Protocol Analyzer* frame. However, if the `INVOKE_UTB_BROWSER` environment variable is set to 0, this command adds the selected sequence object to the *Transaction Browser*.

Figure 73 Adding a Sequence to the Transaction Browser

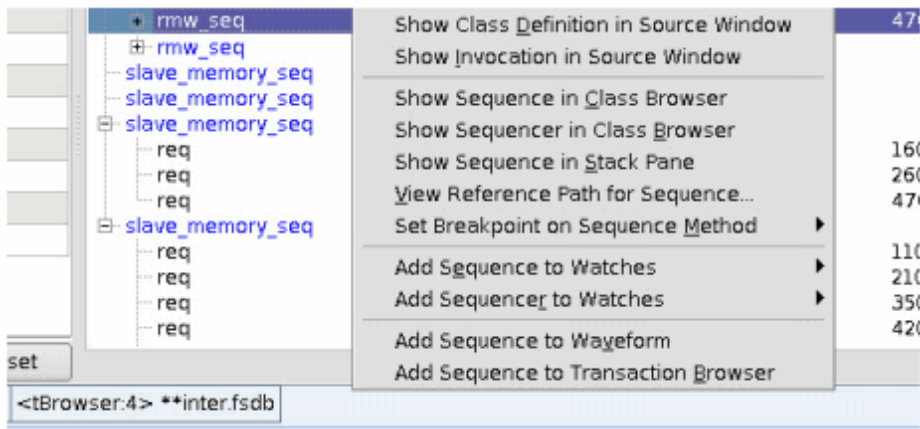
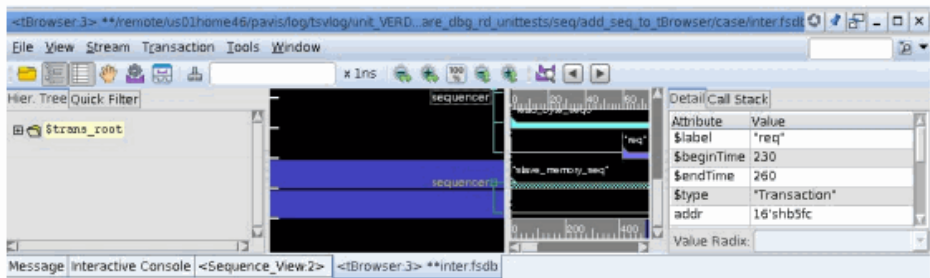


Figure 74 Sequence ID in a Transaction Browser

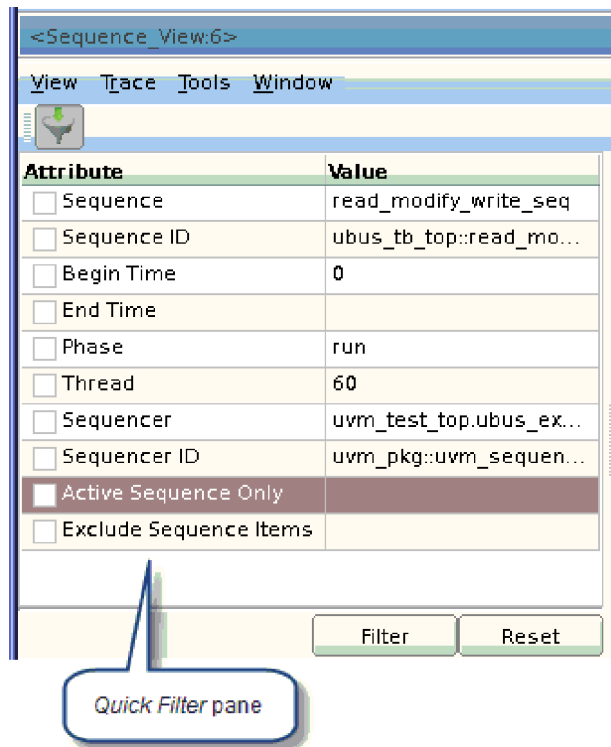


Applying Quick Filter View in Sequence View

Perform the following steps to use the *Quick Filter* pane:

1. Select the filtering attributes in the *Quick Filter* pane.
2. Click the **Filter** button.

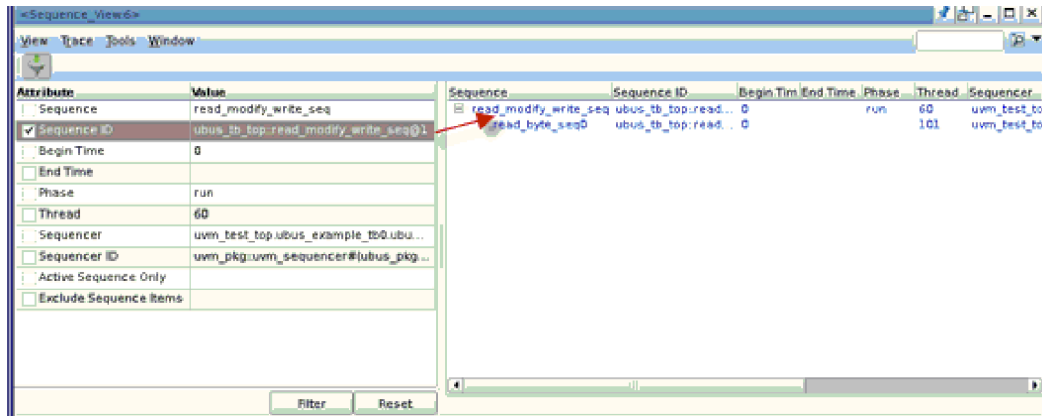
Figure 75 Filtering Attributes in the Quick Filter Pane



You can also select the **Active Sequence Only** attribute to view all the sequences in the simulator at the current simulation time.

The following figure shows the result for filtering sequences with the condition `Sequence ID equals to ubus_tb_top::read_modify_write_seq@1` in the *Sequence View* pane as an example.

Figure 76 Filtering Result in the Sequence View Pane



Register View

A register model in a UVM register layer typically contains hierarchy of blocks that map to the design hierarchy. Register blocks can contain registers, register files, register maps and memories, and other blocks.

Verdi allows you to debug UVM REG based registers and memories using the **Register View** page of the *UVM Debug* pane. This feature allows you to analyze and debug the runtime status of the UVM REG based testbench. You must use the `+UVM_VERDI_TRACE=RAL` option at simulation runtime (for both interactive and post-processing modes) to view register information in the **Register View** page.

For information on usage of the `+UVM_VERDI_TRACE=RAL` option in post-processing mode, see [Dumping UVM Register Information in Post-Processing Mode](#) section.

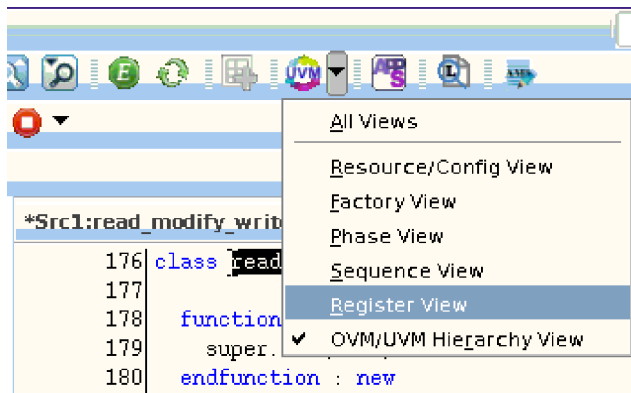
This section describes the *Register View* in the following topics:

- [Viewing the Register View](#)
- [Viewing Registers in Different Panes](#)
- [Operations in the Register Attribute Pane](#)
- [Applying Quick Filter in the Register Attribute and Access History Panes](#)
- [Searching Registers and Register Field Items in the Register View](#)
- [Viewing Registers and Register Fields in the Register Hierarchy Pane by Default](#)
- [Displaying Warning Messages for Incomplete Register Hierarchy](#)
- [Displaying Register Access Status in the Register View](#)

Viewing the Register View

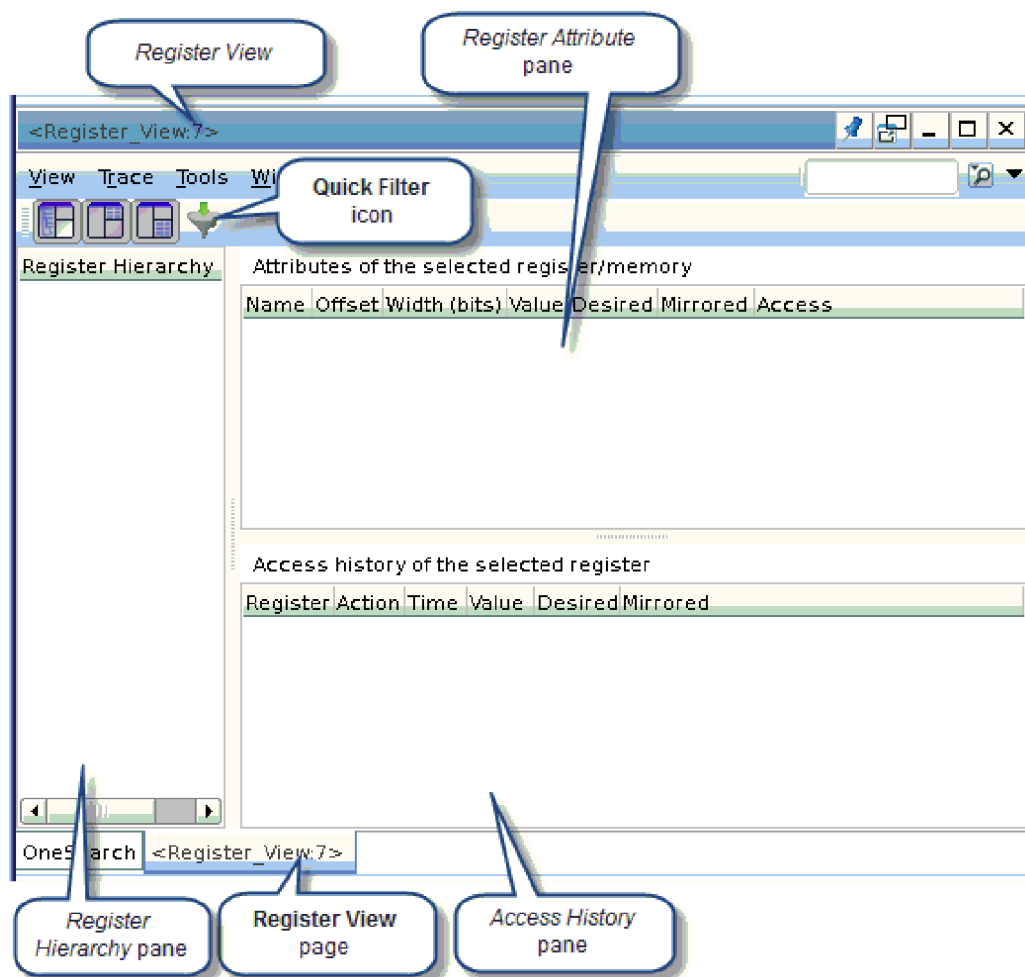
To view the *Register View*, click the **UVM/OVM/VMM Debug Windows** drop-down button and select the **Register View** option from the drop-down menu as shown in the following figure:

Figure 77 Viewing the Register View



You can view the *Register View* in the **Register View** page of the *UVM Debug* pane. It contains *Register Hierarchy* pane, *Register Attribute* pane, and *Access History* pane, as shown in [Figure 78](#).

Figure 78 Register View



The *Register Hierarchy* pane displays the UVM register block hierarchy. The hierarchical structure contains register blocks, register files, and register maps.

The *Register Attribute* pane displays the components (registers/register arrays/memories and its attribute values) of a register block item selected in the *Register Hierarchy* pane.

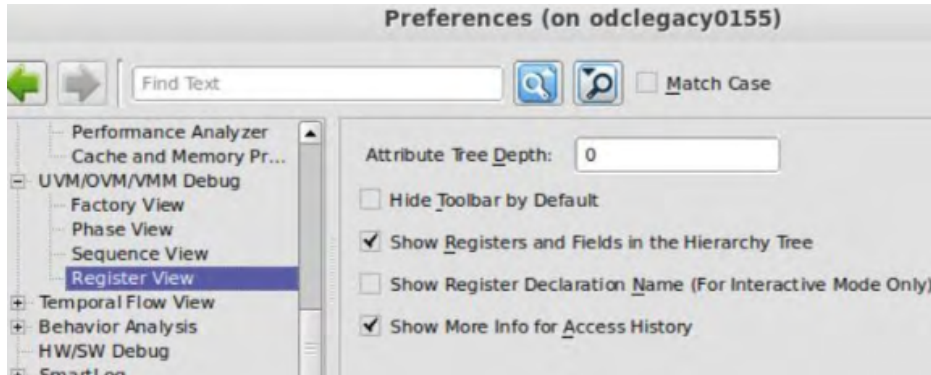
Note:

To view the register read/write access history in the *Register View*, add the `+UVM_VERDI_TRACE=RAL` option at simulation runtime to enable register history dumping.

You can click the **Quick Filter** icon to view the *Quick Filter* panes for the *Register Attribute* pane and *Access History* pane.

You can use the *Register View* page of the *Preferences* form (invoked with **Tools -> Preferences**) to customize the display of the *Register View*.

Figure 79 Register View Preference Setting



Viewing Registers in Different Panes

You can use the following options of the right-click option menu in the *Register Hierarchy* and *Register Attribute* panes to check the register models:

Option	Description
Show Definition	Shows the definition of the selected register model in the <i>Source Code</i> pane. Double-clicking an item has the same effect.
Show Class Definition	Shows the class declaration of the selected register model in the <i>Source Code</i> pane.
Show in Object Browser	Shows the selected register model in the <i>Object Browser</i> pane.
Show in Class Browser	Shows the selected register model in the <i>Class Browser</i> pane.
Show HDL Path	Shows the related HDL of DUT in the <i>Source Code</i> pane. In the Message, a warning message is displayed if the HDL path cannot be found for the register model.

Operations in the Register Attribute Pane

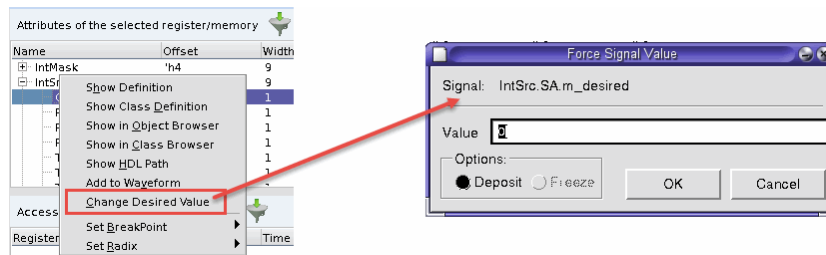
The Verdi platform also provides the following operations in the *Register Attribute* pane for convenient debugging in the register model:

- [Changing Desired Value](#)
- [Setting Breakpoints](#)
- [Setting Radix](#)

Changing Desired Value

Click the **Change Desired Value** option in the right-click option menu to force the desired value of the selected register model. Click the **Change Desired Value** option and the **Force Signal Value** form appears as shown in the following figure:

Figure 80 Changing Desired Value

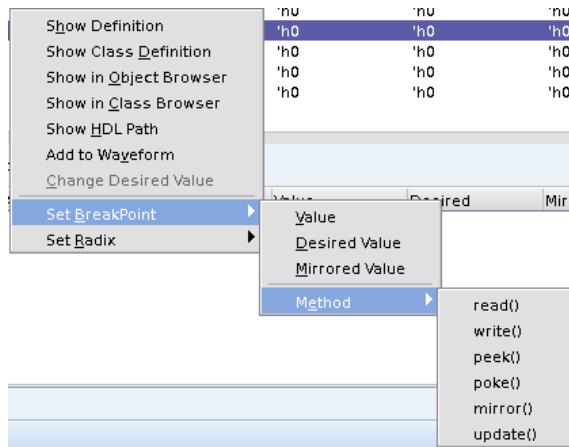


Setting Breakpoints

Perform the following steps to set a breakpoint:

1. Create a breakpoint on a register model and specify the condition for the breakpoint.
2. Use the **Set Breakpoint** option in the right-click option menu and select a breakpoint type as shown in the [Figure 81](#).

Figure 81 Setting Breakpoint Option

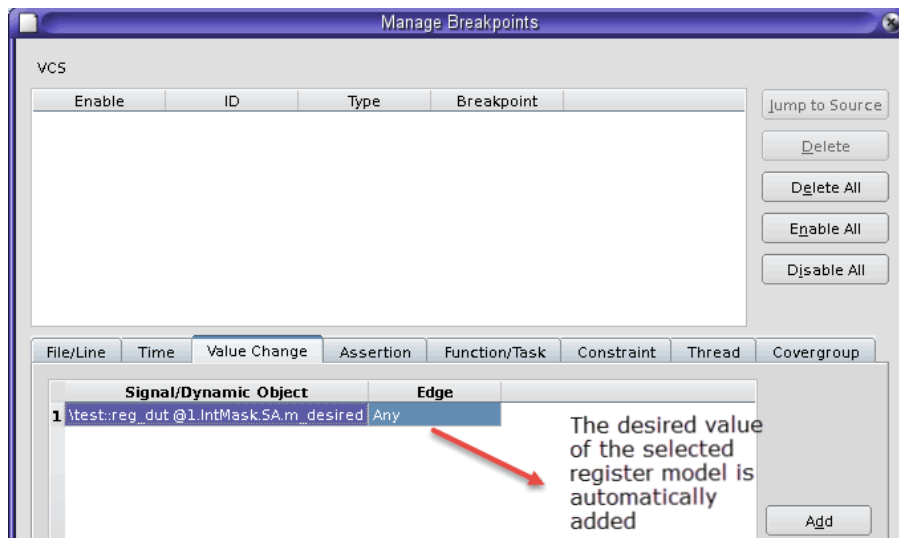


3. Set the breakpoint for the value change, desired value change, and mirrored value change with the **Value**, **Desired Value**, and **Mirrored Value** options.

As shown in the following figure, the corresponding desired value of the selected register model is automatically entered in the *Signal/Dynamic Object* list.

4. Click the **Add** button to add this breakpoint.

Figure 82 Setting Breakpoint in the Manage Breakpoint Form



Setting Radix

Select the **Set Radix** option in the right-click option menu to change the radix for the selected register model. As shown in the following figure, you can check the different radices for different register models.

Name	Offset	Width (bits)	Value	Desired	Mirrored	Access
IntMask	'h4	9	'h0	'h0	'h0	
OOSA	[8:8]	1	'o0	'o0	'o0	RW
RxFull	[6:6]	1	'h0	'h0	'h0	RW
RxHigh	[5:5]	1	'b0	'b0	'b0	RW
RxEmpty	[4:4]	1	'd0	'd0	'd0	RW
TxFull	[2:2]	1	'h0	'h0	'h0	RW
TxLow	[1:1]	1	'h0	'h0	'h0	RW
TxEmpty	[0:0]	1	'h0	'h0	'h0	RW

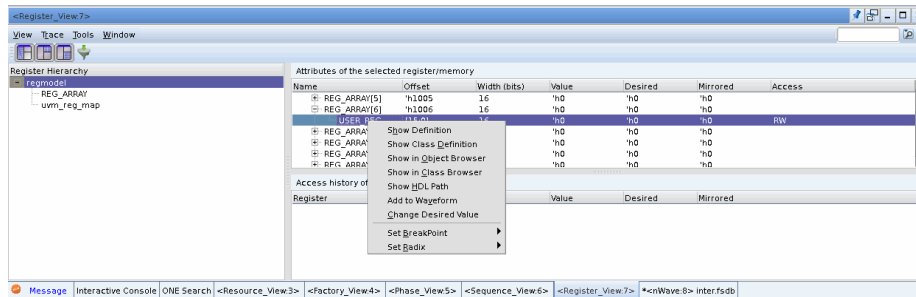
Setting Radix Option

Adding a Register to the Waveform

Perform the following steps to add a register to the waveform:

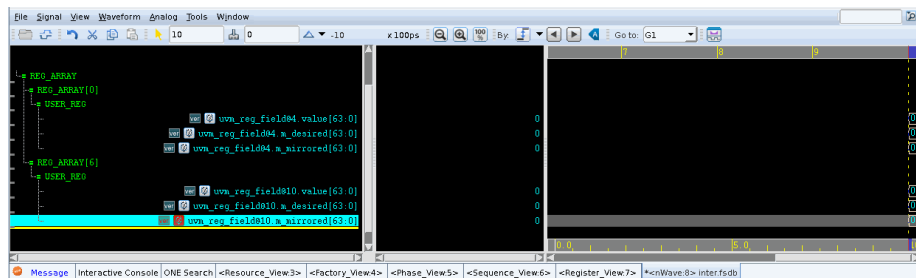
1. Select a Register in the *Register Attribute* pane.
2. Select **Trace -> Add to Waveform** option or alternatively right-click on the selected sequence and select the **Add to Waveform** option as shown in the following figure.

Figure 83 Adding Register to Waveform Option



On adding the register to the waveform, the waveform window is displayed as shown in [Figure 84](#):

Figure 84 Register Attribute in Waveform



Applying Quick Filter in the Register Attribute and Access History Panes

Perform the following step to use the quick filter in the *Register Attribute* and *Access History* panes:

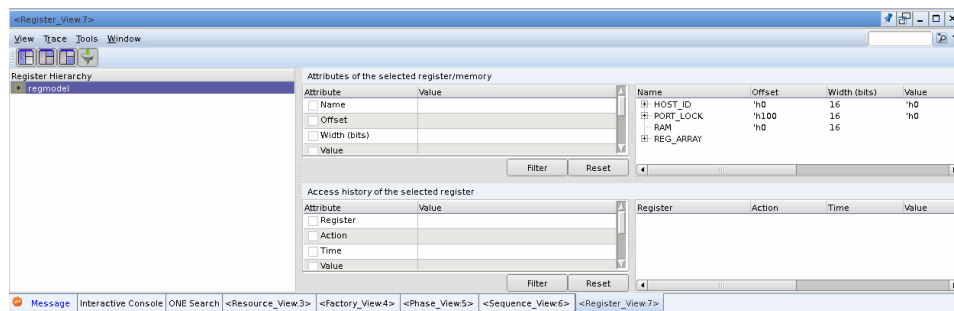


Click the icon to display the *Quick Filter* panes and use them to filter the content as shown in [Figure 85](#). The usage is the same as the description of the Quick Filter pane in the [Resource View](#) section.

Note:

Clicking the **Quick Filter** button on the toolbar enables the quick filtering feature on both the *Register Attribute* and *Access History* panes.

Figure 85 Show Quick Filter



Searching Registers and Register Field Items in the Register View

Verdi allows you to search the register hierarchy tree for a specific register or register field item in the *Register View* using the **Show Search** right-click option. This option allows you to view or hide the **Search** text field in the *Register Hierarchy* pane.

To search for a register or field in the *Register Hierarchy* pane of the *Register View*, perform the following steps:

1. Right-click a register or register field item in the *Register Hierarchy* pane and select the **Unified Find** right-click option to view the **Search** text field in the top-right corner of the main window. You can also use the keyboard shortcut **Ctrl+F** to enable/disable the search functionality. The search applies to the active window only.

Figure 86 Unified Find Option

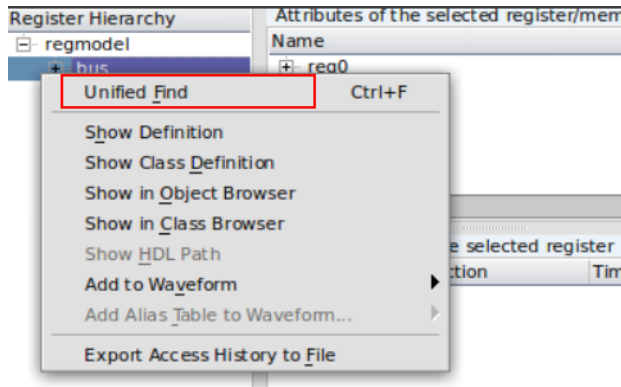
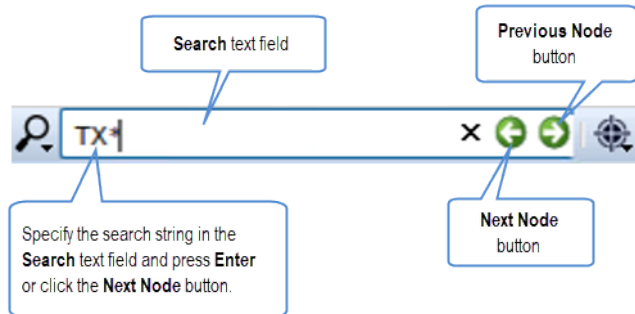


Figure 87 Search Text Field



2. Specify the search string in the **Search** text field and press **Enter** or click the **Next Node** button.

The matched item is selected in the register hierarchy tree. The **Next Node** and **Previous Node** buttons allow you to traverse the matching items up or down in the *Register Hierarchy* pane.

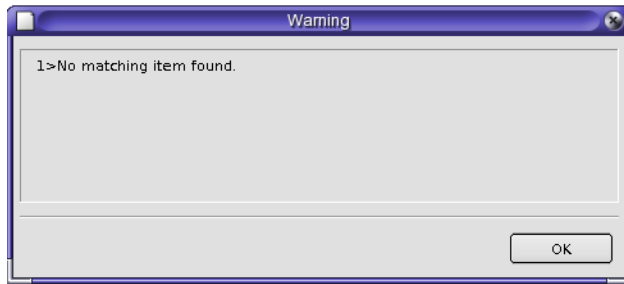
For information on the usage of Unified Find feature, see the *Using Unified Find* section in the *Verdi User Guide and Tutorial* guide.

Wildcards are supported in the **Search** text field. The case-sensitivity of the search string entered in the **Search** text field depends on the **Match case** right-click option.

UVM *Register View* does not support full path search, such as `regmodel.SESSION[1]` or `regmodel.myclass`. You can only search for the name of a register or register field item in register hierarchy tree. For example, you can search register/field name that contains `_ID` by specifying the search string as `*_ID`.

The following warning dialog box is displayed when no item matches the search string during one loop of search around the register hierarchy tree.

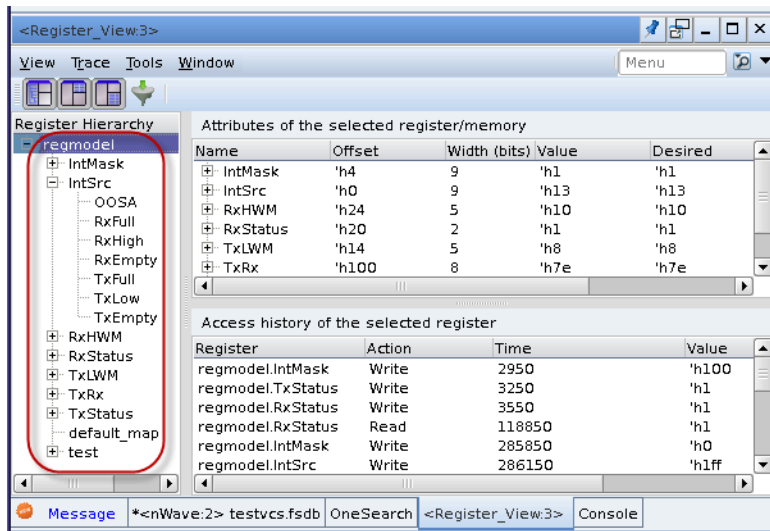
Figure 88 Warning Dialog Box



Viewing Registers and Register Fields in the Register Hierarchy Pane by Default

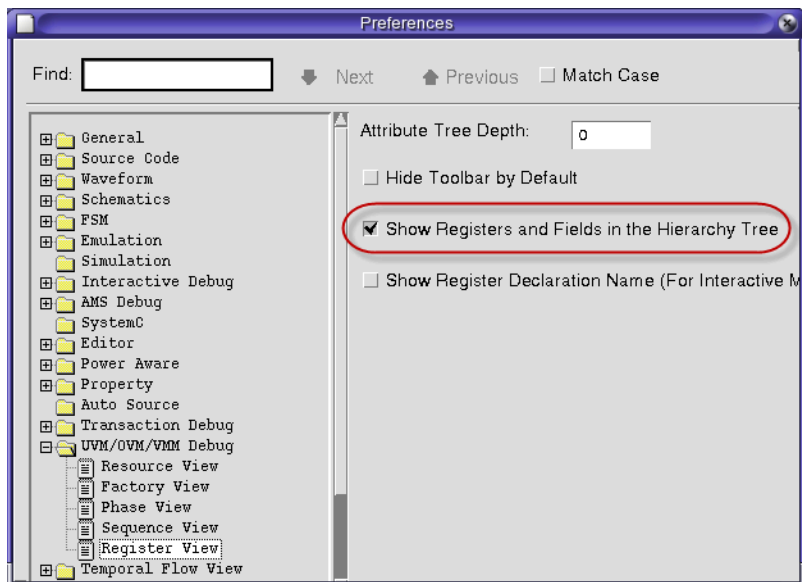
Verdi displays the registers and register fields in *Register Hierarchy* pane by default, as shown in the following figure:

Figure 89 Register Hierarchy Tree



The **Show Registers and Fields in the Hierarchy Tree** preference option is enabled by default in the **UVM/OVM/VMM Debug -> Register View** page of the *Preferences* dialog box, as shown in the following figure.

Figure 90 Register View Page

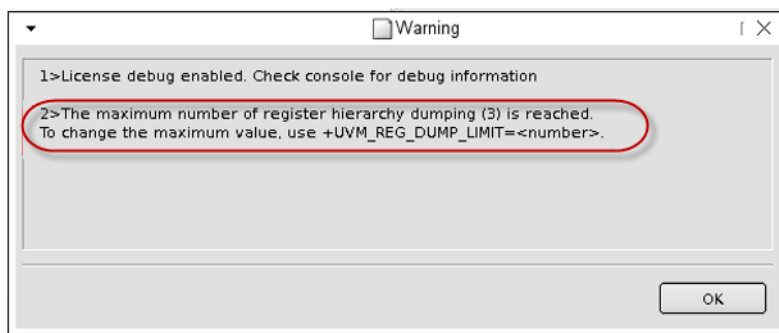


Displaying Warning Messages for Incomplete Register Hierarchy

The *Register Hierarchy* pane of the *Register View* is built by the data recorded in both interactive and post-processing mode. However, it takes time to record the register hierarchy data if the hierarchy tree is huge. Therefore, Verdi provides a default maximum count for the recorded data. If the maximum count is exceeded, the recording process is stopped. This makes the register hierarchy tree incomplete.

For incomplete register hierarchy, Verdi issues a warning message as follows:

Figure 91 Warning Message for Incomplete Register Hierarchy



The `+UVM_REG_DUMP_LIMIT` environment variable is used to limit the number of recorded registers in the register hierarchy. The default value of `+UVM_REG_DUMP_LIMIT` is 5000.

Following is the use model:

+UVM_REG_DUMP_LIMIT=<number>

If +UVM_REG_DUMP_LIMIT=0, then there is no limitation and all the registers are recorded.

Displaying Register Access Status in the Register View

Verdi displays register access status of a register in the *Register Access History* pane of the *Register View*. The register access status represents the operation result and is displayed in the **Status** column of the *Register Access History* pane.

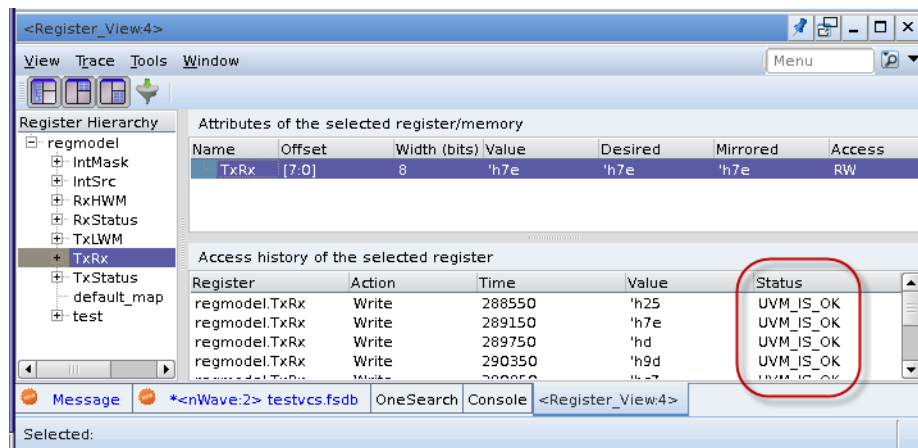
There are three register access status types:

- UVM_IS_OK
- UVM_NOT_OK
- UVM_HAS_X

The following table describes the status of the register operations:

UVM_IS_OK	Operation completed successfully
UVM_NOT_OK	Operation completed with error
UVM_HAS_X	Operation completed successfully, bit has unknown bits

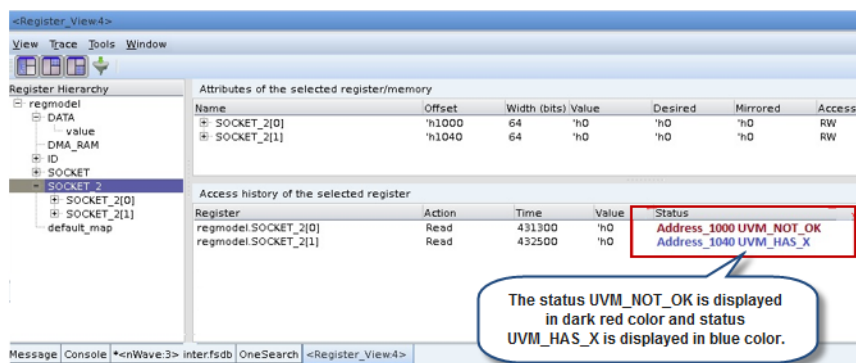
Figure 92 Register Access Status in the Register View



Key Points to Note

- If the access history of the selected register has no status information, the **Status** field of the register is hidden in the *Register Access History* pane.
- The UVM_IS_OK status is displayed in black color.
- The UVM_NOT_OK status is displayed in dark red color, and shows the address (hexadecimal) with it, as shown in the following figure.
- The UVM_HAS_X status is displayed in blue color, and shows the address (hexadecimal) with it, as shown in the following figure.

Figure 93 Register Access Status Display in Register View



Multitasking Task

Verdi supports multitasking update in the UVM debug windows. That is, when UVM debug windows are being updated with data, Verdi allows you to perform other operations without blocking the GUI.

Verdi supports multitasking update for the following UVM debug windows:

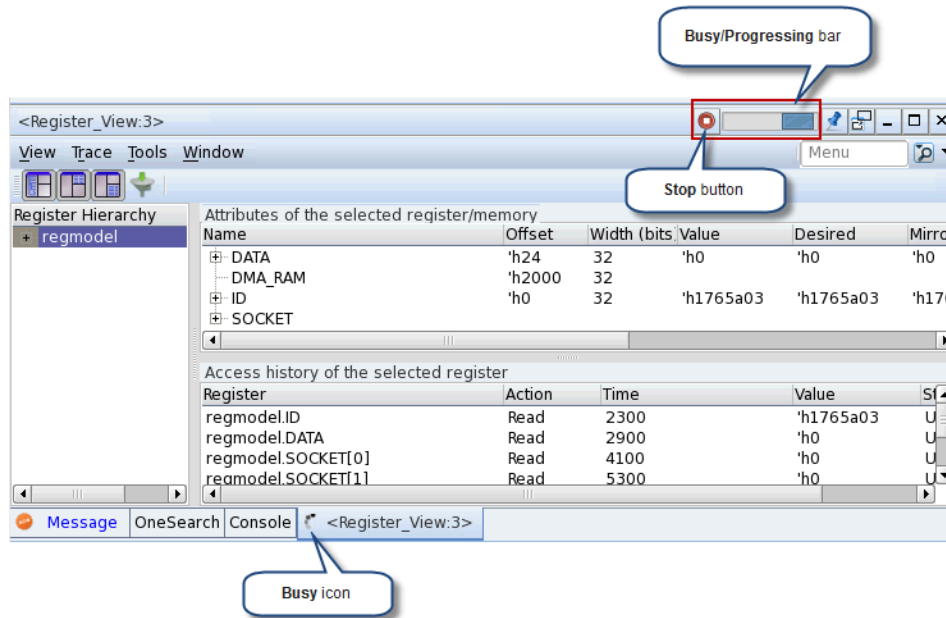
- Register View
- Resource View
- Phase View
- Sequence View

GUI Operations for Multitasking Update in UVM Debug Windows

During multitasking update, Verdi displays the **Busy/Progressing** bar on top right corner of the view, and shows the **Busy** icon on the tab, as shown in the following figure.

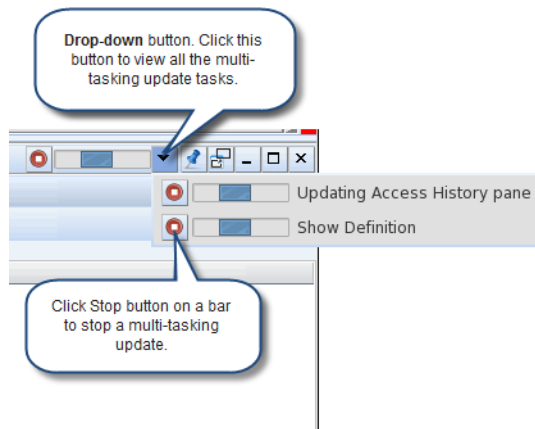
UVM debug windows can have multiple multitasking updates processing at the same time. Verdi displays **Busy/Progressing** bar for each update. You can hover the mouse pointer on a **Busy/Progressing** bar to view its information in a tooltip.

Figure 94 GUI to Notify Data Update in UVM Debug Windows



You can click the **Drop-down** button, as shown in the following figure, to view all the multitasking updates. You can use the **Stop** button on each bar to stop each update.

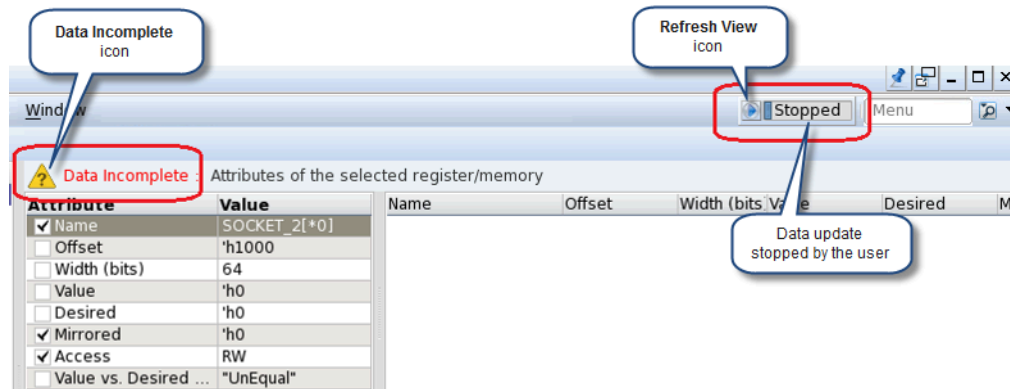
Figure 95 Viewing Busy/Progressing Bar for Each Multitasking Update Task



When a multitasking update is initiated, Verdi clears all the invalid data in the view. If the data is already available during updating, Verdi performs incremental update of the data in the view.

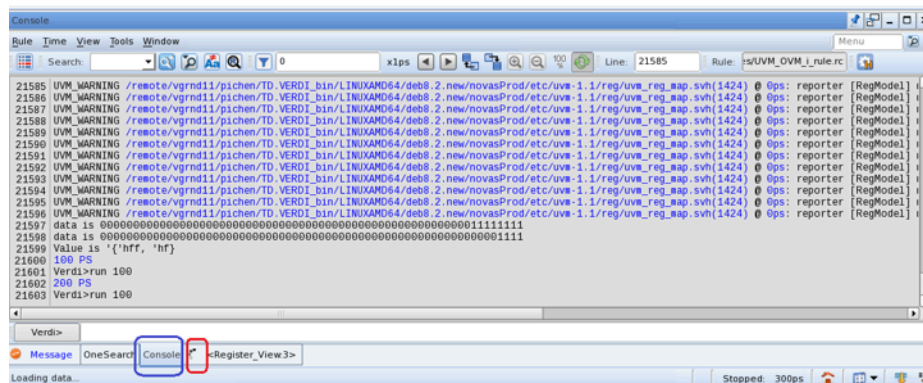
If you click the **Stop** button to stop a multitasking update, Verdi displays **Data Incomplete** icon on the title bar of the current pane, as shown in the following figure, to notify that the data update is incomplete. Also, Verdi displays the bar for UVM windows if the updating of the data in the view is stopped. You can refresh the view by clicking the **Refresh View** icon, as shown in the following figure, to reload or refresh its multitasking update.

Figure 96 Data Incomplete Icon



When a multitasking update is still happening, you can jump to or open another pane to do other operations. You can see that the **Busy** icon on the tab is still active as the data update still happens in the background. This icon disappears once the data update is completed.

Figure 97 Switching to Another Pane



Applying New Settings for the Current View With multitasking Update Tasks

You can perform the following operations on the current view with multitasking update tasks:

- Change preference settings
- Perform right-click operations
- Apply new quick filter settings

When you apply new preference settings, Verdi reloads all the views in UVM aware windows. If the current view has a multitasking update, Verdi restarts it to upload the new data to the view.

You can apply new quick filter settings to the current view even when the multitasking update is happening. When you apply new quick filter settings, Verdi stops the original multitasking update and triggers a new update to get the new data for the view.

The following sections describe the status of quick filters and right-click options when multitasking updates are processing data in the UVM Aware windows:

- [Register View](#)
- [Resource View](#)
- [Phase View](#)
- [Sequence View](#)

Register View

The search functionality is disabled in the *Register Hierarchy* pane.

For both the *Register Attribute* and *Access History* panes, when new quick filter settings are applied, Verdi stops the current multitasking update task and triggers a new multitasking update to get the new data for the view.

The following tables list the status of the right-click options when multitasking updates are processing data in the *Register View*:

Right-click Option	In Register Hierarchy Pane
Show Search	Enabled
Show Definition	Enabled

Right-click Option	In Register Hierarchy Pane
Show Class Definition	Enabled
Show in Class Browser	Enabled
Add to Waveform	Enabled
Add Alias Table to Waveform	Enabled

Right-click Option	In Attribute Pane
Show Definition	Enabled
Show Class Definition	Enabled
Show in Class Browser	Enabled
Show HDL Path	Enabled
Add to Waveform	Enabled
Change Desired Value	Disabled
Set Breakpoint	Enabled
Set Radix	Enabled

Resource View

For the *Resource View* pane, when new quick filter settings are applied, Verdi stops the current multitasking update and triggers a new update to get the new data for the view.

The following tables list the status of the right-click options when multitasking updates are processing data in the *Resource View*:

Right-click Option	In Resource View Pane
Show Source for Virtual Interface	Enabled
Set Radix	Enabled

Right-click Option	In Access History Pane
Show call in Source Window	Enabled
Show Accessor in Object Browser	Enabled
Show Accessor in Class Browser	Enabled

Phase View

Following tables list the status of the quick filters and right-click options when multitasking updates are processing data in the *Phase View*:

Quick Filter	Status	Action
Show Only Active Phase (Check box)	Enabled	
For <i>Phase History</i> pane	Enabled	Verdi stops the current multitasking update and triggers a new multitasking update to get the new data for the view.
Active Objection Only (Check box)	Enabled	
For <i>Objection History</i> pane	Enabled	

Right-click Option	In Phase History Pane
Manage Breakpoints	Enabled
Set Breakpoint for Phase	Enabled

Right-click Option	In Objection History Pane
Show Source	Enabled
Show Object in Class Browser	Enabled
Show Object in Source Window	Enabled

Sequence View

Following tables list the status of the quick filters and right-click options when multitasking updates are processing data in the *Sequence View*:

Quick Filter	Status	Action
For <i>Sequence</i> pane	Enabled	Verdi stops the current multitasking update and triggers a new multitasking update to get the new data for the view.

Right-click Option	In Sequence View Pane
Show Class Definition in Source Window	Enabled
Show Invocation in Source Window	Enabled
Show Sequence in Class Browse	Enabled
Show Sequencer in Class Browser	Enabled
Show Sequence in Stack Pane	Enabled
Set Breakpoint on Sequence Method	Enabled

Right-click Option	In Sequence View Pane
Add Sequence to Watches	Enabled
Add Sequencer to Watches	Enabled
Add Sequence to Waveform	Enabled
Add Sequence to Transaction Browser	Enabled

Tcl Commands for Multitasking Update in UVM Debug Windows

UVM debug windows provide multitasking stop Tcl command for each multitasking update. Following is the syntax of the Tcl command:

```
uvmDebugNBTaskStop -win [rsrc|phase|seq|reg] -task [TASK_NAME]
```

The following table lists `TASK_NAME` for all the UVM debug windows:

UVM Windows	Task description	TASK_NAME
Resource (<i>rsrc</i>)	Update Top pane	top
	Update <i>Access History</i> pane	rah
Sequence (<i>seq</i>)	Update main pane	top
Phase (<i>phase</i>)	Update Objection pane	ohist
Register (<i>reg</i>)	Update <i>Register Hierarchy</i> pane	hier
	Update <i>Register Attribute</i> pane	attr
	Update <i>Access History</i> pane	access
	Trigger right-click option: Show Definition	showDef

UVM debug windows provide the following Tcl commands to refresh view/pane if the current view/pane is aborted while updating:

UVM Windows	Refresh data to target view	Tcl command
Resource	Refresh view	<code>uvmRsrcWinRefresh -task [top rah]</code>

UVM Windows	Refresh data to target view	Tcl command
Sequence	Refresh main pane	<code>uvmSeqWinRefresh -task top</code>
Phase	Refresh Objection view	<code>uvmPhaseWinRefresh -task ohist</code>
Register	Refresh Register view	<code>uvmRegWinRefresh -task [hier attr access showDef]</code>

Debugging UVM Registers in Verdi Post-Processing Mode

A register model in a UVM register layer typically contains hierarchy of blocks that map to the design hierarchy. Register blocks can contain registers, register files, register maps, memories, and other blocks.

Verdi allows you to debug UVM register information in the post-processing mode using the *UVM Register View*. UVM Aware debug refers to the annotation FSBD file when running in post-processing mode.

This feature allows you to perform the following:

- View the register hierarchy, register attribute values, and access history of an interface FSDB.
- Add register objects to the waveform.
- Synchronize cursor time between *Register View* and waveform and display the register attribute values according to the cursor time. You can also specify cursor time to display the register attribute values.
- Select a hardware address signal in register map to create address alias table in the waveform, so that the waveform can display the register alias names to replace address on the signal.

Dumping UVM Register Information in Post-Processing Mode

This section describes the compile-time and runtime options required to dump the necessary information for UVM *Register View* in the post-processing mode.

Dumping Register Hierarchy and Access History Information

For VCS:

Use the `-ntb_opts uvm` compile-time option to include Verdi UVM library. Following is the syntax to generate `simv` with register post-processing data:

```
% vcs -kdb -sverilog -debug_access+all -ntb_opts uvm -lca filename.sv
% ./simv +UVM_VERDI_TRACE=RAL +fsdbfile+<name>.fsdb -l sim.log
```

Example:

```
% vcs -full64 -kdb -sverilog -debug_access+all -ntb_opts uvm -lca
-timescale=1ns/1ns +incdir+./apb +incdir+vip tb_top.sv test.sv
% ./simv +UVM_VERDI_TRACE=RAL +fsdbfile+test.fsdb -l sim.log
```

For Third-Party Simulators:

Specify the following include paths for the UVM library:

- `-incdir $VERDI_HOME/etc/uvm-1.1/verdi/`
- `$VERDI_HOME/etc/uvm-1.1/verdi/uvm_custom_install_verdi_recorder.sv`
- `$VERDI_HOME/etc/uvm-1.1/verdi/dpi/uvm_verdi_dpi.cpp`

Dumping Class Objects for Register View Post-Processing in VCS

If you want to add registers to waveform and view both register hierarchy and access history, use the `+UVM_VERDI_TRACE=RALWAVE` runtime option to enable register object, register hierarchy, and register access history recording.

Following is an example to generate object dumping data for *Register View* post-processing:

```
% vcs -kdb -sverilog -debug_access+all -ntb_opts uvm -lca filename.sv
% ./simv +UVM_VERDI_TRACE=RALWAVE +fsdbfile+test.fsdb -l sim.log
```

To specify dumping level for UVM component object, you need to specify `+UVM_OBJ_DUMP_LEVEL=N`.

The default value of `N` is 1. If `N` is specified as 0, all level of objects are dumped.

Simulation Options for Generating FSDB File With Register Information in Post-Processing Mode

[Table 1](#) shows the runtime options required to generate FSDB file with register information in the post-processing mode.

Table 1 Runtime Options

Option	Description
+UVM_VERDI_TRACE=RAL	Dumps the register hierarchy and access history data into the FSDB file.
+UVM_VERDI_TRACE=RALWAVE	Controls whether to load the configuration file to dump all register-related object data when Verdi recorder is used. It also dumps the register hierarchy and access history data, so that the +UVM_VERDI_TRACE=RAL option is not required if you use this option. This option is supported only in VCS.

Following is the example for VCS simulator to generate FSDB file with the register hierarchy and access history data:

```
% ./simv +UVM_VERDI_TRACE=RAL +fsdbfile+<name>.fsdb -l sim.log
```

Following is the example for VCS simulator to generate FSDB file with register hierarchy, register access history, and related object dumping:

```
% ./simv +UVM_VERDI_TRACE=RALWAVE +fsdbfile+<name>.fsdb -l sim.log
```

Invoking Verdi in Post-Processing mode

To open UVM *Register View* in post-processing mode, invoke Verdi with an FSDB file. Following is the VCS example to run Verdi by importing the design and loading the FSDB file:

```
% verdi -ssf <filename>.fsdb
```

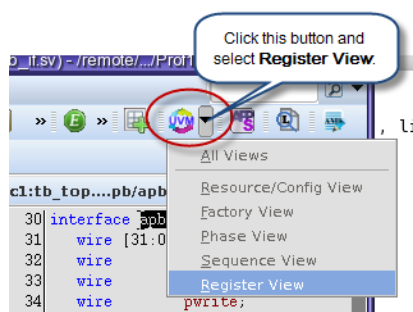
Verdi displays the **UVM icon**  when it is invoked with the FSDB file containing the register hierarchy and register access history information.

Click **UVM icon** drop-down button or click **Tools > UVM/OVM/VMM Debug** and select **Register View** from the drop-down list to view the Register View in post-processing mode, as shown in [Figure 98](#).

Chapter 4: UVM Debug Usage

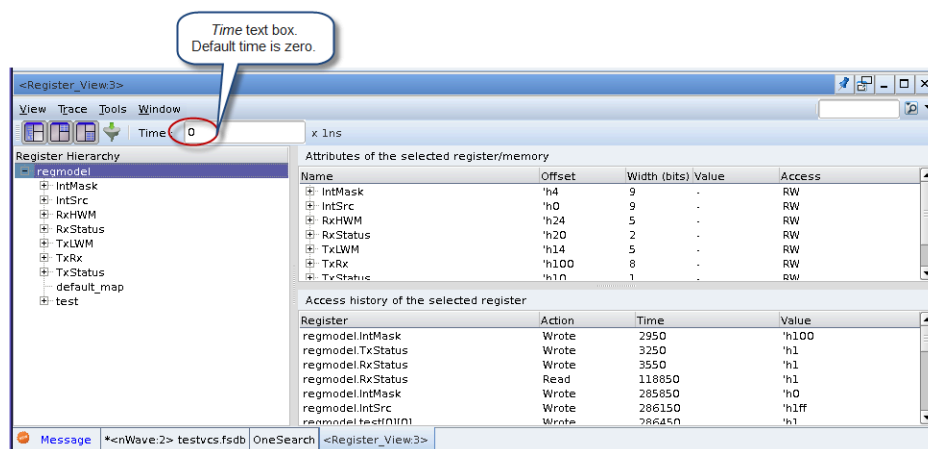
Debugging UVM Registers in Verdi Post-Processing Mode

Figure 98 Selecting Register View



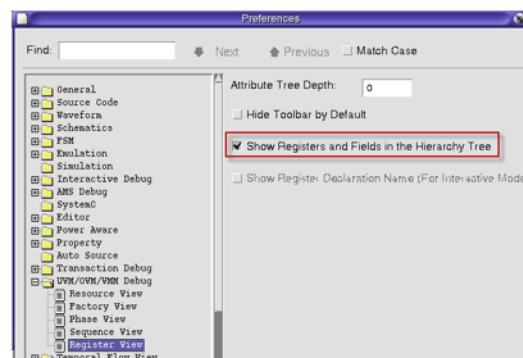
The *Register View* is enabled only when the FSDB file is loaded. The *Time* text box displays the cursor time, the default start time is 0 as shown in the following figure.

Figure 99 Register View in Post-Processing Mode



You can use the UVM debug preference options in the *Preferences* form to change the layout of the hierarchy in the UVM *Register View*.

Figure 100 Preferences Form



Debugging UVM Register Information in Post-Processing Mode

The *Register Attribute* pane of the *Register View* displays the components of a register item selected in the *Register Hierarchy* pane. The registers are displayed in a hierarchical tree structure. The hierarchical structure is collapsed by default, you can click '+' to expand a register item and view its fields.

Functions Supported in Register View

Table 2 describes the functions supported in the *Register View*.

Function	Description
Supporting mode	<i>Register View</i> supports VCS simulator in interactive mode, and supports VCS and third-party simulators in post-processing mode.
Synchronize access time to attribute values	Double-click an item in the <i>Access History</i> pane to set the cursor time as the access time. For more information, see " Updating Register Attribute Values According to the Cursor Time or Selected Access History in Post-Processing Mode ".
Synchronize time with waveform	Select View-> Sync Time with Waveform option to synchronize time with the <i>nWave</i> . For more information, see " Synchronizing Time Between nWave and Register View in Post-Processing Mode ".

Table 3 lists the support of these register functions in VCS and third-party simulators.

UVM Register Function	VCS		Third-Party	Simulator
	<i>Interactive</i>	<i>Post-Processing</i>	<i>Interactive</i>	<i>Post-Processing</i>
Supporting mode	Supported	Supported	Not Supported	Supported
Synchronize access time to attribute values	Not Supported	Supported	Not Supported	Supported
Synchronize time with waveform	Not Supported	Supported	Not Supported	Supported

Right-click Menu Options in the Register Hierarchy and Register Attribute Panes

Table 4 describes the right-click menu options in the *Register Hierarchy* and *Register Attribute* panes.

Option	Description
Show Definition	Displays the object definition of the selected item in the <i>Source Code</i> pane.
Show Class Definition	Displays the class definition of the selected item in the <i>nTrace</i> window.
Show in Object Browser	Displays the selected register item in the <i>Object Browser</i> pane.
Show in Class Browser	Displays the selected register item in the <i>Class Browser</i> pane.
Show HDL Path	Displays the hierarchical signal path of the selected item in the <i>nTrace</i> window.
Add to Waveform	Allows you to add register value to the waveform.
Add Alias Table to Waveform	Allows you to add address alias table to the <i>nWave</i> .

Table 5 lists the support of these right-click menu options in VCS and third-party simulators.

UVM Register Function	VCS		Third-Party	Simulator
	<i>Interactive</i>	<i>Post-Processing</i>	<i>Interactive</i>	<i>Post-Processing</i>
Show Definition	Supported	Not Supported	Not Supported	Not Supported
Show Class Definition	Supported	Supported	Not Supported	Supported
Show in Object Browser	Supported	Not Supported	Not Supported	Not Supported
Show in Class Browser	Supported	Not Supported	Not Supported	Not Supported

Show HDL Path	Supported	Supported	Not Supported	Supported
Add to Waveform	Supported	Supported	Not Supported	Not Supported
Add Alias Table to Waveform	Supported	Supported	Not Supported	Supported

Updating Register Attribute Values According to the Cursor Time or Selected Access History in Post-Processing Mode

You can double-click an item in the *Access History* pane to set the cursor time as the access time in the *Time* text box. Also, you can specify a valid cursor time in the *Time* text box. When the cursor time is set, the *Register Attribute* pane updates the values according to the cursor time.

This feature is supported for VCS and third party simulators in post-processing mode.

To view the attribute values of a specific access time:

Specify valid cursor time in the *Time* text box and press *Enter*. The default cursor time is 0.

Or,

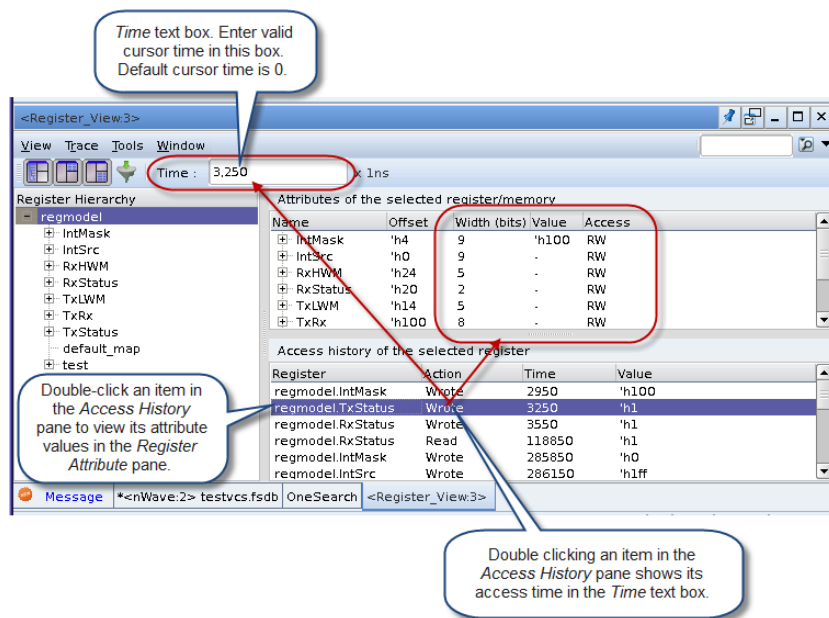
Double-click an item in the *Access History* pane.

The values in the *Register Attribute* pane are updated according to the new cursor time, as shown in [Figure 101](#).

You can change the cursor time in the *Time* text box. If the time specified in the *Time* text box does not match the time in the *Access History* pane, then Verdi searches for the register with the time nearest to the time specified in the *Time* text box and displays its values in the *Register Attribute* pane.

For example, in the following figure, if you enter time 3000 in the *Time* text box, the *Register Attribute* pane displays the data of the register with time 2950.

Figure 101 Specifying Cursor Time in the Register View



Synchronizing Time Between nWave and Register View in Post-Processing Mode

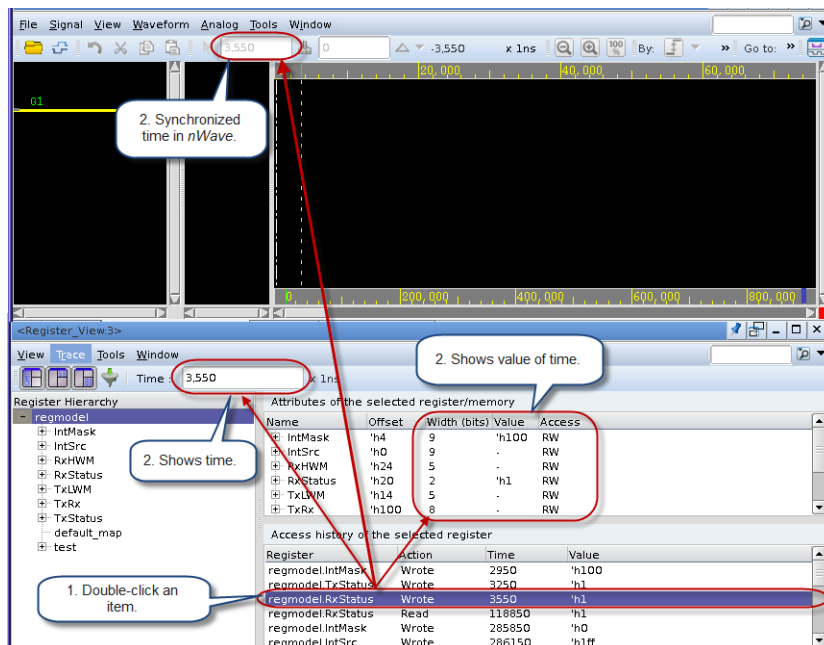
Verdi supports time synchronization between the *Register View* and the *nWave* in post-processing mode.

If you select **View-> Sync Time with Waveform** option, you can synchronize time with the *nWave* either by double-clicking an item in the *Access History* pane or by specifying cursor time in *Time* text box as shown in Figure 102.

Also, any cursor time change in the *nWave* applies to the *Register View*, and the values of the *Register Attribute* pane are updated according to the new cursor time.

This feature is supported for VCS and third-party simulators in post-processing mode.

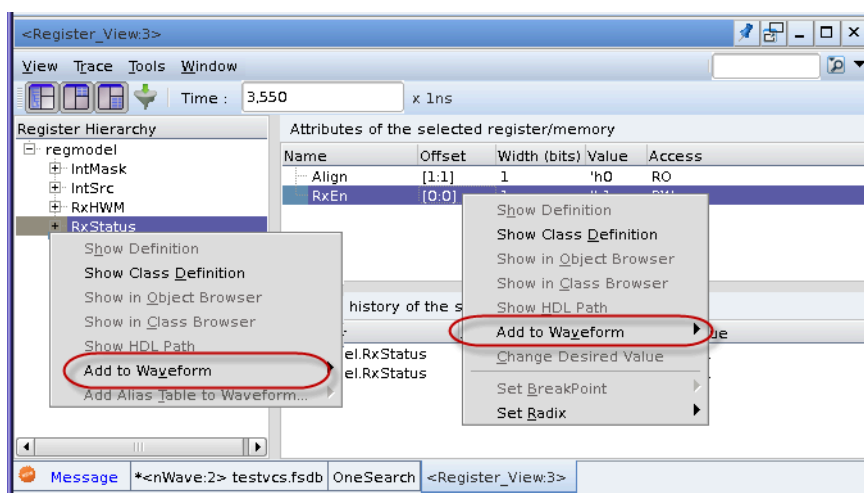
Figure 102 Synchronizing Time Between *nWave* and Register View



Adding Register Items to the Waveform

You can use the **Add to Waveform** right-click option in the *Register Hierarchy* pane and *Register Attribute* pane, as shown in Figure 103, to add register objects into any existing or a new waveform.

Figure 103 Add to Waveform



If you select **Add to Waveform** option in the **Trace** menu, Verdi considers the items selected in the *Register Attribute* pane.

[Table 2](#) describes the operations of the **Add to Waveform** option for different types of register objects.

Table 2 *Operation of Add to Waveform Option for Register Objects*

Register Type	Operation
Register Block/File	All register fields under the selected block/file are added to the <i>nWave</i> .
Register	All register fields of the selected register are added to the <i>nWave</i> .
Register Field	Only the selected register field are added to the <i>nWave</i> .
Memory	The menu item should be disabled.
Register Map	The menu item should be disabled.
Array of Registers/Blocks/Files	A dialog box should appear and allow you to specify the ranges of the array. Only the fields of the selected ranges are added to the <i>nWave</i> .

Key Point to Note

If you perform **Add to Waveform** for a register object, the waveform displays the value changes of the object being dumped.

For example, if the class `uvm_reg_field` is as follows:

```
class uvm_reg_field extends uvm_object;
  rand uvm_reg_data_t value; // Mirrored after randomize()
  local uvm_reg_data_t m_mirrored; // Present in the HW
  local uvm_reg_data_t m_desired; // Mirrored after set()
```

When you perform **Add to Waveform**, the waveform displays the value changes of variables `value`, `m_desired` and `m_mirrored`.

However, the value on *Register View* indicates the value when the register operation **read**, **write**, **peek**, or **poke** is performed. The nature of values on the waveform and *Register View* are different. It is inadequate to compare these values.

Adding Address Alias Table to nWave

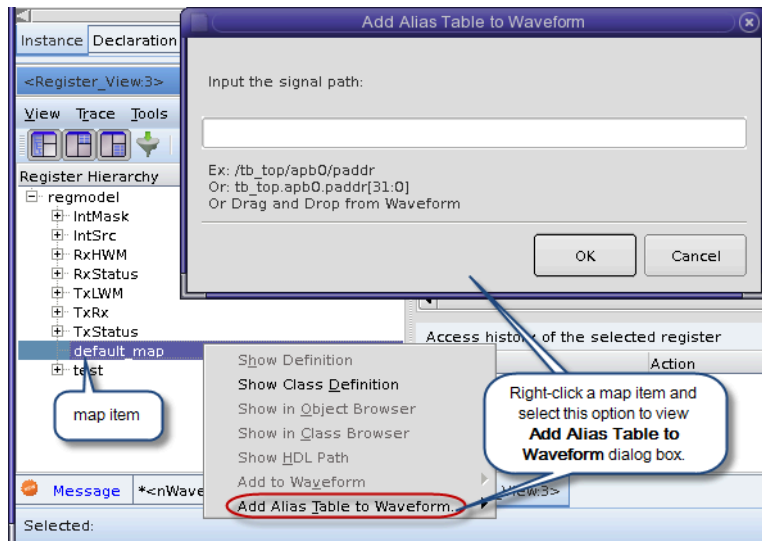
You can use the **Add Alias Table to Waveform** right-click option in the *Register Hierarchy* pane to add address alias table from *Register View* to any existing or a new *nWave* window.

Perform the following steps to add address alias table to the *nWave*:

1. Right-click a map item in the *Register Hierarchy* pane and select the **Add Alias Table to Waveform** right-click option.

The **Add Alias Table to Waveform** dialog box appears, as shown in the following figure:

Figure 104 Selecting the “Add Alias Table to Waveform” Option

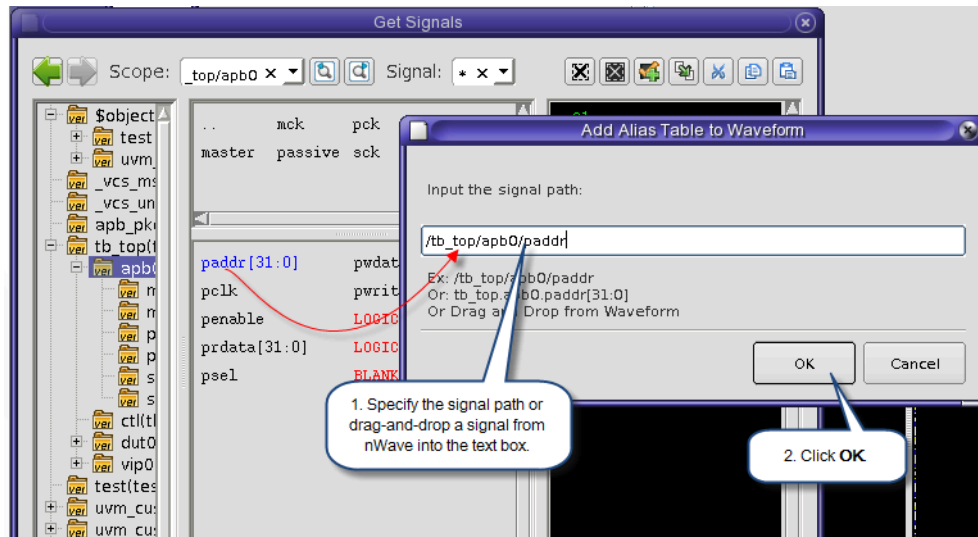


2. Specify the signal path in the text box.

Or,

Drag-and-drop a signal from the *nWave* window into the text box as shown in [Figure 105](#).

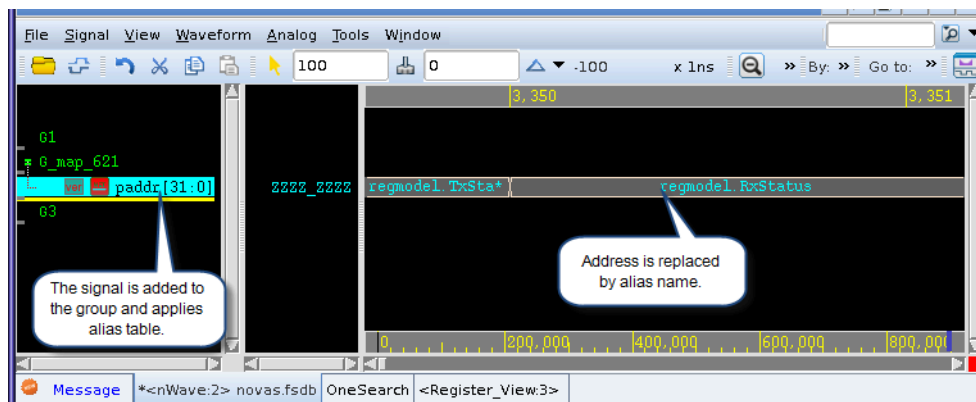
Figure 105 Specifying Signal Path



3. Click **OK** to add alias table and apply the alias name to this signal in the *nWave*.

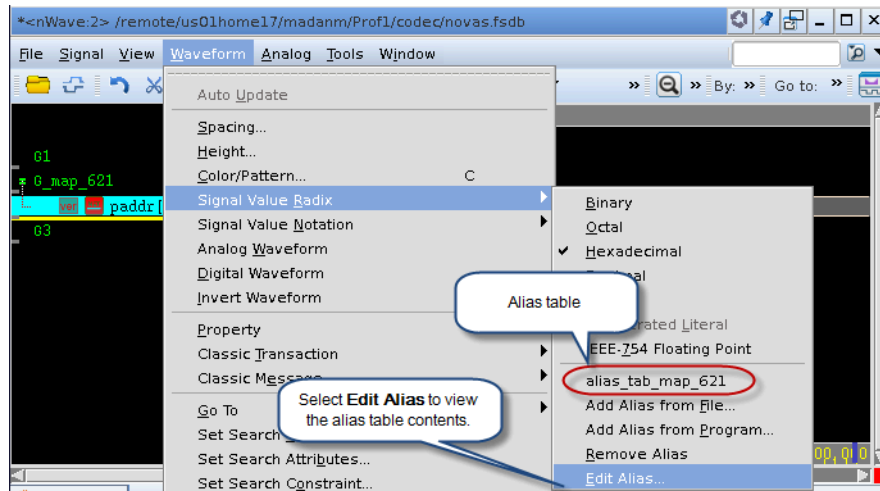
The signal is added to a new signal group in *nWave*, and the memory addresses are replaced by the alias names as shown in Figure 106.

Figure 106 Replacing the Address With Alias Name



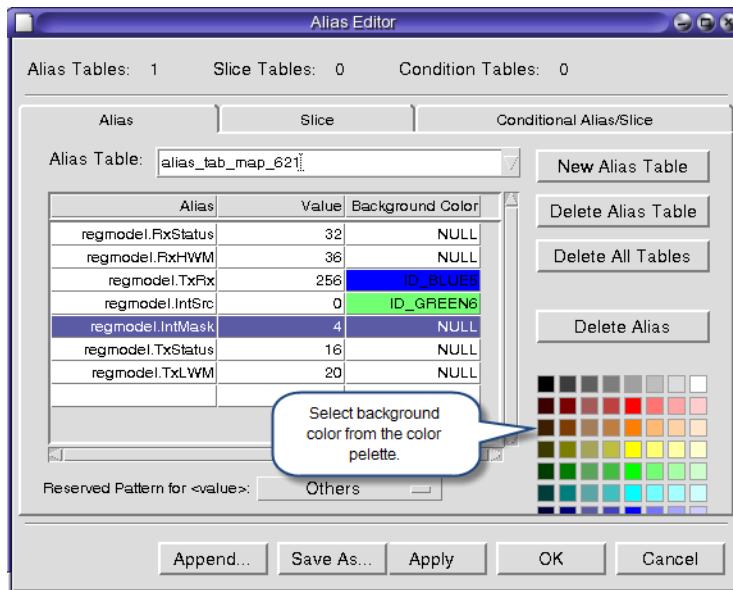
Click **Waveform > Signal Value Radix** to view the new alias table in the menu. Select the **Edit Alias** menu option, as shown in Figure 107, to view and edit the alias table contents.

Figure 107 Viewing Alias Table



The **Alias Editor** dialog box allows you to rename or change the colors for each item, as shown in Figure 108. Click **Apply** and then click **OK** to save the alias table for future usage.

Figure 108 Alias Editor Dialog Box

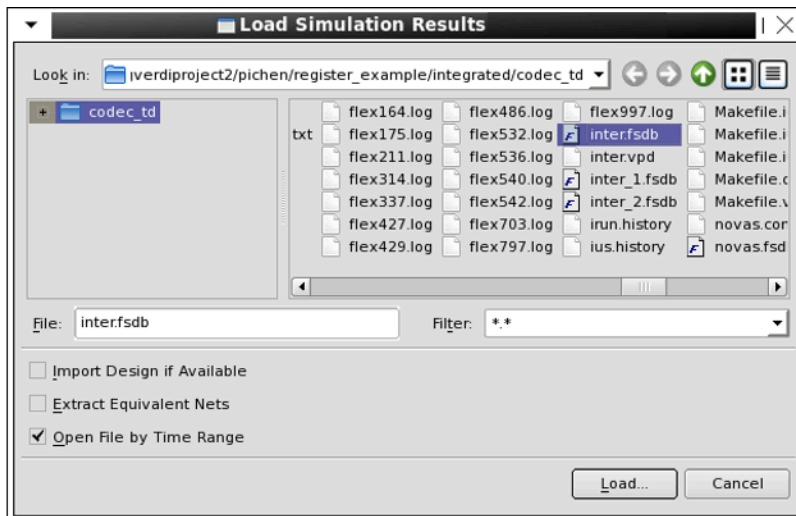


Displaying Register Data in Register View by Time Range in Post-Processing Mode

Opening FSDB File by Time Range

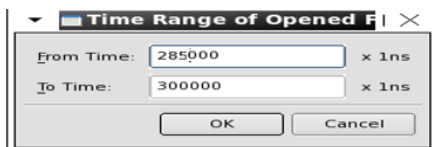
Verdi allows you to open FSDB file by time range in post-processing mode using the *Load Simulation Results* dialog box.

Figure 109 Load Simulation Results Dialog Box



If you select the **Open File by Time Range** check box in the *Load Simulation Results* dialog box, Verdi displays the *Time Range of Opened File* dialog box. This dialog box allows you to specify the time range, as shown in the following figure:

Figure 110 Time Range of Opened File dialog box

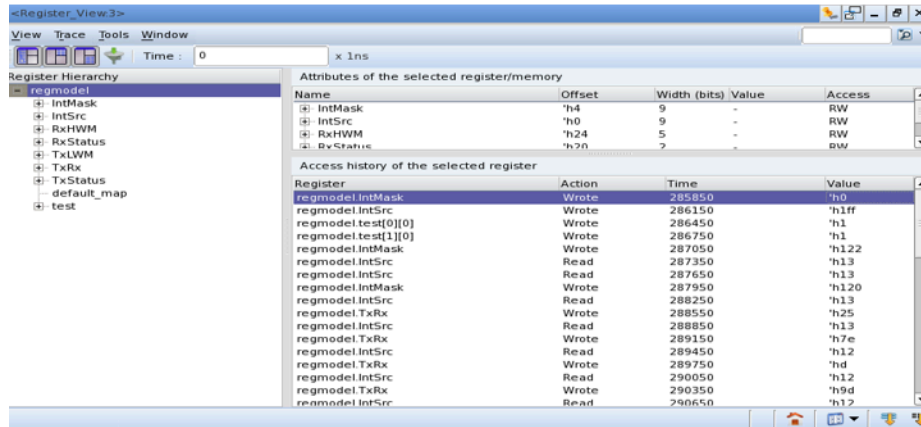


Displaying Register Data by Time Range

In post-processing mode, if the FSDB file is opened by time range, the *Register View* also displays data by time range in the *Register Access History* view. For example, if the FSDB

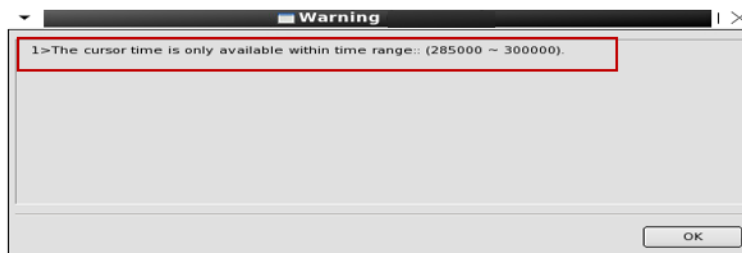
file is opened by time range 285000 - 300000 ns, *Register View* displays access history from time 285000 to 300000 ns as follows:

Figure 111 Displaying Register Data by Time Range



If the specified cursor time is out of time range, Verdi displays a warning dialog as shown in the following figure:

Figure 112 Warning Dialog



Save/Restore Session Support for the Register View in Post-Processing Mode

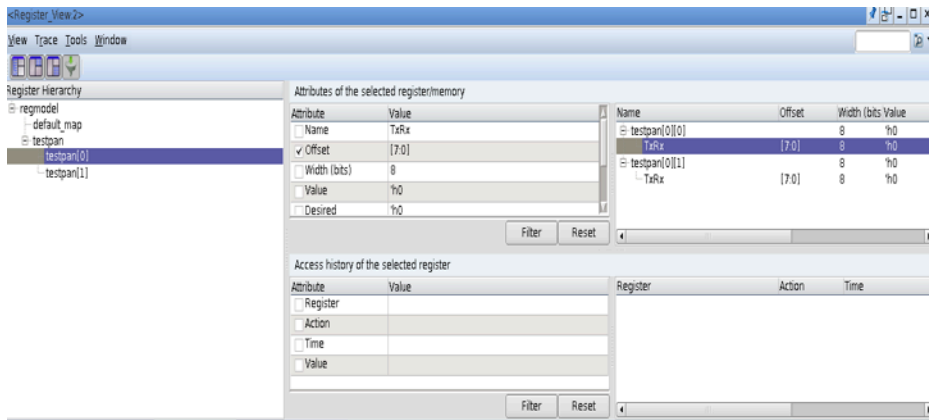
In post-processing mode, you can use the **File -> Save Session** command to save the current debugging session, including the UVM *Register View*.

In the *Register View*, the following items are restored as they were before the save session:

- *Register View* pane layout (*Register Hierarchy/Register Attribute/Register Access History* pane on or off)
- Register hierarchy data and selected items if exist
- Register attribute data and selected items if exist

- Register access history of the selected register data and selected items if exist
- Quick Filter on/off status, the values inside it and the status of check boxes
- If you have performed the radix display change on some signals before save session, those are restored too.

Figure 113 Register View Save/Restore Session



Limitations

See section *UVM Register View Debug in Post-Processing Mode: Limitations* for limitations of this feature.

Viewing OVM/UVM Component Waveforms in nWave

You can dump UVM component waveforms into FSDB using the Verdi UVM library. Verdi *Dynamic Object Dumping* technology is used to enable the dumping. In addition, the UVM component hierarchy is dumped as well.

You can use the Verdi **OVM/UVM Hierarchy View** to display the testbench hierarchy. You can also choose a UVM component waveform and add it to **nWave** from the **OVM/UVM Hierarchy View**.

Dumping UVM Component Waveforms

This section describes how you can enable UVM component waveform dumping during the simulation and then how you can view this waveform in **nWave** in the following steps:

1. Use the UVM library shipped with either Verdi or VCS starting with 2015.09-SP2 release and later. If you do not want to use any extra UVM file options, use the `-ntb_opts` option.
2. Add the `-debug_access+all` option at VCS compile command line before running simulation for component object dumping.
3. Enable FSDB dumping either using the `$fsdbDumpvars` system task or the dump UCLI command. Note that this step is optional.
4. Add the runtime option `+UVM_VERDI_TRACE=COMPWAVE`. You can stack this option on other `+UVM_VERDI_TRACE` options. Since component waveform is triggered from **OVMM/ UVM Dynamic Hierarchy Tree**, the `+UVM_VERDI_TRACE=COMPWAVE` option enables `+UVM_VERDI_TRACE=HIER` also.
5. Launch Verdi.

The following is a sample Makefile:

```
all:

comp run verdi_pp

comp:

% vcs -full64 -sverilog -ntb_opts uvm \ +incdir+../sv -debug_access+all
-kdb ubus_tb_top.sv -l compile.log

run:

% simv -l vcs.log +UVM_TESTNAME=test_read_modify_write
+UVM_VERDI_TRACE=COMPWAVE

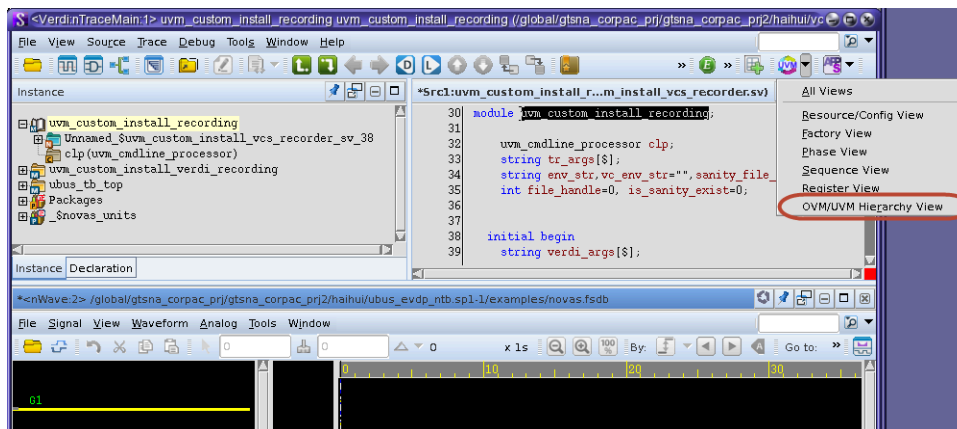
verdi_pp:

% verdi -simBin ./simv -ssf novas.fsdb -nologo
```

Viewing the UVM Hierarchy Tree

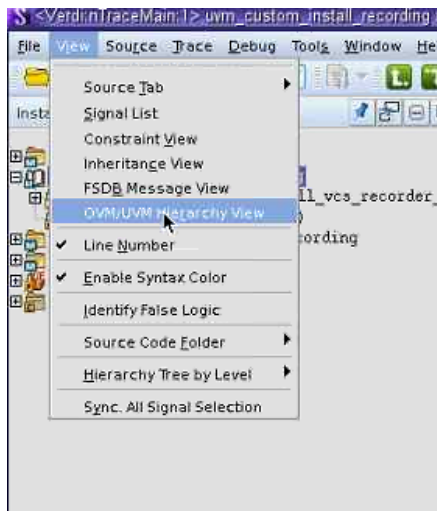
To view the UVM Hierarchy Tree, load the FSDB generated from the simulation as described previously.

From the toolbar, click on  icon on the upper right corner, then select **OVMM/UVM Hierarchy Tree** option as shown in the following figure:

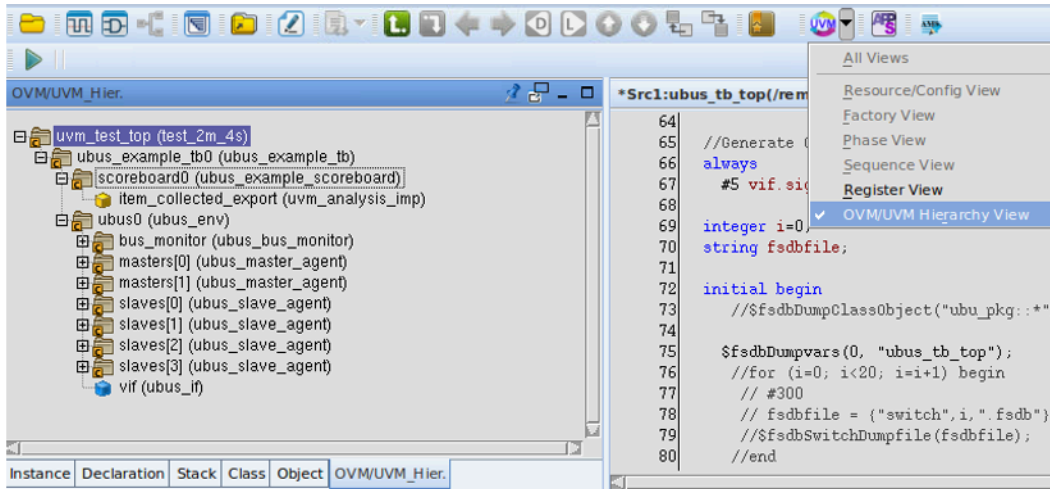


You can also access this option data can be viewed in the OVM/UVM Dynamic Hierarchy tree in the OVM/UVM Hierarchy pane by using the command **View -> OVM/UVM Hierarchy View** as shown in the following figure:

Figure 114 OVM/UVM Hierarchy View Menu Item



The **OVM/UVM Hierarchy View** tab is opened and the component hierarchy tree is displayed. Click on the component tree items to expand the hierarchy as shown in the following figure:



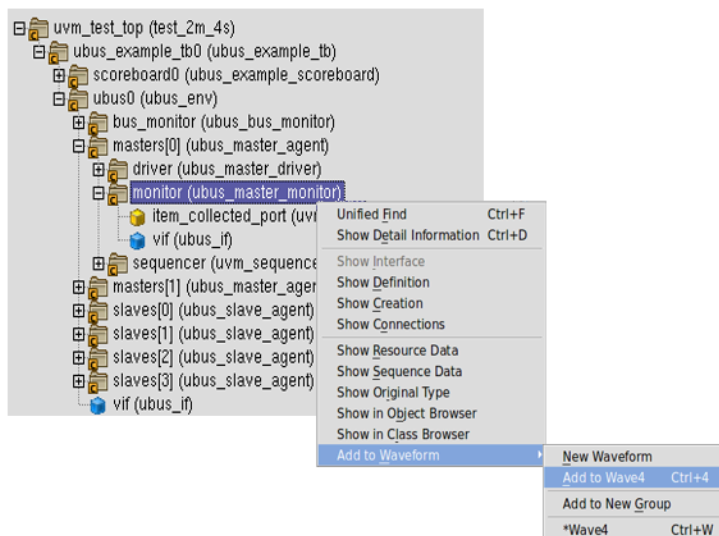
Adding a Component Object to nWave

Verdi provides a right-click menu item **Add to Waveform** that you can use to add a component object to **nWave**. To use this functionality, right-click on the component object for which you want to add the waveform to **nWave**, and select the option **Add to Waveform**. You will now be able to see the component object in the nWave window as shown in [Figure 115](#).

Note:

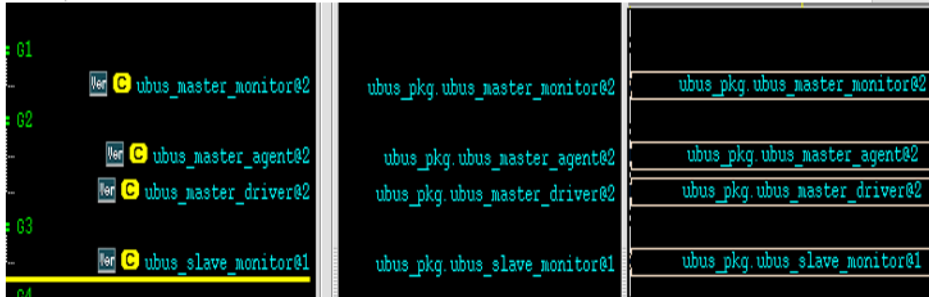
The **Add to Waveform** menu option is only enabled when the selected component object exists in FSDB.

Figure 115 Adding a Waveform to nWave



The component waveform is displayed in the **nWave** window as shown in the following figure:

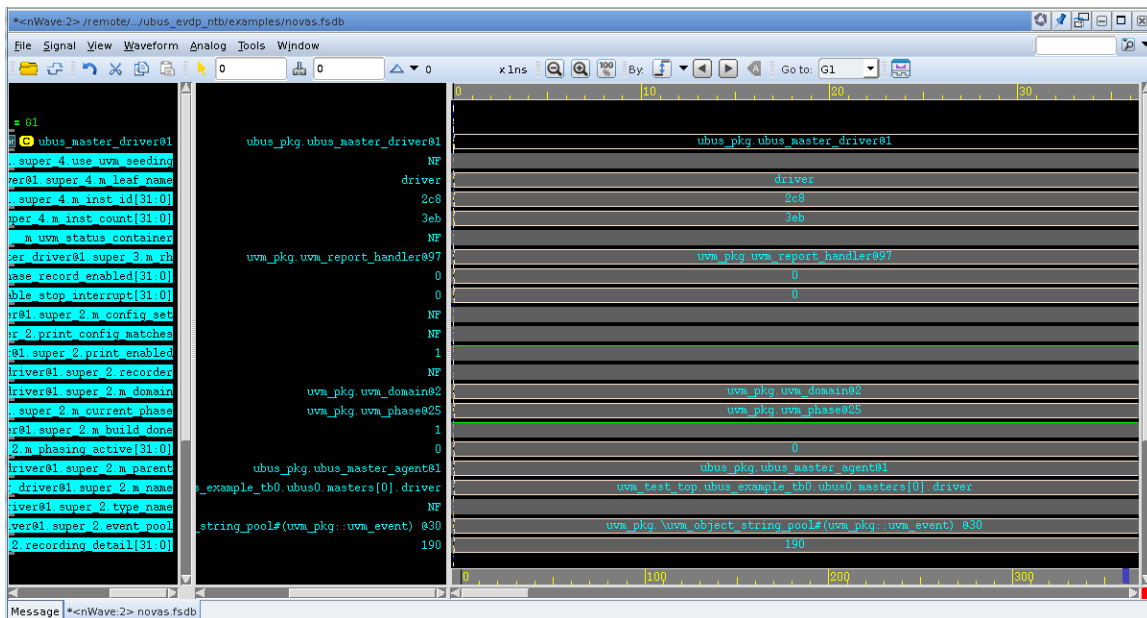
Figure 116 Component Waveform Displayed in nWave



In component waveform shown in figure [Figure 116](#). The signal name is in the format <class name>@<object id>, for example, ubus_master_driver@1. The <object id> is the internal ID from simulator indicating the ordering number of the object for the specified class.

Each component signal can be expanded to view the waveforms of each member of the component, as shown in the following figure:

Figure 117 Component Member Expanded



Limitations

This feature functions under the following limitations:

- Only VCS simulator is supported.
- Only `uvm_component` and `uvm_port_base` classes and classes extended from them are supported.
- The following data types are not supported:
 - Char var
 - Dynamic array (Including queue, associate array)
- Only one level of members of the class is dumped. For example, the members of class of `uvm_component` is not displayed in **nWave**.
- VCS Partition Compile is not supported.
- Dumping of objects in VIP encrypted classes is not supported.

5

Practice

The following practice example illustrates the following:

- How to generate the FSDB file that includes the Testbench Hierarchy Tree information, and how to load this dynamic data into the Verdi platform for viewing the hierarchy and the connection information.
- View the detailed call-stacks of this dynamic information.
- Correlate the dynamic recorded information with the source code.
- Understand the source of testbench objects generation from the source code.

The practice example uses the OVM UART example located in the `$VERDI_HOME/share/vmlib/ovm/ovm/lib/examples/uart16550` directory. It demonstrates the use of the *Testbench Hierarchy* tree to debug the connection of a port.

1. Copy the case to your local directory.

```
%> cp -r $VERDI_HOME/share/vmlib/ovm/ovm/lib/ .
```

2. Change directory to `sv_latest`.

```
%> cd lib/examples/uart16550/verif/testbench/sv_latest
```

3. Setup the environment variable for this practice.

```
%> setenv VERDI_HOME <Verdi_Install_Directory>
%> setenv OVM_HOME $VERDI_HOME/share/vmlib/ovm/ovm/lib
```

4. Generate an FSDB file with the TBH tree dumped.

- a. Setup the variable for your simulation machine:

```
%> setenv MACHINE LINUX64
```

- b. Open the `run_vcs` file, modify the `SNPSLMD_LICENSE_FILE` and `VCS_HOME` variables to match your simulation path.

- c. Add the **+sps_enable_hier_trace** option to simv:

```
./simv +sps_enable_hier_trace +fsdb+trans_begin_callstack
+OVM_TESTNAME=uart_testcase_modem_status_test -l vcssim.log
```

- d. Run VCS to dump the FSDB:

```
%> source run_vcs
```

5. Import the design with OVM/UVM and load the generated FSDB file.

```
%> verdi +OVM_TESTNAME=uart_testcase_modem_status_test
-sv +incdir+$OVM_HOME/src $OVM_HOME/src/ovm_pkg.sv
+incdir+../../rtl/verilog_lasttest
-f ../../rtl/verilog_lasttest/compile.f uart_duv_if.sv
uart_duv_top.sv -ssf novas.fsdb &
```

6. Enter the **Testbench Debug Mode**:

- In the *Welcome Page*, click the **Work Mode** icon.
- Select the **Testbench Debug Mode** option.
- Click the **Go to Work** button to enter **Testbench Debug Mode**.

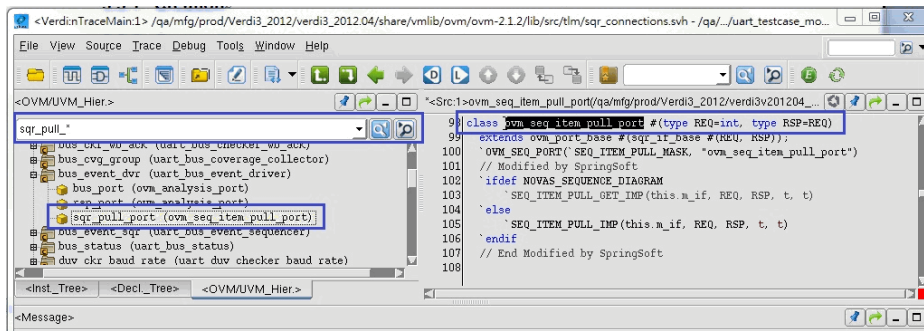
All the necessary panes open automatically to help you debug the testbench code.

7. Run the simulation for a specified amount of time (For example, 10ns), and click the

waveform icon  on the toolbar to open the *nWave* pane.

- Open the *OVM/UVM Hierarchy* view by invoking the **View -> OVM/UVM Hierarchy View** command from the main menu. The **<OVM/UVM_Hier.>** tab opens as a new tab in the same location as the design browser.
- In the **<OVM/UVM_Hier.>** tab, identify the node that represents *ovm_test_top.testbench.bus_event_dvr.sqr_pull_port*
- Double-click the identified node to view the class definition in the *Source Code* pane.

Figure 118 Select a Node and Locate Corresponding Source Code

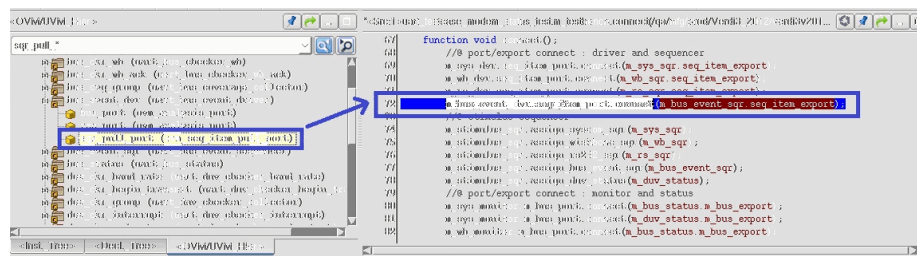


11. Verify the connection.

- a. With the node selected, select the **Show Connections** command (refer to the **Show Connections Command** section).

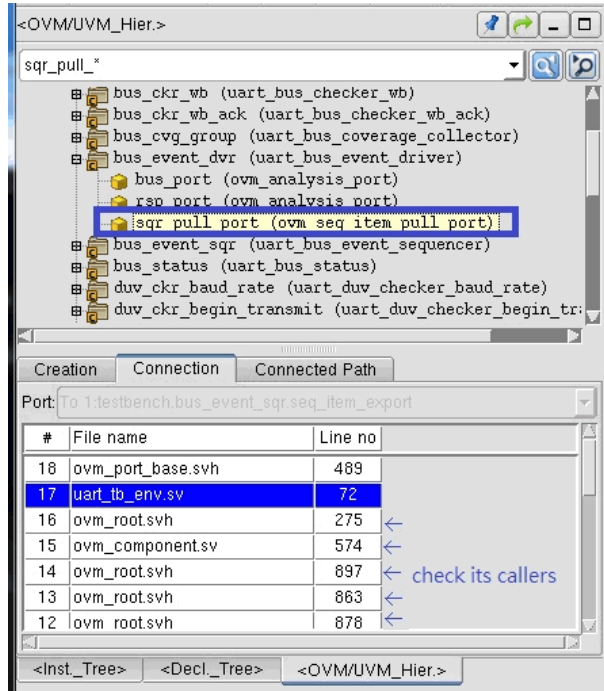
The following figure is the result of the **Show Connections** command. Assume that the connection is erroneous, that is, the **connect** function should not be called.

Figure 119 Show Connections in the Source Code Pane



- b. Select the **Show Detail Information** command from the right-click command menu to examine the call stack and identify where the SystemVerilog program flow goes wrong. The following figure shows the call-stack of the connection.

Figure 120 Connection Call Stack



6

Known Issues and Limitations

UVM Debug Limitations

The following known issues and limitations exist in the UVM Debug feature:

- Inactive sequence data (that is, data which is no longer inside the simulator) is available only when the UVM default recorder is replaced by the Verdi recorder.
- A performance issue exists when retrieving sequence data from the simulator.
- Drag-and-drop operations are not supported.
- Time synchronization with other windows is not supported.
- The transaction data unrelated to sequence data may be displayed in the *Sequence View* pane. You can add a stream attribute preventing this transaction data from being displayed in the *Sequence View* pane.
- With Verdi 2013.09, if you let the simulator run to the end of simulation, the simulator quits and the Verdi platform exits the interactive mode. At this time, simulator data is not available. This is addressed starting Verdi 2014.03 release onwards.
- The number of phases is fixed for the usage of VMM/OVM libraries.
- Although the *Sequence View* pane is supported for OVM/UVM libraries, the menu and the window is not disabled in VMM.
- For OVM recording flow, Verdi recorder may not record some parent-child relations. As a result, the relations of these events are retrieved from the simulator. However, if the simulation stops after these relations are cleared from simulator, the Verdi platform cannot retrieve these relations.
- Virtual registers and memory management models are currently not supported.
- An array of registers in the hierarchy tree is either completely expanded or completely collapsed. Partial array expansion in the hierarchy tree is not supported.

- If two defined registers are referencing the same register object, only one register is displayed on the hierarchy tree.
- If the history of configurations/resources cannot be retrieved from the FSDB, the content of the history in the *Resource History* pane does not match the content of the configuration/resource in the *Resource* pane.

UVM Register View Debug in Post-Processing Mode: Limitations

The following known issues and limitations exist in the UVM *Register View* debug in post-processing mode feature:

- The *Register View* in interactive mode is supported in VCS only. The *Register View* in post-processing mode is supported with VCS and third-party simulators.
- Since the object dumping is only supported in VCS, the **Add to Waveform** option is not supported in third-party simulators. This option is not supported without the complete register object dumping. To enable object dumping into FSDB file, you must use the `+UVM_VERDI_TRACE=RALWAVE` VCS runtime option. You must also include Verdi UVM library and recorder while compiling.
- The register memory may be big in size, so array or hierarchy view for memory items are not created. Currently, the access history of the memory item are listed, but the attribute values in any specific time for register or field data in memory type are not updated.
- The register declaration name is available only in the interactive mode.
- The **UVM icon**  -> **Register View** option in the toolbar is available in the post-processing mode if the FSDB file is loaded. However, if the design without the FSDB file is imported, then this option is not available.
- Register View does not support virtual FSDB file.
- Verdi does not support “set” and “get” actions of a register from UVM post-processing mode. Therefore, the desired and mirrored values are used only for reference.
- Register View in post-processing mode does not support time range access.
- If you write data to register field, UVM does not display the message with complete information. Therefore, you cannot view the register field action in the *Register View*.

If you turn on the UVM compiler directive `UVM_REG_NO_INDIVIDUAL_FIELD_ACCESS`, the register field action is reflected to register, so that you can view the register action in the register *Access History* pane.

Testbench Hierarchy Limitations

- For IUS, the call stack cannot be displayed. You can see only a single call in the call-stack on the **Creation** and **Connection** tabs.
- For MTI, the *Testbench Hierarchy* tree is not supported.